



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki

Filia w Gdańsku

Artem Stakhovski

Nr albumu s27297

Nazwa specjalizacji: Aplikacje Internetowe

Kacper Demczyna

Nr albumu s26979

Nazwa specjalizacji: Sztuczna Inteligencja

Paweł Anuszkiewicz

Nr albumu s26970

Nazwa specjalizacji: Cyberbezpieczeństwo

SZYSZKA- projekt i implementacja aplikacji internetowej dla zuchów

Rodzaj pracy

inżynierska

Imię i nazwisko promotora

mgr inż. Adam Urbanowicz

Gdańsk, luty, 2026

Streszczenie:

Celem pracy inżynierskiej opisanej w niniejszym dokumencie było zaprojektowanie oraz implementacja aplikacji internetowej SZYSZKA, przeznaczonej do wspierania komunikacji i organizacji w gromadach zuchów oraz zastępach harcerskich działających w ramach Związku Harcerstwa Rzeczypospolitej. Projekt powstał w odpowiedzi na problemy związane z rozproszoną komunikacją, brakiem centralnego systemu informacyjnego oraz koniecznością korzystania z ogólnych, komercyjnych platform niedostosowanych do specyfiki pracy wychowawczej z dziećmi.

W ramach pracy przeprowadzono analizę problemu, analizę konkurencji oraz zaproponowano autorskie rozwiązanie dopasowane do struktury organizacyjnej i potrzeb ZHR. Następnie zaprojektowano architekturę systemu, model bazy danych oraz interfejs użytkownika, a także zaimplementowano dostępną poprzez przeglądarkę aplikację internetową. System umożliwia zarządzanie użytkownikami o różnych rolach, komunikację poprzez forum i czaty, organizację wydarzeń oraz ewidencję sprawności zuchów, z zachowaniem zasad bezpieczeństwa informacji i ochrony danych osobowych.

Efektem pracy jest funkcjonalny system, która może realnie usprawnić komunikację i organizację w ramach struktur ZHR, wspierając przy tym wartości wychowawcze i społeczne harcerstwa.

Słowa kluczowe:

Zuch, aplikacja internetowa, komunikacja.



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: Aplikacja internetowa dla zuchów	Akronim: SZYSZKA (System Zarządzania Zuchami i Sprawnościami Kształtującymi Aktywność)	
Promotor: mgr inż. Adam Urbanowicz	Konsultanci:	1. Drużynowy Gromady zuchów 1 GGZ
Cele projektu zrobienie aplikacji internetowej która: Umożliwi zuchom monitorowanie swoich postępów oraz aktywności– każdy użytkownik będzie miał własne konto, na którym znajdzie szczegółowy zapis zdobytych sprawności oraz ich opis. Da drużynowemu narzędzie do lepszej organizacji pracy i planowania wyjazdów oraz komunikacji z rodzicami zuchów		
Rezultaty i zakres	Oczekiwane produkty/usługi	Aplikacja internetowa dla zuchów z funkcjonalnościami takimi jak: monitorowanie postępów zuchów, zarządzanie sprawnościami, postami i chatami organizowanie zbiórek i wyjazdów. System umożliwi drużynowym lepsze planowanie wyjazdów oraz kontaktowanie się z rodzicami. Udostępnienie zuchom informacji o zdobytych sprawnościach i historiach aktywności.
	Główne funkcjonalności i/albo cechy	Zarządzanie wydarzeniami, postami i sprawnościami oraz chat pomiędzy użytkownikami
Miary sukcesu: Zapewnienie funkcjonalności w ramach wymagań oraz stworzenia łatwego w obsłudze narzędzia. System będzie w pełni funkcjonalny zgodnie z założeniami projektowymi i spełni wymagania wszystkich użytkowników (drużynowy, przyboczny, zuchy, rodzice). Zadowolenie drużynowego i rodziców z możliwości łatwego zarządzania danymi zuchów oraz organizacją wyjazdów.		
Ograniczenia: Aplikacja działa jedynie w Polsce Aplikacja wymaga połączenia z internetem, co może być problemem w rejonach o słabej dostępności sieci. Czas trwania jest ograniczony, co wpłynie na zakres funkcjonalności, jakie zostaną wdrożone w pierwszej wersji aplikacji.		

Wykonawcy	Numer albumu	Specjalizacja	Tryb studiów:
Artem Stakhovski	s27297	Aplikacje internetowe	stacjonarne
Paweł Anuszkiewicz	s26970	Cyberbezpieczeństwo	stacjonarne
Kacper Demczyna	s26979	Sztuczna inteligencja	stacjonarne

Data ukończenia projektu: 23.01.2026	Recenzent: dr hab. Aleksander Denisiuk
---	---

Spis Treści

1 Wstęp.....	9
2 Słownik pojęć.....	10
3 Przedstawienie problemu.....	11
3.1 Skala zjawiska harcerstwa.....	11
3.1.1 Wprowadzenie – harcerstwo ZHR.....	11
3.1.2 Problem komunikacji w środowiskach ZHR.....	11
3.1.3 Potrzeba nowoczesnego rozwiązania.....	12
3.2 Rich Picture.....	12
3.3 Analiza konkurencji.....	13
3.3.1 Facebook.....	13
3.3.2 ScoutLink.....	14
3.3.3 Strona przykładowej drużyny harcerskiej.....	15
3.4 Propozycja rozwiązania.....	16
3.5 Analiza SWOT.....	17
4 Kontekst projektu.....	19
4.1 Kontekst systemowy.....	19
4.2 Kontekst biznesowy.....	20
4.2.1 Wprowadzenie.....	20
4.2.2 Segmentacja klienta.....	20
4.2.3 Kanały komunikacji.....	21
4.2.4 Relacje z klientami.....	21
4.2.5 Propozycja wartości.....	22
4.2.6 Strumienie przychodów.....	23
4.2.7 Kluczowe zasoby.....	23
4.2.8 Kluczowe zadania.....	24
4.2.9 Kluczowe partnerstwa.....	24
4.2.10 Struktura kosztów.....	24
4.3 Kontekst społeczny.....	25
4.3.1 Wprowadzenie.....	25
4.3.2 Pozytywne konsekwencje społeczne.....	26
4.3.3 Negatywne konsekwencje społeczne.....	27
4.3.4 Wnioski.....	27
4.4 Kontekst etyczny.....	28
4.4.1 Wprowadzenie.....	28
4.4.2 Interesariusze projektu.....	28
4.4.3 Pozytywne aspekty społeczne.....	29
4.4.4 Potencjalne ryzyka i dylematy etyczne.....	29
4.4.5 Ocena zgodności z normami etycznymi i prawnymi.....	30
4.4.6 Wnioski analizy etycznej dla projektu SZYSZKA.....	31

5 Organizacja pracy.....	33
5.1 Metodyka pracy.....	33
5.2 Podział ról i zadań.....	33
5.3 Harmonogram pracy.....	34
5.4 Infrastruktura i sposoby komunikacji.....	35
5.4.1 Backend i warstwa dostępu do danych.....	35
5.4.2 Mechanizmy bezpieczeństwa.....	36
5.4.3 Organizacja pracy zespołu i komunikacja.....	36
5.5 Analiza ryzyka.....	37
6 Wymagania.....	40
6.1 Udziałowcy.....	40
6.2 Diagram przypadków użycia.....	44
6.3 Wymagania ogólne i dziedzinowe.....	48
6.4 Wymagania funkcjonalne.....	52
6.5 Wymagania pozafunkcjonalne.....	67
6.6 Interfejs z otoczeniem.....	72
6.7 Wymagania na środowisko.....	82
7 Projektowanie.....	84
7.1 Architektura.....	84
7.1.1 Ogólny opis architektury.....	84
7.1.2 Diagram architektury.....	84
7.2 Projekt bazy danych.....	87
7.2.1 Cel i rola bazy danych.....	87
7.2.2 System zarządzania bazą danych.....	89
7.2.3 Model danych.....	89
7.2.4 Występujące relacje między tabelami.....	90
7.2.5 Przechowywanie plików multimedialnych.....	90
7.2.6 Bezpieczeństwo danych.....	90
7.3 Projekt wizualny systemu.....	91
7.3.1 Paski nawigacyjne.....	91
7.3.2 Prototyp. Strona główna.....	92
7.3.3 Prototyp. Strona logowania.....	92
7.3.4 Prototyp. Strona wydarzeń.....	93
7.3.5 Prototyp. Strona sprawności.....	94
7.3.6 Prototyp. Strona forum.....	94
7.3.7 Prototyp. Strona czaty.....	95
7.3.8 Prototyp. Strona profilu.....	96
8 Implementacja.....	97
8.1 Architektura.....	97
8.2 Implementacja frontendu.....	100
8.2.1 Struktura plików.....	100

8.2.2 Wykorzystane biblioteki.....	101
8.2.3 Folder app.....	102
8.2.4 Layout.....	103
8.2.5 Hooki.....	104
8.2.6 Komunikacja z backendem.....	105
8.2.7 Pasek nawigacyjny.....	106
8.2.8 Strona główna.....	109
8.2.9 Logowanie.....	110
8.2.10 Forum.....	114
8.2.11 Czaty.....	115
8.2.12 Wydarzenia.....	117
8.2.13 Responsywność.....	119
8.3 Implementacja backendu.....	120
8.3.1 Struktura.....	121
8.3.2 Budowa aplikacji.....	129
8.3.3 Mechanizm bezpieczeństwa.....	136
9 Testowanie.....	146
9.1 Testy jednostkowe.....	146
9.2 Testy integracyjne.....	147
9.3 Analiza pokrycia kodu testami.....	147
9.4 Interpretacja Wyników.....	148
10 Prezentacja rozwiązania.....	149
10.1 Strony dostępne bez logowania.....	149
10.1.1 Strona główna.....	149
10.1.2 Strony logowania i rejestracji.....	150
10.1.3 Strona sprawności.....	152
10.1.4 Strona wydarzeń.....	154
10.2 Strony dostępne do zalogowanego użytkownika.....	154
10.2.1 Strona profilu.....	154
10.3 Strony dostępne dla zucha.....	155
10.3.1 Strona postów.....	155
10.3.2 Strona czatów.....	156
10.4 Strony dostępne dla rodzica.....	157
10.4.1 Obserwowanie danych dzieci.....	157
10.5 Strony dostępne dla drużynowego/przybocznego.....	158
10.5.1 Strona administracyjna.....	158
10.5.2 Strona szóstek.....	159
10.5.3 Edycja i dodawanie danych.....	160
11 Wkład własny.....	161
11.1 Artem Stakhovski wkład pracy.....	161
11.2 Paweł Anuszkiewicz wkład pracy.....	162

11.3 Kacper Demczyna wkład pracy.....	163
12 Podsumowanie.....	164
12.1 Osiągnięte rezultaty.....	164
12.2 Plany na przyszłość.....	164
12.2.1 Przechowywanie zdjęć.....	164
12.2.2 Publikacja.....	165
13 Spis tabel.....	166
14 Spis rysunków.....	168
15 Załączniki.....	171

Rozdział 1

Wstęp

Przedmiotem pracy jest projekt i implementacja aplikacji webowej, która ma za zadanie wspierać funkcjonowanie 1 Gdyńskiej gromady zuchów działającej w ramach Związku harcerstwa Rzeczypospolitej. Przyczyną realizacji projektu był brak stosownego narzędzia dla drużynowego gromady, które pozwoliłoby mu na łatwe zarządzanie gromadą i udostępnianie informacji członkom gromady oraz ich rodzicom a także wspólnego narzędzia pozwalającego na skuteczną i łatwą komunikację pomiędzy drużynowym, przybocznym, rodzicami oraz zuchami. Informacje organizacyjne były do tej pory przekazywane za pośrednictwem wielu różnych kanałów komunikacji, co utrudniało drużynowemu zarządzanie gromadą oraz archiwizację informacji. Przedstawiony w pracy system umożliwia zarządzanie informacjami gromady w tym obsługę wydarzeń, zarządzanie użytkownikami, komunikację między członkami poprzez czaty oraz forum, oraz wgląd do listy sprawności a także możliwość śledzenia swoich postępów przez zuchy i ich rodziców. Aplikacja została zaprojektowana w formie klient-serwer, z wydzielonymi warstwami - backendową oraz frontendową z uwzględnieniem zasad bezpieczeństwa, skalowalności oraz czytelnej architektury. Projekt został zrealizowany jako grupowa praca inżynierska w ramach studiów na kierunku informatyka w Polsko-Japońskiej Akademii Technik Komputerowych w Gdańsku. Założeniem tej pracy nie było tylko rozwiązanie rzeczywistego problemu organizacyjnego ale także praktyczne zastosowanie wiedzy zdobytej podczas studiów z zakresu projektowania systemów informatycznych, aplikacji webowych i zabezpieczania dostępu do danych.

Rozdział 2

Słownik pojęć

1. **ZHR** - Związek Harcerstwa Rzeczypospolitej – organizacja harcerska powołana 12 lutego 1989.
2. **Discord** - Bezpłatna usługa internetowa oparta na chmurze pozwalająca na rozmowy głosowe i wysyłanie wiadomości tekstowych.
3. **Harcerstwo** - Polski ruch społeczny i wychowawczo pedagogiczny, będący częścią ruchu skautowego.
4. **Zuch** - członek gromady zuchowej w wieku od 7 do 12 lat.
5. **Drużynowy** - zarządzający gromadą zuchową lub drużyną harcerską.
6. **Przyboczny** - osoba pomagająca drużynowemu w zarządzaniu gromadą.
7. **Analiza SWOT** - Narzędzie służące do analizy przedsięwzięć, jego nazwa jest skrótem od następujących kategorii przez nie opisywanych: S(Strengths – mocne strony), W(Weaknesses - słabe strony), O(Opportunities - Szanse), T(Threats - zagrożenia).
8. **Figma** - Program służący do projektowania aplikacji który pozwala na tworzenie interaktywnych prototypów.
9. **REST** - Representational state transfer – Styl Architektury oprogramowania używany przy komunikacji za pomocą protokołu HTTP..
10. **Frontend** - warstwa aplikacji odpowiedzialna za interfejs użytkownika oraz interakcję z nim. Obejmuje elementy widoczne w przeglądarce, takie jak układ strony, formularze oraz obsługę zdarzeń użytkownika.
11. **Backend** - część aplikacji odpowiedzialna za logikę biznesową, przetwarzanie danych oraz komunikację z bazą danych. Backend obsługuje żądania wysyłane przez frontend i zwraca odpowiednie odpowiedzi.
12. **HTTP** - Hypertext Transfer Protocol - protokół komunikacyjny wykorzystywany do przesyłania danych w sieci Internet pomiędzy klientem (np. przeglądarką) a serwerem.

Rozdział 3

Przedstawienie problemu

3.1 Skala zjawiska harcerstwa

3.1.1 Wprowadzenie – harcerstwo ZHR

Związek Harcerstwa Rzeczypospolitej (ZHR) jest jednym z głównych nurtów polskiego harcerstwa, kontynuuje tradycję przedwojennego Związku Harcerstwa Polskiego, odwołując się do wartości patriotyzmu, służby i wychowania chrześcijańskiego. Organizacja ZHR powstała w 1989 roku i opiera się na zasadzie wychowania przez działanie, samodzielność i służbę bliźniemu.

Według danych opublikowanych w profilu organizacyjnym ZHR w międzynarodowej bazie organizacji pozarządowych liczebność organizacji szacowana jest na 17500 członków [1]. Organizacja dzieli się na dwa piony – męski i żeński – prowadząc działalność w drużynach i gromadach w hufcach na terenie całej Polski.

ZHR kładzie nacisk na wychowanie przy zastosowaniu systemu stopni, sprawności i służby, które rozwijają kompetencje społeczne, charakter i postawę obywatelską. Celem organizacji jest nie tylko organizacja czasu wolnego, ale również kształtowanie dojrzałych postaw poprzez współpracę w grupach takich jak zastępy i drużyny. Zmiany społeczne, rozwój technologiczny i wzrost tempa życia powodują, że prowadzenie pracy wychowawczej wymaga coraz lepszej organizacji i komunikacji.

3.1.2 Problem komunikacji w środowiskach ZHR

Podstawową jednostką w ZHR są niewielkie grupy, takie jak zastępy, szóstki oraz drużyny, w których realizowane są bieżące działania organizacyjne i programowe. W praktyce funkcjonowania tych jednostek znaczna część komunikacji oraz zarządzania informacjami odbywa się na poziomie lokalnym. Brak spójnych i dedykowanych narzędzi

komunikacyjnych na poziomie podstawowych jednostek prowadzi do utrudnień organizacyjnych oraz zwiększa obciążenie administracyjne kadry.

Badania sektora organizacji pozarządowych wskazują, że stosowanie wielu równoległych narzędzi komunikacyjnych takich jak media społecznościowe, poczta elektroniczna lub kontakt telefoniczny, utrudnia skuteczny przepływ informacji, co negatywnie wpływa na przejrzystość działań organizacji [2].

Dodatkowym problemem jest brak centralnego systemu informacyjnego, który umożliwiłby bezpieczną i uporządkowaną wymianę informacji, prowadzenie ewidencji aktywności uczestników oraz komunikację z rodzicami w sposób zgodny z obowiązującymi przepisami o ochronie danych osobowych [3].

3.1.3 Potrzeba nowoczesnego rozwiązania

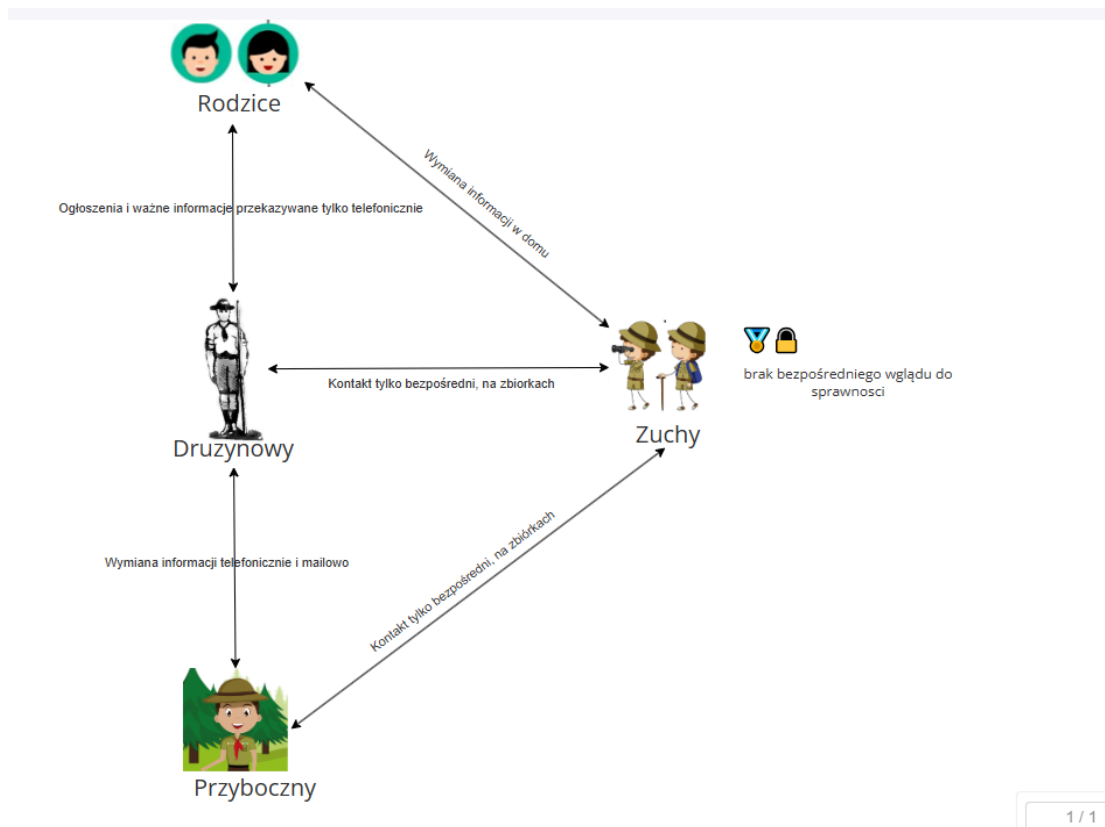
W odpowiedzi na opisane problemy pojawia się potrzeba opracowania zintegrowanego rozwiązania informatycznego, które łączyłoby funkcje komunikacyjne i organizacyjne w ramach jednego systemu, dostosowanego do struktury i specyfiki działalności ZHR.

Literatura dotycząca cyfryzacji organizacji pozarządowych wskazuje, że wdrożenie dedykowanych systemów informacyjnych pozwala na ułatwienie komunikacji, zmniejszenie obciążenia administracyjnego oraz poprawę przejrzystości zarządzania danymi i procesami organizacyjnymi [2].

System taki powinien zapewniać prywatną przestrzeń komunikacyjną dla drużyn, wspierać ewidencję informacji użytkowników oraz umożliwiać bezpieczny kontakt między poszczególnymi użytkownikami systemu [3].

3.2 Rich Picture

Na rich picture widzimy z jakimi problemami obecnie zmagają się członkowie gromady zuchów oraz rodzice zuchów. Problem leży w komunikacji, nie ma jednego centralnego systemu który pozwala na łatwą i szybką komunikację. Większość informacji jest wymieniana drogą telefoniczną bądź na cotygodniowych zbiórkach co pokazuje jak bardzo potrzebna jest strona internetowa ułatwiająca wymianę informacji oraz zarządzanie gromadą.



Rysunek 3.1 Rich Picture. Źródło: Facebook Groups

3.3 Analiza konkurencji

Celem analizy jest zbadanie istniejących już systemów o podobnym zakresie funkcjonalnym, które mogłyby być porównywalne z projektem SZYSZKA. Ocenie poddaliśmy aplikacje umożliwiające komunikację, zarządzanie grupą użytkowników, publikowanie treści i organizowanie wydarzeń, analizowaliśmy nie tylko systemy przeznaczone dla środowisk harcerzy ale także ogólne platformy społecznościowe.

Przegląd rozwiązań:

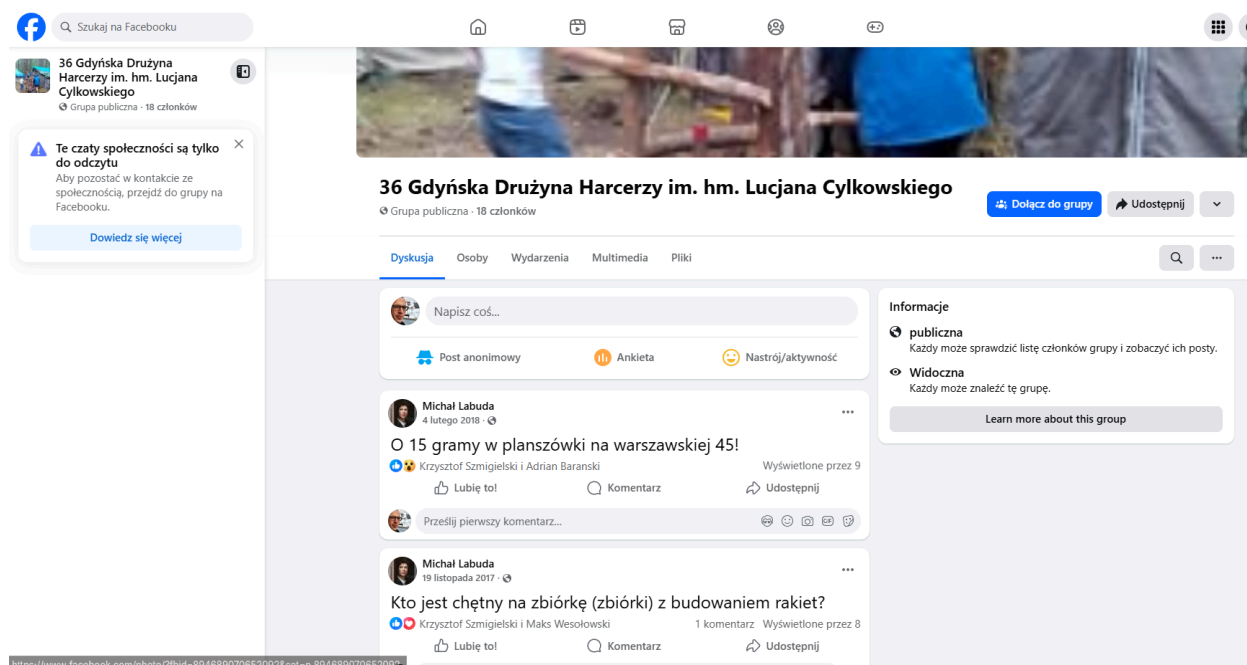
3.3.1 Facebook

Grupy na Facebooku - facebook pozwala na tworzenie grup użytkowników ,

na których można publikować treści, tworzyć wydarzenia i pozwala na komunikację między członkami grupy

Zalety: Duża popularność platformy facebook, dostępność na różnych urządzeniach, intuicyjny interfejs użytkownika.

Wady: Brak dedykowanych funkcji pozwalających na zarządzanie gromadą zuchów, wymaga rejestracji na platformie facebook co ogranicza prywatność, bardzo ogólny charakter - grupy na facebooku wszystkie wyglądają tak samo i różnią się tylko tym jacy użytkownicy się na danej grupie znajdują i jaka jest tematyka grupy.



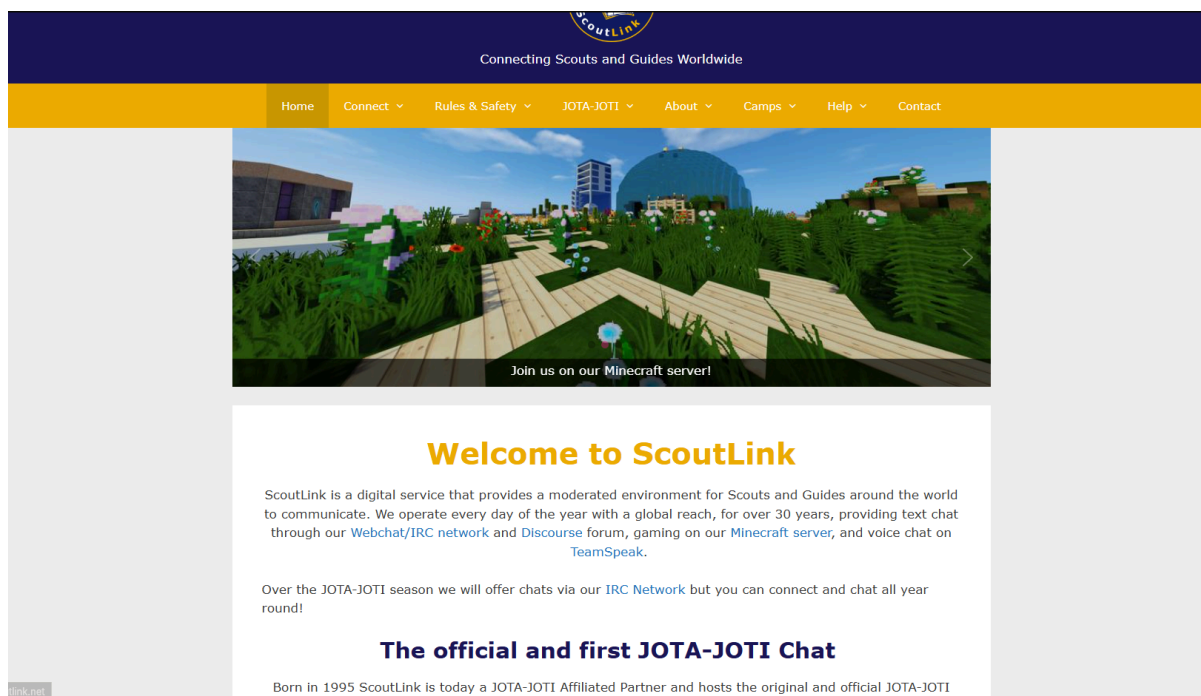
Rysunek 3.2 Konkurencja-Facebook. Źródło: Facebook Groups

3.3.2 ScoutLink

ScoutLink - międzynarodowa platforma, która umożliwia komunikację między harcerzami na całym świecie.

Zalety: Proste narzędzie do komunikacji, społeczność w pełni harcerska, możliwość komunikacji z harcerzami z innych krajów.

Wady: Strona nie oferuje narzędzi do zarządzania gromadą zuchów bądź drużyną harcerską. Interfejs może być zbyt ciężki do opanowania poprzez młodszych użytkowników.



Rysunek 3.3 Konkurencja-ScoutLink. Źródło - <https://www.scoutlink.net>

3.3.3 Strona przykładowej drużyny harcerskiej

Niektóre drużyny mają statyczne strony internetowe

Zalety: Możliwość prezentacji podstawowych informacji o drużynie (galeria, aktualności, kontakt). Łatwość konfiguracji i szeroki wybór szablonów. Często dostosowane wizualnie do charakteru drużyny.

Wady: Strony są statyczne – brak mechanizmów logowania i pracy z danymi członków. Brak komunikatora i możliwości prowadzenia czatów 1:1 lub grupowych. Brak funkcji zarządzania gromadą, sprawnościami, obecnościami, dokumentami czy wydarzeniami.

o gromadzie

uaktualniono 2025-10-15

1. Gdyńska Gromada Zuchów „Rycerze z Kamiennej Góry” jest jednostką organizacyjną [Związku Harcerstwa Rzeczypospolitej](#). Gromada działa od 11 października 2008 r.

Należy do [Gdyńskiego Hufca Harcerzy „Pasieka”](#). Do gromady należą chłopcy w wieku od 7 do 11 lat.

Każdego roku na przełomie września i października przyjmujemy kilku nowych kandydatów na zuchów. Najstarszym i najdzielniejszym, po ukończeniu 4 klasy szkoły podstawowej proponujemy wstąpienie do drużyny harcerzy. Nasze zuchy najczęściej wstępują do [36. Gdyńskiej Drużyny Harcerzy](#). Naszym zuchowym sąsiadem jest [3. Gdyńska Gromada Zuchów „Witomici”](#).

Rysunek 3.4 Konkurencja-Strona poszczególniej drużyny harcerskiej

3.4 Propozycja rozwiązania

Projekt SZYSZKA powstał jako odpowiedź na powyższe problemy w środowisku ZHR. Celem projektu jest stworzenie zintegrowanego systemu informacyjno-komunikacyjnego, który pozwoli drużynowym i instruktorom na skuteczne zarządzanie informacjami o zbiórkach, obecnościach i wydarzeniach oraz zapewni sprawną komunikację wewnątrz drużyny.

SZYSZKA łączy w sobie:

- funkcje portalu społecznościowego (posty, tablica aktualności, czaty),
- funkcje ewidencyjne (frekwencja, statystyki aktywności, sprawności),
- funkcje organizacyjne (informacje o wydarzeniach, listy uczestników).

Dzięki temu użytkownicy – harcerze, zuchy, instruktorzy oraz rodzice – zyskują jedno bezpieczne narzędzie dopasowane do realiów ZHR. Projekt ten wspiera organizację pracy oraz pomaga budować wspólnotę i więzi wewnątrz środowiska harcerskiego, zgodnie z jego ideami.

3.5 Analiza SWOT

Zdecydowaliśmy się użyć analizę SWOT, która pozwala na zidentyfikowanie mocnych stron i słabych stron oraz potencjalne szanse i zagrożenia aplikacji

Mocne strony(Strengths)	Słabe strony(Weaknesses)
• jeszcze nie ma takich rozwiązań na rynku	• korzystanie z aplikacji potrzebuje ciągłego połączenia do Internetu • ograniczona grupa docelowa
Szanse(Opportunities)	Zagrożenia(Threats)
• Możliwość rozwinięcia projektu do całej organizacji ZHR • Digitalizacja harcerstwa	• szansa niskiej adopcji naszej aplikacji przez użytkowników • problemy prawne związane z ochroną danych

Tabela 3.1 Analiza SWOT. Źródło: opracowanie własne

Analiza SWOT pokazuje, że do mocnych stron naszej aplikacji to, że jeszcze nie ma takich aplikacji na rynku co może zachęcić nowych klientów, szukających konkretnego rozwiązania.

Do słabych stron można odnieść ograniczoną grupę docelową, co powoduje brak klientów poza tej grupy. Słabą stroną jest również to, że aplikacja potrzebuje ciągłego połączenia do internetu, powodując zmniejszenie przypadków użycia.

Szansami są digitalizacja harcerstwa i możliwość rozwinięcia projektu do całej organizacji ZHR, co oznacza że jest przestrzeń do rozwoju aplikacji.

Największym zagrożeniem jest to, że aplikacja chroni dane osobowe, co może powodować problemy prawne.

Podsumowując, analiza SWOT pokazała, że główne nasza aplikacja polega na unikatowości rozwiązania, ale potrzebowanie ciągłego połączenia do Internetu może zniechęcić część

klientów. Również aplikacja ma niemały potencjał do rozwoju, ale trzeba dobrze zabezpieczyć się przed wyciekiem danych osobowych.

Rozdział 4

Kontekst projektu

4.1 Kontekst systemowy

System, który jest przez nas projektowany to aplikacja internetowa mająca na celu wspieranie zarządzania gromadą zuchową i stworzenie miejsca do dzielenia się swoimi myślami czy postami w formie forum. System będzie działał jako platforma webowa dostępna przez przeglądarkę, przeznaczona dla różnych kategorii użytkownika takich jak: Zuch, Rodzic zucha, przyboczny gromady oraz drużynowy(drużynowy będzie pełnił rolę administratora).

Aplikacja ma za zadanie pomagać w procesach organizacyjnych w gromadzie takich jak nadawanie sprawności zuchom, rejestrowanie wszystkich wydarzeń, które się odbywają oraz ułatwi komunikację między Zuchami i ich rodzicami a drużynowym i przybocznymi oraz gromadzenie danych kontaktowych.

System nie wymaga integracji z zewnętrznymi usługami informatycznymi ale wykorzystuje standardowe narzędzia takie jak biblioteki frontendowe, serwer aplikacji java oraz bazy danych SQL. Dane będą przechowywane w jednej współdzielonej bazie danych w której będą się mieściły informacje o użytkownikach, sprawnościach, wydarzeniach oraz postach.

System będzie wykorzystywał standardowe wzorce projektowe i dobre praktyki programowania , takie jak architektura warstwowa oraz kontrola dostępu oparta na rolach co pozwoli na podzielenie uprawnień między użytkownikami.

- **Zuch** będzie miał dostęp do swoich danych oraz podstawowych danych innych użytkowników, wgląd do zdobytych sprawności oraz wymagań na pozostałe, historię wydarzeń oraz przyszłe wydarzenia, możliwość wstawiania postów na forum gromady oraz kontakt poprzez czat z innymi użytkownikami.
- **Rodzic** - wgląd w dane dziecka, wydarzenia oraz komunikaty, czat z drużynowym
- **Przyboczny** - dostęp do danych zuchów oraz rodziców bez możliwości ich edycji, możliwość moderacji forum.
- **Drużynowy** - wszystkie prawa administracyjne, zarządzanie użytkownikami, wydarzeniami i nadawaniem sprawności.

Zakładania liczba użytkowników systemu to 1 Drużynowy, 1-4 przybocznych, około 30 zuchów (nowe zuchy mogą dołączyć do gromady, więc ich liczba się będzie zwiększała ale też starsze zuchy będą odchodziły z gromady, więc będzie można usunąć ich konta).

System nie ma konkurencyjnych rozwiązań dedykowanych zarządzania gromadą zuchów, istnieją jednak rozwiązania bardziej ogólne (np. aplikacje dla uczniów szkół pozwalające na zarządzanie wydarzeniami, czy grupy na platformie facebook oraz strony innych gromad zuchowych/drużyn harcerskich). Są to jednak rozwiązania ogólne, i ich wadą jest to, że brakuje im dostosowania do pracy z zuchami (większość stron internetowych drużyn oferuje jedynie informacje o drużynie i byłych wydarzeniach oraz kontakt do drużynowego). Projektowana aplikacja będzie lepiej dopasowana do potrzeb gromady zuchów.

Większość funkcjonalności systemu ma wspierać prace drużynowego i przybocznych ale także pozwoli on rodzicom i zuchom na łatwiejszy dostęp do informacji dotyczących ich postępów oraz aktywności. System usprawni organizację pracy gromady oraz poprawi komunikację i zmniejszy ilość ręcznie wykonywanych zadań.

4.2 Kontekst biznesowy

4.2.1 Wprowadzenie

Do analizy kontekstu biznesowego zdecydowano się na wykorzystanie narzędzia Business Model Canvas, które umożliwia przejrzyste przedstawienie kluczowych elementów funkcjonowania systemu. Model Business Model Canvas składa się z dziewięciu obszarów: segmentów klientów, kanałów komunikacji, relacji z klientami, propozycji wartości, strumieni przychodów, kluczowych zasobów, kluczowych działań, kluczowych partnerstw oraz struktury kosztów.

Analiza poszczególnych elementów pozwala na lepsze zrozumienie mechanizmów generowania kosztów i przychodów aplikacji, a także identyfikację grup użytkowników oraz motywacji stojących za korzystaniem z systemu.

4.2.2 Segmentacja klienta

Podczas analizy klienty zostały podzielone na 3 grupy:

Lokalizacja: Trójmiasto i okolice

Klient 1:

Dzieci w wieku od 6-12 lat który są harcerzami

Ta aplikacja pomoże im w śledzeniu swoich postępów oraz w komunikacji z innymi zuchami lub drużynowym. Możliwość udziału i aktywności w społeczności harcerskiej w formie postowania na forum.

Klient 2:

Rodzice harcerzy

Aplikacja pozwoli rodzicom śledzić aktywność i postępy dzieci oraz brać aktywny udział w życiu gromady. Ułatwiony i lepiej zorganizowany kontakt z drużynowym i kadrą drużyny.

Klient 3:

Drużynowi oraz przyboczni

Pozwoli drużynowemu łatwiej operować na danych gromady i śledzić postępy każdego zucha z poziomu aplikacji. Ułatwi i przyspieszy kontakt z rodzicami zuchów.

4.2.3 Kanały komunikacji

Kanał własny pośredni - mailowy kontakt z przedstawicielami firmy.

Kanał własny pośredni - kontakt za pomocą czata w stworzonej aplikacji.

Kanał własny bezpośredni - możliwy też spotkanie na żywo

4.2.4 Relacje z klientami**1. Wszyscy klienci:**

- a. **Usługi za zautomatyzowane:** Różne funkcjonalności aplikacji dostępne zależnie od tego jaki Klient z niej korzysta. Np. Drużynowi/Przyboczni będą posiadali prawa administratorskie.
- b. **Społeczności/Współtworzenie:** Wszyscy użytkownicy aplikacji będą wspólnie tworzyli jedną społeczność poprzez wstawianie postów na forum, komunikację między sobą poprzez czat oraz możliwością publikacji swoich postępów.

2. Wyróżniające dla grupy klientów

a. Rodzice i zuchy:

- i. **Samoobsługa:** Klient posiada narzędzia, które pozwalają tworzyć nowe posty oraz kontaktować się poprzez czat z innymi klientami.
- ii. **Współtworzenie:** użytkownicy mogą tworzyć swoje posty które potem będą widoczne dla pozostałych użytkowników

b. Drużynowi oraz przyboczni (administratorzy):

- i. **Doradztwo personalne:** Przed wdrożeniem produktu przedstawienie wszystkich funkcji oraz omówienie potencjalnych problemów.
- ii. **Dedykowana pomoc osobista:** W razie jakichkolwiek pytań klient kontaktuje się z przedstawicielem firmy mailowo i zależnie od skali problemu ustalamy spotkanie lub rozwiązujemy dany problem bez potrzeby spotkania.
- iii. **Samoobsługa:** Klient posiada narzędzia, które pozwalają mu moderować treści udostępniane na forum, tworzyć nowe wydarzenia, publikować informacje oraz zmieniać lub dodawać sprawności.
- iv. **Współtworzenie:** Drużynowy zarządza tym co będzie się pojawiało na stronie.

4.2.5 Propozycja wartości

• Dostosowanie do klienta

Aplikacja została zaprojektowana specjalnie dla środowiska harcerskiego, nie jest to uniwersalny komunikator czy platforma edukacyjna, ale narzędzie wspierające realne życie gromady.

System komunikacji dopasowany do struktury gromady (Zuch, Przyboczny, Drużynowy, Rodzic), bezpieczny dla dzieci i moderowany.

Dostosowanie emocjonalne i wartościowe - Aplikacja wzmacnia poczucie wspólnoty i współpracy między członkami gromady i pozwala na dzielenie się swoimi doświadczeniami z pozostałymi użytkownikami.

- **Wygoda i użyteczność**

- Dla zuchów: aplikacja dozwolona łatwiejsze śledzenie swoich postępów oraz komunikację z innymi zuchami
- Dla rodziców: aplikacja udostępnia czat z drużynowym i śledzenie postępów dzieci
- Dla drużynowego: wszystkie elementy gromady będą umieszczone w jednym miejscu pozwoli to na łatwiejsze zarządzanie gromadą i udostępnianie ważnych informacji członkom gromady.

4.2.6 Strumienie przychodów

- Aplikacja ma być sprzedawana z kodem źródłowym, dokumentacją techniczną oraz bazą danych w ramach sprzedaży praw własności gromadzie ZHR.
- Dodatkowe przychody możliwe z usług wdrożenia, personalizacji, szkoleń po sprzedaży i zrobienia dodatkowych funkcji do aplikacji.

4.2.7 Kluczowe zasoby

- **Materialne**

- serwer do uruchomienia aplikacji
- komputery osobiste

- **Ludzkie**

- zespół projektowy

- **Intelektualne**

- Znajomość języków programowania
- dokumentacja

- **finansowe**

- środki własne

4.2.8 Kluczowe zadania

1. Badania i rozwój:
 - a. Możliwość napisania email przez użytkowników aplikacji z uwagami do naszego produktu. Po analizie przewidziana jest możliwość modyfikacji lub dodania niektórych funkcji
2. Produkcja:
 - a. Zarządzanie repozytorium kodu i kontrola wersji
 - b. Utrzymanie serwerów
 - c. Przyrostowa metodyka wytwarzania produktu.
3. Marketing:.
 - a. Stworzenie wersji prezentacyjnej produktu DEMO
 - b. Planujemy sprzedać nasz produkt organizacji mającej gotową bazę klientów.
4. Sprzedaż i obsługa klienta
 - a. Wsparcie przy wdrożeniu aplikacji po sprzedaży
 - b. Szkolenie zespołu obsługującego aplikację po stronie klienta.

4.2.9 Kluczowe partnerstwa

Kluczowe partnerstwa:

- Związek Harcerstwa Rzeczypospolitej ZHR

4.2.10 Struktura kosztów

Koszty stałe:

- Opłata za domenę w której będzie widoczna aplikacja
- Certyfikaty SSL w celu zabezpieczenia transmisji danych
- Koszty monitoringu aplikacji pod kątem błędów lub niepoprawnego działania.
- księgowość

Koszty zmienne:

- Koszty związane z utrzymaniem aplikacji będą stopniowo rosnąć, ponieważ wraz z upływem czasu konieczne będzie zabezpieczenie coraz większej ilości danych. W efekcie po pewnym czasie może pojawić się potrzeba zwiększenia pojemności wynajmowanego serwera.
- Koszty naprawy błędów znalezionych podczas pracy aplikacji.

4.3 Kontekst społeczny

4.3.1 Wprowadzenie

Projekt SZYSZKA, jako wewnętrzna sieć społecznościowa przeznaczona dla organizacji harcerskiej ZHR, funkcjonuje nie tylko jako narzędzie technologiczne, lecz także jako przestrzeń społecznych interakcji. System wpływa na sposób komunikacji, integracji i budowania wspólnoty wśród dzieci, młodzieży, instruktorów oraz rodziców.

Kontekst społeczny obejmuje analizę, w jaki sposób platforma zmienia relacje między użytkownikami, jakie tworzy możliwości, a jakie ograniczenia. W odróżnieniu od innych platform do komunikacji, SZYSZKA stanowi zamkniętą, moderowaną i wychowawczą przestrzeń, co bezpośrednio wpływa na dynamikę grup, zasady współpracy oraz poziom bezpieczeństwa.

Celem niniejszej analizy jest określenie społecznych konsekwencji wdrożenia systemu — zarówno pozytywnych, jak i potencjalnie problematycznych — oraz wskazanie, jak platforma może wspierać rozwój wspólnoty harcerskiej.

4.3.2 Pozytywne konsekwencje społeczne

- **Ułatwienie koordynacji działań:** System pozwala na organizację wydarzeń, zarządzanie sprawnościami, tworzenie postów oraz czat, co pozwala na przechowywanie wszystkich informacji w jednym miejscu
- **Wzmacnianie zaangażowania w działalność gromady:** członkowie drużyny mogą w dowolnym czasie komunikować z innymi członkami lub dzielić się swoimi sukcesami, co powoduje większe zaangażowanie w działalność gromady
- **Rozwój kompetencji społecznych i cyfrowych:** aktywne uczestnictwo w tworzeniu postów oraz czatach będzie rozwijać umiejętności w tych czynnościach
- **Bezpieczniejsze środowisko społecznościowe:** moderowane środowisko minimalizuje ryzyko napotkania nieodpowiednich treści przez dzieci
- **Wzmocnienie zaufania i wspólnoty:** przejrzystość działania oraz jasno określone zasady korzystania z platformy budują zaufanie między kadrą, rodzicami i harcerzami. Projekt wspiera integrację środowiska wychowawczego oraz umacnia więzi wewnątrz organizacji.
- **Zgodność z misją wychowawczą ZHR:** system wspiera realizację wartości ZHR, takich jak służba, odpowiedzialność i współdziałanie. Poprzez świadome wykorzystanie nowoczesnych technologii projekt promuje etyczne i odpowiedzialne postawy w cyfrowym świecie.

4.3.3 Negatywne konsekwencje społeczne

- **Wykluczenie uczestników o ograniczonym dostępie do Internetu:** uczestnicy którzy mają ograniczony dostęp do Internetu nie zможą aktywne korzystanie z aplikacji, co może powodować, że inne użytkownicy nie będą włączać ich w swoje zespoły
- **Zmniejszenie komunikacji na żywo:** częstsza komunikacja online będzie powodować zmniejszenie komunikacji na żywo i nie otrzymanie potrzebnych umiejętności społecznych
- **Problemy z moderacją:** niewystarczające umiejętności moderatorów mogą powodować umieszczenie nieodpowiednich treści lub nie zauważone problemy społeczny

4.3.4 Wnioski

Analiza społeczna wskazuje, że system SZYSZKA może mieć znaczący wpływ na funkcjonowanie wspólnoty harcerskiej. Może wzmacniać zaangażowanie, ułatwiać współpracę i budować poczucie bezpieczeństwa, o ile będzie stosowany w sposób zgodny z zasadami wychowawczymi ZHR.

Najważniejsze wnioski z analizy kontekstu społecznego:

- Wzmacnianie zaangażowania w działalność gromady
- Wzmacnianie wspólnoty
- Ułatwienie koordynacji działań
- Bezpieczniejsze środowisko społecznościowe
- Bardzo ważne jest żeby przy wykorzystaniu aplikacji nie zniknęła komunikacja na żywo

4.4 Kontekst etyczny

4.4.1 Wprowadzenie

Projekt SZYSZKA stanowi system informacyjno-komunikacyjny przeznaczony dla organizacji harcerskiej ZHR. Ze względu na specyfikę użytkowników - dzieci, młodzież, instruktorów oraz rodziców - wymaga on szczególnej analizy etycznej.

Kontekst etyczny obejmuje nie tylko przestrzeganie prawa (w tym RODO), ale również zgodność z wartościami wychowawczymi ZHR oraz zasadami odpowiedzialnego wykorzystania technologii. Należy także uwzględnić młody wiek użytkowników i wynikające z tego wymogi bezpieczeństwa.

Celem tej analizy jest określenie, w jaki sposób system SZYSZKA wspiera etyczne aspekty komunikacji i organizacji pracy w drużynach oraz jakie potencjalne ryzyka mogą pojawić się podczas jego wdrażania.

4.4.2 Interesariusze projektu

Analiza etyczna wymaga identyfikacji najważniejszych grup, na które projekt może oddziaływać.

W przypadku systemu SZYSZKA do kluczowych interesariuszy należą:

- Zuchy i harcerze – największa grupa użytkowników systemu, których dane osobowe oraz aktywność są przetwarzane. Potrzebują oni również dostępu do swoich danych oraz komunikacji z innymi użytkownikami.
- Rodzice i opiekunowie – zainteresowani dostępem do informacji o obecności, wydarzeniach i postępach wychowawczych, a także kontaktem z instruktorami i drużynowymi.
- Instruktorzy i drużynowi – administratorzy systemu odpowiedzialni za prawidłowe prowadzenie, nadzorowanie i ochronę danych.
- ZHR jako organizacja – właściciel i administrator systemu, odpowiedzialny za zgodność z prawem, bezpieczeństwo danych oraz promowanie etycznych zasad jego użytkowania.
- Twórcy systemu (zespół projektowy) – podmiot komercyjny odpowiedzialny za projektowanie i rozwój systemu. Decyzje technologiczne i etyczne wpływają na bezpieczeństwo danych oraz realizację wartości organizacyjnych.

- Społeczeństwo i instytucje publiczne – odbiorcy pośredni, dla których transparentność działań organizacji ma znaczenie wychowawcze i społeczne.

Identyfikacja tych grup umożliwia analizę wpływu systemu SZYSZKA na relacje międzyludzkie, poziom zaufania oraz odpowiedzialność za przetwarzanie danych w całym cyklu życia projektu.

4.4.3 Pozytywne aspekty społeczne

Projekt SZYSZKA przynosi szereg pozytywnych konsekwencji etycznych. Do najważniejszych z nich należą:

- Bezpieczna komunikacja wewnętrzna – system umożliwia kontakt między członkami drużyn w sposób bezpieczny i kontrolowany, ograniczając potrzebę korzystania z otwartych, masowych platform społecznościowych. Dzięki temu minimalizuje ryzyko ekspozycji dzieci i młodzieży na reklamy oraz niepożądane treści i profilowanie marketingowe.
- Ochrona prywatności – mimo komercyjnego charakteru projektu, dane użytkowników są przetwarzane w sposób zgodny z obowiązującymi przepisami i polityką ochrony danych ZHR, a baza danych pozostaje własnością tej organizacji, co dodatkowo zwiększa kontrolę nad bezpieczeństwem informacji. Dostęp do danych jest ściśle ograniczony do upoważnionych osób, co zwiększa bezpieczeństwo i zaufanie użytkowników.
- Transparentność i odpowiedzialność – funkcje monitorowania aktywności i obecności pomagają w budowaniu świadomości oraz odpowiedzialności wśród uczestników, wspierając proces wychowawczy i zarządzanie drużyną.

Projekt SZYSZKA realizuje zasadę dobra wspólnego, umożliwiając etyczne wykorzystanie technologii komercyjnej w działalności wychowawczej ZHR i przyczyniając się do rozwoju bezpiecznej przestrzeni komunikacyjnej.

4.4.4 Potencjalne ryzyka i dylematy etyczne

Mimo licznych pozytywnych aspektów SZYSZKA wiąże się również z potencjalnymi zagrożeniami etycznymi, które należało uwzględnić na etapie projektowania systemu. Do najważniejszych ryzyk należą:

- Ryzyko naruszenia prywatności – błędy w konfiguracji systemu, niewłaściwe zarządzanie dostępem lub brak aktualizacji zabezpieczeń mogą prowadzić do nieautoryzowanego ujawnienia danych użytkowników.
- Odpowiedzialność za treści publikowane w systemie – konieczne jest opracowanie zasad moderacji postów, aby zapobiegać hejtowi, wykluczeniu, nadużyciom oraz naruszeniom godności użytkowników.
- Uzależnienie od technologii – nadmierne przenoszenie aktywności harcerskiej do środowiska cyfrowego może osłabiać bezpośrednie relacje społeczne i więzi wychowawcze.
- Wykluczenie cyfrowe – w środowiskach o ograniczonym dostępie do internetu korzystanie z systemu może być utrudnione, co prowadzi do nierówności w komunikacji i uczestnictwie.
- Brak kompetencji cyfrowych części kadry – niewystarczające umiejętności techniczne mogą skutkować błędami w obsłudze systemu, co może stwarzać zagrożenia dla bezpieczeństwa danych.

Z etycznego punktu widzenia projekt wymaga więc wdrożenia nie tylko odpowiednich zabezpieczeń technicznych, ale również działań edukacyjnych i organizacyjnych, takich jak szkolenia kadry, jasne zasady administrowania dostępem oraz nadzór nad korzystaniem z systemu.

4.4.5 Ocena zgodności z normami etycznymi i prawnymi

System SZYSZKA został zaprojektowany z uwzględnieniem podstawowych zasad etyki informatycznej oraz przepisów dotyczących ochrony danych osobowych (RODO). Wśród kluczowych zasad przyjętych w projekcie SZYSZKA znajdują się:

- Zasada minimalizacji danych – system gromadzi wyłącznie informacje niezbędne do prowadzenia ewidencji i komunikacji, ograniczając przetwarzanie danych wrażliwych.
- Świadoma zgoda użytkownika – przetwarzanie danych odbywa się na podstawie zgody użytkowników lub ich opiekunów prawnych, zgodnie z obowiązującymi przepisami.
- Zasada przejrzystości – każdy użytkownik ma dostęp do informacji o zakresie, celu oraz sposobie przetwarzania swoich danych.
- Bezpieczeństwo danych – system wykorzystuje mechanizmy kontroli dostępu oraz nadzór administracyjny w celu ochrony informacji przed nieuprawnionym dostępem.
- Odpowiedzialność za dane – administratorem systemu jest ZHR, który jako organizacja ponosi odpowiedzialność za zgodność działań z RODO oraz za utrzymanie wysokiego poziomu ochrony danych użytkowników.

W świetle powyższych założeń projekt SZYSZKA spełnia standardy etyczne i prawne obowiązujące w systemach informatycznych przetwarzających dane osobowe, zapewniając równowagę między funkcjonalnością a ochroną prywatności użytkowników.

4.4.6 Wnioski analizy etycznej dla projektu SZYSZKA

Analiza etyczna systemu SZYSZKA wskazuje, że jego realizacja stanowi odpowiedzialne podejście do wykorzystania technologii w organizacji wychowawczej. System zaprojektowano z myślą o bezpieczeństwie i przejrzystości, wspiera wartości ZHR i przyczynia się do podnoszenia jakości pracy wychowawczej.

Wnioski końcowe:

- Projekt SZYSZKA wspiera etyczne wykorzystanie technologii w edukacji i wychowaniu.
- Jego wdrożenie ogranicza zależność od komercyjnych platform społecznościowych i wzmacnia ochronę prywatności.

- Kluczowym wyzwaniem pozostaje utrzymanie równowagi między wygodą cyfrową a osobistą relacją wychowawczą.
- Wymagane jest systematyczne doskonalenie kompetencji instruktorów i administratorów systemu, a także rozwijanie kultury odpowiedzialnego korzystania z narzędzi informatycznych.

Podsumowując, projekt SZYSZKA może stanowić przykład etycznego wdrożenia technologii w organizacji społeczno-wychowawczej, łącząc nowoczesne rozwiązania z tradycyjnymi wartościami harcerskimi.

Rozdział 5

Organizacja pracy

5.1 Metodyka pracy

W projekcie zastosowano klasyczny model kaskadowy zakładający przechodzenie przez etapy wytwarzania systemu sekwencyjnie [6]. Każdy etap musiał zostać zakończony i zatwierdzony przed rozpoczęciem następnego. To były etapy takie jak: analiza wymagań, projekt systemu, implementacja, testowanie oraz przygotowanie dokumentacji. Dzięki zastosowaniu tego modelu możliwe było wczesne wykrycie niespójności w wymaganiach oraz lepsze uporządkowanie prac.

5.2 Podział ról i zadań

W ramach projektu określono podział obowiązków dopasowany do etapów modelu kaskadowego.

- Artem Stakhovski
 - praca nad interfejsem użytkownika
 - stworzenie dokumentacji technicznej
 - specyfikacja i analiza wymagań
- Kacper Demczyna
 - praca nad warstwą aplikacji i jego testy
 - projekt bazy danych
 - specyfikacja i analiza wymagań
 - stworzenie dokumentacji technicznej
- Paweł Anuszkiewicz
 - backend i jego testy
 - specyfikacja i analiza wymagań
 - stworzenie dokumentacji technicznej

5.3 Harmonogram pracy

Harmonogram pozwala na jasne planowanie prac nad projektem oraz minimalizuje ryzyko opóźnień. Zakres czasowy projektu SZYSZKA jest równy 11 miesięcy od marca 2025 do stycznia 2026.

Marzec-czerwiec 2025:

- **Specyfikacja wymagań**
- **Prototyp w Figmie**
- **Model bazy danych**

Lipiec-sierpień 2025:

- **frontend:**
 - **Widok logowania**
 - **Widok profilu użytkownika**
 - **Widok postów**
 - **Widok rejestracji**
 - **Widok sprawności**
 - **Widok wydarzeń**
 - **Widok czatów**
 - **widok administratora**
- **backend:**
 - **Integracja JWT Token**
 - **Endpointy do użytkowników**
 - **Endpointy do sprawności**
 - **Endpointy do wydarzeń**
 - **Endpointy do postów**
 - **Endpointy do czatów**

Wrzesień 2025:

- **Połączenie widoków z frontenda z endpointami backendu**

Październik-listopad 2025:

- **Testowanie aplikacji**
- **Poprawa wszystkich niedokładności**

Grudzień 2025-styczeń 2026:

- **ostatnie poprawki**
- **przygotowanie do prezentacja rozwiązania**

5.4 Infrastruktura i sposoby komunikacji

Środowisko programistyczne backendu oparto na języku Java 17 oraz frameworku Spring Boot 3.5, który stanowił podstawową platformę do wytwarzania warstwy serwerowej systemu SZYSZKA. Prace programistyczne związane z tworzeniem warstwy backendu realizowano w środowisku IntelliJ IDEA, zapewniając integrację z narzędziami takimi jak Maven oraz system kontroli wersji Git. Warstwa frontendowa była rozwijana równolegle na osobnym branchu przy użyciu środowiska WebStorm, umożliwiającym wygodną pracę z językami JavaScript, HTML oraz CSS. Zarządzanie zależnościami oraz procesem budowania aplikacji realizowano przy użyciu systemu Apache Maven, który był odpowiedzialny za kontrolę wersji wykorzystywanych bibliotek, automatyczną kompilację kodu oraz uruchamianie testów. Pozwalało to na łatwiejsze utrzymanie spójności projektu oraz ułatwiało proces integracji kolejnych modułów.

5.4.1 Backend i warstwa dostępu do danych

Warstwa backendowa wykorzystywała Spring Data JPA, co jest zgodne ze standardem JPA. Integracja z relacyjną bazą danych MySQL była możliwa dzięki sterownikom JDBC dostępnym w zależnościach projektu. Repozytoria Spring umożliwiły implementację operacji CRUD (Create, Read, Update, Delete), co pozwoliło uniknąć ręcznego tworzenia zapytań.

Komunikacja między warstwą frontendową a backendową odbywała się poprzez REST API, udostępnione za pomocą modułu Spring Web. Kontrolery odpowiedzialne były za przetwarzanie żądań oraz zwracanie odpowiedzi.

5.4.2 Mechanizmy bezpieczeństwa

Ze względu na konieczność ochrony danych użytkowników zaimplementowano systemy oparte na Spring Security. Uwierzytelnianie i autoryzacja odbywały się przy użyciu tokenów JWT. Pozwalało to na zarządzanie sesją użytkownika oraz ograniczanie dostępu do wybranych zasobów.

5.4.3 Organizacja pracy zespołu i komunikacja

Podstawowym narzędziem komunikacji był Discord, który wykorzystywano zarówno do kontaktu asynchronicznego, jak i spotkań głosowych.

Komunikacja przebiegała na następujące sposoby:

- Mniej istotne informacje przekazywano na kanałach tekstowych lub na prywatnych czatach.
- Problemy techniczne wymagające konsultacji omawiano na kanałach głosowych.
- Spotkania zespołu odbywały się średnio co dwa tygodnie, z omówieniem postępów, napotkanych problemów oraz zadań przeznaczonych na okres do kolejnego spotkania.
- Po spotkaniach zamieszczono podsumowania na głównym czacie.
- Podczas spotkań korzystano z funkcji udostępniania ekranu.

Do wymiany dokumentów i plików wykorzystywano kanały prywatne na Discordzie, a także GitHub oraz Dysk Google.

Zastosowana infrastruktura oraz metody komunikacji umożliwiły sprawną organizację pracy, równoległy rozwój modułów oraz utrzymanie wysokiej jakości kodu.

5.5 Analiza ryzyka

W celu znalezienia potencjalnych ryzyk opracowana została analiza ryzyk, wyniki której są przedstawione w tabeli poniżej (Tabela 5.2.). Analiza obejmuje typy ryzyka: techniczne, bezpieczeństwo, prawne oraz organizacyjne oraz strategie ich mitygacji.

Ryzyko	Rodzaj	opis ryzyka	prawdopodobieństwo wystąpienia	wpływ na projekt	Planowane działania
Błędy w kodzie i awarie systemu	Techniczne	Możliwość wystąpienia błędów skutkujących awarią lub utratą danych	Niskie	Wysoki	Automatyzacja testów, system zgłaszania błędów
Brak skalowalności systemu	Techniczne	Spadek wydajności przy dużej liczbie użytkowników	Średnie	Średni	Implementacja cache, testy obciążeniowe
Kompatybilność technologiczna	Techniczne	System może nie działać na starszych przeglądarkach lub urządzeniach mobilnych	Niskie	Średni	Przegląd kompatybilności, fallback dla starszych technologii
Wycieki danych osobowych	Bezpieczeństwo	Nieautoryzowany dostęp do danych użytkowników, w tym niepełnoletnich	Niskie	Wysoki	Wdrożenie systemu logów i korzystanie ze Spring Security
Ataki hakerskie	Bezpieczeństwo	Możliwość ataków DDoS, SQL Injection, phishingu	Średnie	Średni	Wdrożenie WAF, testy penetracyjne

Brak akceptacji użytkowników	Organizacyjne	Użytkownicy (instruktorzy, harcerze) mogą niechętnie korzystać z systemu	Średnie	Wysoki	Warsztaty UX, pilotażowe wdrożenie
Problemy z wdrożeniem	Organizacyjne	Trudności z implementacją systemu w harcerstwie	Niskie	Wysoki	Opracowanie planu wdrożeniowego, testy użytkowników
Nieprzestrzeganie RODO	Prawne	Nieprawidłowe przetwarzanie danych osobowych	Średnie	Wysoki	Wdrożenie narzędzi do zarządzania zgodami
Brak zgód na przetwarzanie danych	Prawne	Brak zgody od opiekunów niepełnoletnich harcerzy	Średnie	Średni	Automatyzacja procesu zgód, dokumentacja prawna

Tabela 5.2 Analiza ryzyka. Źródło: opracowanie własne.

Rozdział 6

Wymagania

6.1 Udziałowcy

W tym rozdziale przedstawiono udziałowców projektu oraz określono ich rolę oraz wymagania w procesie wytwarzania systemu. Określenie udziałowców pozwala na zrozumienie ich wymagań, ograniczeń i punktu widzenia co pozwala na lepsze sformułowanie wymagań systemowych. W ramach tej analizy zidentyfikowano udziałowców biorących udział w rozwój system jak i tych korzystających z jego funkcjonalności.

KARTA UDZIAŁOWCA	
Identyfikator:	UOB 01
Nazwa:	<i>Paweł Anuszkiewicz</i>
Opis:	<i>Menadżer projektu oraz współtwórca dokumentacji oraz programista</i>
Typ udziałowca:	<i>Ożywiony bezpośredni</i>
Punkt widzenia:	<i>Student piątego semestru informatyki na oddziale w PJATK w Gdańsku profilu Cyberbezpieczeństwo oraz technik informatyk</i>
Ograniczenia:	<i>Menadżer projektu oraz współtwórca dokumentacji</i>
Wymagania:	<i>Aktywny udział w wytwarzaniu dokumentacji oraz produktu</i>

Tabela 6.1 UOB 01. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	UOB 02
UOB 01 Nazwa:	<i>Artem Stakhovski</i>
Opis:	<i>Koordynator projektu oraz współtwórca dokumentacji, oraz programista</i>
Typ udziałowca:	<i>Ożywiony bezpośredni</i>
Punkt widzenia:	<i>Student piątego semestru informatyki na oddziale PJATK w Gdańsku profilu Aplikację internetową oraz technik informatyk</i>
Ograniczenia:	<i>Koordynator projektu oraz współtwórca dokumentacji</i>
Wymagania	<i>Aktywny udział w wytwarzaniu dokumentacji oraz produktu i przechowanie plików</i>

Tabela 6.2 UOB 02. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOB 03</i>
Nazwa:	<i>Kacper Demczyna</i>
Opis:	<i>Kierownik projektu/programista</i>
Typ udziałowca:	<i>Ożywiony, bezpośredni</i>
Punkt widzenia:	<i>Student piątego semestru informatyki na oddziale PJATK w Gdańsku profilu Sztuczna inteligencja oraz technik informatyk</i>
Ograniczenia:	<i>Kierownik projektu oraz współtwórca dokumentacji</i>
Wymagania:	<i>Aktywny udział w wytwarzaniu dokumentacji oraz produktu</i>

Tabela 6.3 UOB 03. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOB 04</i>
Nazwa:	<i>Wojciech Demczyna</i>
Opis:	<i>Kilent</i>
Typ udziałowca:	<i>Ożywiony, bezpośredni</i>
Punkt widzenia:	<i>Patrzy z perspektywy użytkownika i administratora</i>
Ograniczenia:	<i>nie bierze udziału w tworzeniu kodu</i>
Wymagania:	<i>Określenie oczekiwań produktu</i>

Tabela 6.4 UOB 04. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOP 01</i>
Nazwa:	<i>Zuch</i>
Opis:	<i>Kilent</i>
Typ udziałowca:	<i>Ożywiony, pośredni</i>
Punkt widzenia:	<i>Patrzy z perspektywy użytkownika</i>
Ograniczenia:	<i>nie bierze udziału w tworzeniu kodu ani wymagań</i>
Wymagania:	<i>Brak</i>

Tabela 6.5 UOP 1. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOP 02</i>
Nazwa:	<i>Rodzie</i>
Opis:	<i>Użytkownik aplikacji</i>
Typ udziałowca:	<i>Ożywiony, pośredni</i>
Punkt widzenia:	<i>Patrzy z perspektywy użytkownika</i>
Ograniczenia:	<i>nie bierze udziału w tworzeniu kodu ani wymagań</i>
Wymagania:	<i>Brak</i>

Tabela 6.6 UOP 2. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOP 03</i>
Nazwa:	<i>Przyboczny</i>
Opis:	<i>Użytkownik aplikacji</i>
Typ udziałowca:	<i>Ożywiony, pośredni</i>
Punkt widzenia:	<i>Patrzy z perspektywy użytkownika</i>
Ograniczenia:	<i>nie bierze udziału w tworzeniu kodu ani wymagań</i>
Wymagania:	<i>Brak</i>

Tabela 6.7 UOP 3. Źródło: opracowanie własne

KARTA UDZIAŁOWCA	
Identyfikator:	<i>UOP 04</i>
Nazwa:	<i>Kandydat na Zucha</i>
Opis:	<i>Użytkownik aplikacji</i>
Typ udziałowca:	<i>Ożywiony, pośredni</i>
Punkt widzenia:	<i>Patrzy z perspektywy użytkownika którego interesuje zapisanie się do gromady</i>
Ograniczenia:	<i>nie bierze udziału w tworzeniu kodu ani wymagań</i>
Wymagania:	<i>Brak</i>

Tabela 6.8 UOP 4. Źródło: opracowanie własne

Drużynowy (administrator) - zarządza stroną, kontami innych użytkowników, ma możliwość dodawać/edytować wydarzenia oraz sprawności. Nadaje sprawności zuchom. Publikuje komunikaty. Monitoruje publikowane treści.

Przyboczny - pomaga drużynowemu w zarządzaniu gromadą. Ma wgląd do danych zuchów i rodziców, podgląd wydarzeń możliwość publikowania ogłoszeń oraz moderacji forum.

Zuch - ma wgląd do listy sprawności oraz swojego profilu na którym może publikować posty, wgląd do istniejących wydarzeń oraz ogłoszeń. Czat z innymi użytkownikami.

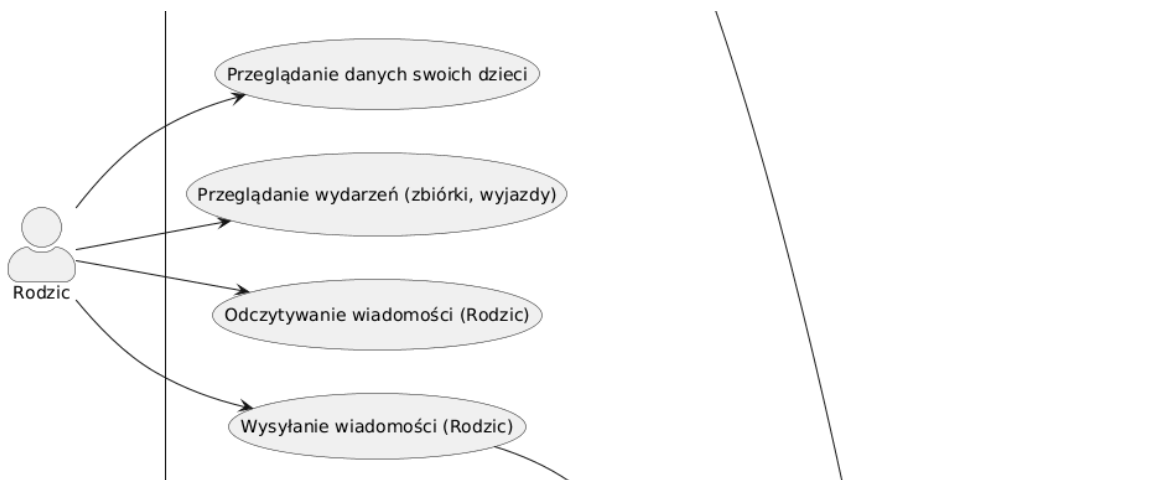
Rodzic - ma wgląd do danych swoich dzieci, wgląd do wydarzeń oraz listy sprawności, możliwość kontaktu z drużynowym i przybocznymi oraz innymi rodzicami poprzez czat.

6.2 Diagram przypadków użycia

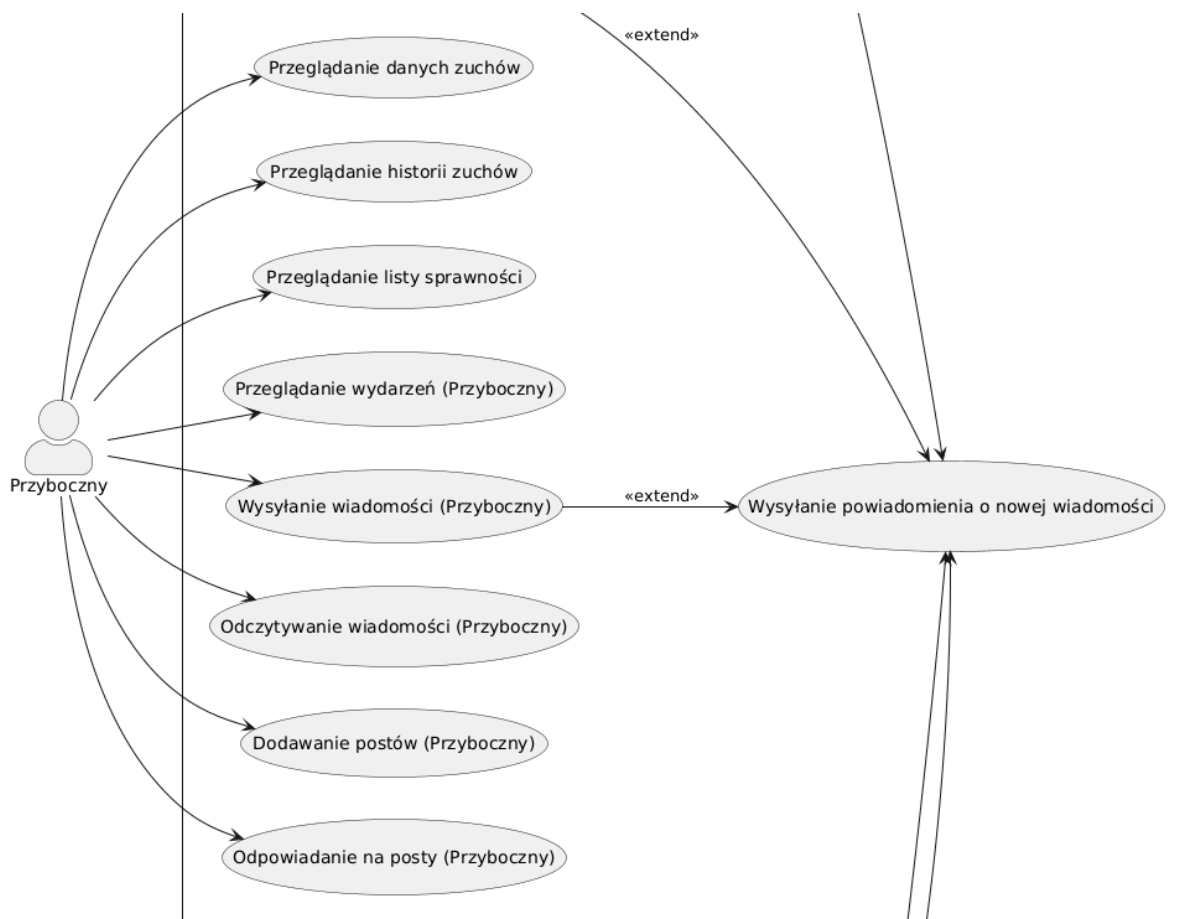
Poniższy diagram (Rysunek 6.1, Rysunek 6.2) został opracowany aby zwizualizować funkcjonalności systemu z perspektywy użytkownika określając co dany aktor może zrobić. Pokazuje konkretne funkcjonalności systemu z wyszczególnionymi aktorami: Drużynowy, Rodzic, Przyboczny, Zuch. Dla każdego z nich system będzie służył do różnych celów oraz każdy z nich będzie posiadał różne uprawnienia co przekłada się na dostęp do poszczególnych funkcjonalności.



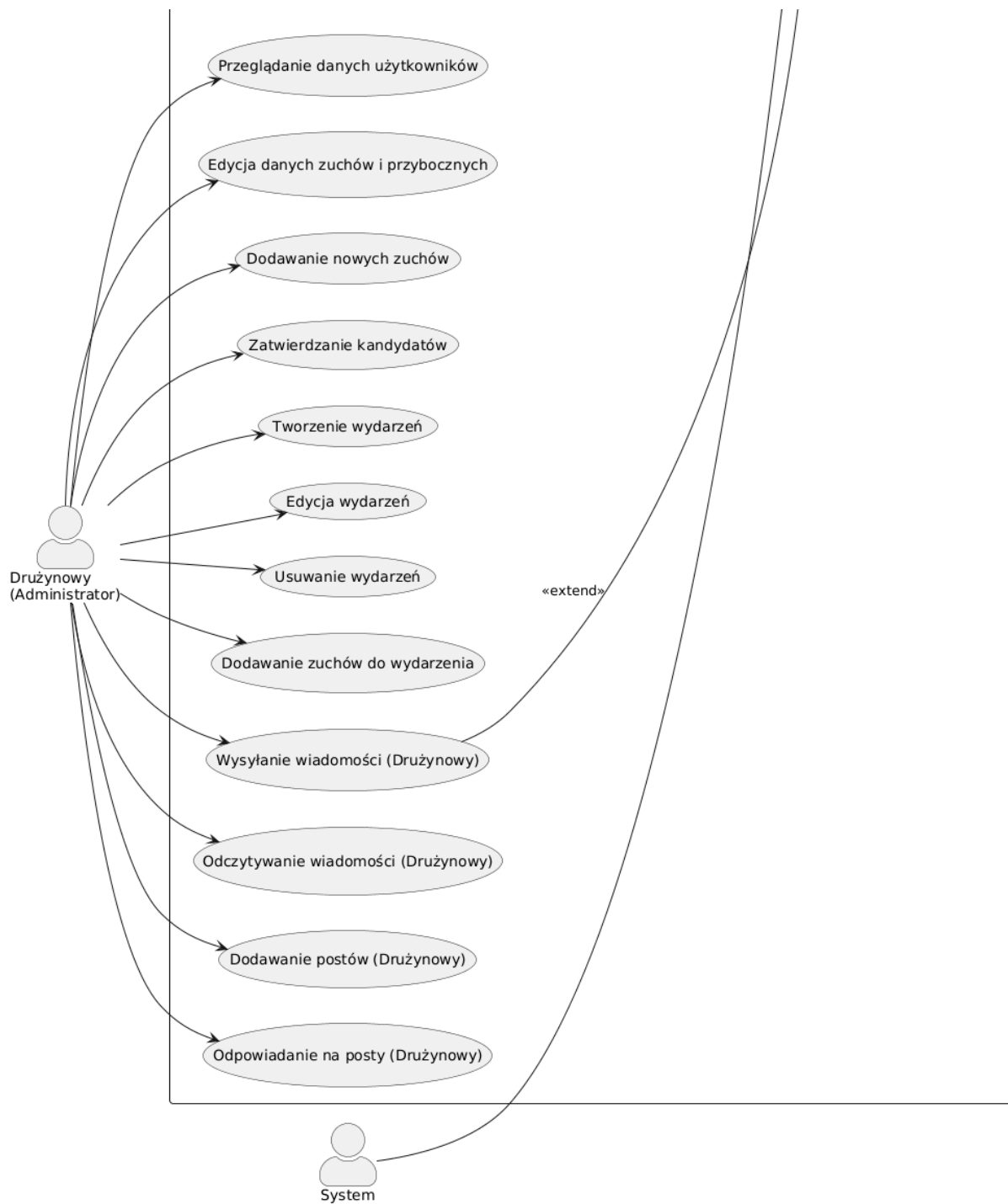
Rysunek 6.1 Diagram przypadków użycia - Zuch. Źródło: opracowanie własne



Rysunek 6.2 Diagram przypadków użycia - Rodzic. Źródło: opracowanie własne



Rysunek 6.3 Diagram przypadków użycia - Przyboczny. Źródło: opracowanie własne



Rysunek 6.4 Diagram przypadków użycia - Drużynowy. Źródło: opracowanie własne

6.3 Wymagania ogólne i dziedzinowe

Zgodne z techniką MoSCoW wszystkie wymagania zostały podzielone na cztery kategorie: Must, Should, Could, Won't. Must to wymagania niezbędne, Should to ważne, Could to pożądane, a Won't to nieplanowane [7]. Dalej w priorytecie: Must - M, Should - S, Could - C, Won't - W.

KARTA WYMAGANIA			
Identyfikator:	WO 01	Priorytet:	M
Nazwa	Komunikacja z bazą danych odbywa się za pośrednictwem internetu		
Opis	System musi mieć możliwość dostępu do bazy danych przy działającym Internecie		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05		

Tabela 6.9 Komunikacja z bazą. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WO 02	Priorytet:	M
Nazwa	Przy włączeniu aplikacji musi być dostęp do informacji ogólnej		
Opis	Na stronie głównej powinny się znaleźć informacje o gromadzie		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05, WO 01		

Tabela 6.10 Informację ogólną. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WO 03	Priorytet:	M
Nazwa	Dane są dostępne po zalogowaniu użytkownika		
Opis	Nikt nie może otrzymać dostęp do danych dowolnego użytkownika bez zalogowania do aplikacji		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 01, WO 05, WO 04		

Tabela 6.11 Dostęp do danych. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WO 04	Priorytet:	M
Nazwa	Strona jest dostępna do sześciu klas użytkowników: drużynowy (administrator), przyboczny, zuchy, rodzice zuchów, default oraz niezalogowany.		
Opis	Przy włączeniu aplikacji użytkownik jest nie zalogowany i jeżeli ma swój profil to może do niego zalogować, otrzymując uprawnienia w zależności od tego do której z klas ten profil należy: drużynowy, przyboczny, zuchy, rodzice zuchów, default, jeśli niezalogowany użytkownik rejestruje profil, to otrzymuje rolę default		
Udziałowiec	UOB 03		
Wymagania powiązane	WO 05, WD 01		

Tabela 6.12 Klasy użytkowników. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WO 05	Priorytet:	M
Nazwa	System musi mieć swoją stronę internetową dostępną przez przeglądarkę		
Opis	System musi mieć stronę internetową, na której będą dostępne wszystkie funkcjonalności aplikacji		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 01		

Tabela 6.13 Strona internetowa. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WO 06	Priorytet:	S
Nazwa	Strona powinna być dostępna na urządzeniach mobilnych i desktopowych.		
Opis	Aplikacja musi działać nie tylko na komputerach, ale także na urządzeniach mobilnych i desktopowych.		
Udziałowiec	UOB 03		
Wymagania powiązane	WO 05		

Tabela 6.14 Strona internetowa. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WD1	Priorytet :	M
Nazwa	System obsługuje specyficzną strukturę organizacyjną zuchów, która uwzględnia drużynowego, przybocznych, rodziców oraz zuchy.		
Opis	System jest oparty na już uformowaną strukturę organizacyjną która składa z zuchów ich rodziców, drużynowego oraz przybocznego		
Udziałowiec	UOB 03		
Wymagania powiązane	WO 04		

Tabela 6.15 Specyficzna struktura. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WD 02	Priorytet :	M
Nazwa	Strona wspiera organizację wydarzeń, a także gromadzi informacje o zdobytych sprawnościach.		
Opis	Użytkownik, który ma dostateczne uprawnienia może organizować wydarzenie, informując o tym innych użytkowników, oraz otrzymać informacje o zbiorkach, wyjazdach i zdobytych sprawnościach,		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05, WD 01, WO 04		

Tabela 6.16 Zbiórki, wyjazdy oraz sprawności. Źródło: opracowanie własne

6.4 Wymagania funkcjonalne

KARTA WYMAGANIA			
Identyfikator:	WF 01	Priorytet:	M
Nazwa	Logowanie użytkownika		
Opis	Każdy zarejestrowany użytkownik chce mieć możliwość zalogowania się na prywatne konto z odpowiednimi dla niego uprawnieniami. Wtedy będzie miał dostęp do wszystkich funkcji systemu odpowiadających jego uprawnieniom		
Kryteria akceptacji	Warunki Satysfakcji (Szczegóły dodane na potrzeby testów akceptacyjnych) Każdy rodzaj użytkownika będzie mógł się zalogować na swoje konto i będzie miał dostęp do wszystkich wdrożonych uprawnień mu odpowiadających		
Dane wejściowe	Wybranie nie zalogowanym użytkownikom opcji zaloguj się, hasło oraz login podane przez użytkownika.		
Warunki początkowe	Użytkownik nie jest zalogowany		
Warunki końcowe	użytkownik zostaje zalogowany w przypadku pomyślnej autoryzacji		
Sytuacje wyjątkowe	Błędne hasło lub login=>użytkownik nie zostaje zalogowany Błędne hasło lub login 3 razy=> użytkownik nie może poprzez 5 minut		
Szczegóły implementacji	Porównywanie podanych loginu i hasła z oryginałami w bazę danych		
Udziałowiec	UOB 04		
Wymagania powiązane	WO 05, WO 04, WO 03, WO 01		

Tabela 6.17 Logowanie. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 02	Priorytet:	M
Nazwa	Dostęp do listy sprawności przez użytkowników		
Opis	Lista sprawności dostępna dla wszystkich użytkowników z pobieżnym opisem wymagań. Możliwość kliknięcia na sprawność w celu rozwinięcia detali na temat wymagania (data zdobycia osiągnięcia, wymagania zaliczenia)		
Kryteria akceptacji	Zalogowany użytkownik może uzyskać dostęp do swoich sprawności		
Dane wejściowe	Użytkownik wybiera opcję sprawdzenia stanu swoich sprawności.		
Warunki początkowe	strona profilu użytkownika		
Warunki końcowe	Strona pokazująca sprawności danego użytkownika		
Sytuacje wyjątkowe	Użytkownik nie ma sprawności=>wyskakuje komunikat ("Nie masz otrzymanych sprawności)		
Szczegóły implementacji	Wysłanie zapytania do bazy danych		
Udziałowiec	UOB 04		
Wymagania powiązane	WO 04, WF 01		

Tabela 6.18 Dostęp do sprawności. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 03	Priorytet:	M
Nazwa	Czat		
Opis	Każdy zalogowany użytkownik ma dostęp do czatów drużyny, szóstki(jeżeli posiada) oraz może tworzyć czaty z innymi użytkownikami		
Kryteria akceptacji	Znajdowanie na stronie aplikacji strony z czatami		
Dane wejściowe	Użytkownik wybiera stronę czaty		
Warunki początkowe	Użytkownik jest zalogowany		
Warunki końcowe	Użytkownik widzi stronę z czatami		
Sytuacje wyjątkowe	błędny login przy tworzeniu czatu=>czat nie zostaje utworzony		
Szczegóły implementacji	Dodane strony z czatami		
Udziałowiec	UOB 03		
Wymagania powiązane	WO 05, WF 01		

Tabela 6.19 Strona z czatami. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 04	Priorytet:	M
Nazwa	Czat drużyny		
Opis	na stronie czatów musi znajdować się czat drużyny który jest widoczny wszystkim zalogowanym użytkownikom.W tym czacie może pisać tylko administratorze		
Kryteria akceptacji	Administratorzy mogą dodawać nowe wiadomości do czatu który jest widoczny dowolnemu użytkownikowi po zalogowaniu		
Dane wejściowe	Nowa wiadomość		
Warunki początkowe	Otwarta strona czatu przez administratora		
Warunki końcowe	nowa wiadomość wiadomość pojawia się w czacie widoczna do wszystkich		
Sytuacje wyjątkowe	wiadomość jest pusta=>wiadomość nie została dodana		
Szczegóły implementacji	Dodanie uprawnień administratorom do dodawania wiadomości w czacie drużyny		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 04,WO 05, WF 03		

Tabela 6.20 Informację ogólne. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 05	Priorytet:	S
Nazwa	Generacja list grup zuchów na poszczególne wydarzenia		
Opis	Drużynowy może dodawać zuchów do list wyjazdowych i automatycznie generować listę tych zuchów. Lista może być dalej przesłana do rodziców, zuchowe mają wgląd do list do których należą po zalogowaniu na serwis		
Kryteria akceptacji	Drużynowy generuje listy i przysyła ich do rodziców, którzy potem mogą ją zobaczyć		
Dane wejściowe	Lista zuchów, wybrana opcja przesyłania		
Warunki początkowe	Użytkownik zalogowany jak administrator wybiera opcję stwórz listę zuchów		
Warunki końcowe	Lista została stworzona i przesłana do rodziców		
Sytuacje wyjątkowe	Dany zuch nie ma rodziców nie ma danych w pole rodzice=>wiadomość przesyłana do niego Podczas tworzenia listy jakiś z zuchów został usunięty z bazy=>wyświetla alert `próbujesz dodać nie istniejącego użytkownika\${id}`		
Szczegóły implementacji	Wysyłanie zapytania do bazy danych, a potem wysłanie komunikatu na konta użytkowników		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WO 04		

Tabela 6.21 Tworzenie list uczestników do wydarzenia. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 06	Priorytet:	M
Nazwa	zarządzanie postami		
Opis	Każdy zalogowany użytkownik może dodawać nowe posty do systemu .Posty mogą być edytowane przez ich właściciela. Posty mogą być usunięty poprzez ich właściciela lub administratora		
Kryteria akceptacji	Użytkownik tworzy nowy post, użytkownik usuwa post,użytkownik edytuje post		
Dane wejściowe	Dane posta		
Warunki początkowe	Użytkownik zalogowany		
Warunki końcowe	Post został stworzony		
Sytuacje wyjątkowe	Podczas tworzenia, edycji ,lub suwania posta wygasł token użytkownika=>przekierowanie na stronę logowania podczas tworzenia lub edycji posta użytkownik wpisał za mały opis=> użytkownik otrzymuje o tym komunikat		
Szczegóły implementacji	Wysyłanie zapytania do bazy danych z danymi posta i id użytkownika		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WO 04		

Tabela 6.22 Tworzenie postów. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 07	Priorytet:	W
Nazwa	Sprawdzenie obecności		
Opis	Sprawdzanie obecności na wydarzeniach przy pomocy NFC, obecności powinny być zapisywane automatycznie w bazie danych		
Kryteria akceptacji	Warunki Satisfakcji (Szczegóły dodane na potrzeby testów akceptacyjnych)		
Dane wejściowe	Sygnał NFC		
Warunki początkowe	Brak-nie ma w tej implementacji		
Warunki końcowe	Brak-nie ma w tej implementacji		
Sytuacje wyjątkowe	Członek gromady nie posiada urządzenia NFC		
Szczegóły implementacji	Brak-nie ma w tej implementacji		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WF 02		

Tabela 6.23 Sprawdzenie obecności. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 08	Priorytet:	S
Nazwa	Dostęp do danych dzieci		
Opis	Każdy rodzic użytkownik będzie miał wgląd do historii wyjazdów w których brało udział jego dziecko, obecności na zbiórkach oraz listy sprawności		
Kryteria akceptacji	Rodzic może zobaczyć informację jego dzieci		
Dane wejściowe	Rodzic wybiera opcję zobacz informację gdzie znajdują się informacja o jego dzieciach		
Warunki początkowe	Użytkownik zalogowany jako rodzic znajduje się na swojej stronie		
Warunki końcowe	Rozwija się pełna informacja o wybranym koncie		
Sytuacje wyjątkowe	Informacja w bazie danych została zedytowana po wybraniu opcji=>na stronie będzie pokazywała nie zedytowana informacja Wygasł czas zalogowania użytkownika=> przekierowanie na logowanie		
Szczegóły implementacji	Zapytanie do bazy danych		
Udziałowiec	UOB 04		
Wymagania powiązane	WF 05, WF 01		

Tabela 6.24 Dostęp do danych dzieci. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 09	Priorytet:	M
Nazwa	Edycja danych użytkowników		
Opis	Użytkownik ma możliwość edycji swoich danych		
Kryteria akceptacji	Użytkownik może edytować swoje dane		
Dane wejściowe	Nowe dane użytkownika		
Warunki początkowe	Dane użytkownika		
Warunki końcowe	Zmienione dane użytkownika		
Sytuacje wyjątkowe	dane nie przeszli sprawdzenia poprawności na przykład, email nie jest poprawnym emailem=>użytkownik otrzymuje komunikat o każdym nie przeszedzszym pole użytkownik został usunięty podczas edycji=>otrzymuje komunikat że nie jest zalogowany		
Szczegóły implementacji	Użytkownik ma uprawnienia na edycję swojego profilu		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WO 01		

Tabela 6.25 Edytowanie profilu przez użytkownika. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 10	Priorytet:	M
Nazwa	Kontrola kont		
Opis	Drużynowy ma możliwość dodawania, edytowania i usuwania kont poprzez interfejs graficzny serwisu. Drużynowy także może modyfikować istniejące konta i zmieniać ich uprawnienia.		
Kryteria akceptacji	Administrator może zarządzać nowymi i istniejącymi kontami używając interfejsu graficznego		
Dane wejściowe	-		
Warunki początkowe	-		
Warunki końcowe	Konto administracyjne z ograniczonymi uprawnieniami ma wgląd do wszystkich danych grupy.		
Sytuacje wyjątkowe	uzupełniane w trakcie sprintu – niepożądane sytuacje i sposoby ich obsługi}		
Szczegóły implementacji	Form		
Udziałowiec	UOB 04		
Wymagania powiązane	WF 01, WO 05, WO 01, WF 09		

Tabela 6.26 Kontrola kont. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 11	Priorytet:	S
Nazwa	ukrycie oraz zgłaszanie postów		
Opis	Użytkownicy mogą ukrywać posty innych użytkowników w sposób, że posty tych użytkowników już nie będą do nich widoczne. Też oni mogą zgłaszać posty do administracji jeżeli uważają, że te posty są złośliwie napisane		
Kryteria akceptacji	Użytkownik może ukryć i zgłosić post		
Dane wejściowe	Post należący do innego użytkownika		
Warunki początkowe	Odkryty post		
Warunki końcowe	Post został zgłoszony lub ukryty		
Sytuacje wyjątkowe	Przed kliknięciem wygasł token użytkownika=>przekierowanie na stronę logowania		
Szczegóły implementacji	Wysyłanie zapytania do bazy danych danymi posta i id użytkownika		
Udziałowiec	UOB 02		
Wymagania powiązane	WF 01, WO 04, WF 06		

Tabela 6.27 Ukrycie oraz zgłaszanie postów. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 12	Priorytet:	S
Nazwa	Zarządzanie wydarzeniami		
Opis	Drużynowy ma możliwość przeglądania i zarządzania wydarzeniami		
Kryteria akceptacji	Przyboczny ma możliwość tylko do odczytu danych odnośnie grupy i poszczególnych zuchów.		
Dane wejściowe	-		
Warunki początkowe	WF 05		
Warunki końcowe	Konto administracyjne z ograniczonymi uprawnieniami ma wgląd do wszystkich danych grupy.		
Sytuacje wyjątkowe	{ uzupełniane w trakcie sprintu – niepożądane sytuacje i sposoby ich obsługi }		
Szczegóły implementacji	Administrator ma możliwość stworzenia nowego konta administracyjnego z ograniczonymi uprawnieniami(przybocznego) lub przydzielenia istniejącym kontem odpowiednie upoważnienie.		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WF 05		

Tabela 6.28 Zarządzanie wydarzeniami. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 13	Priorytet:	S
Nazwa	Dostęp tylko do odczytu danych		
Opis	Osoby uprawnione powinny mieć dostęp do wglądu zapisanych informacji o zuchach bez możliwości modyfikacji lub kasacji danych. Administrator/Drużynowy ma opcje przydzielenia do jakiego typu informacji takie konto ma wgląd.		
Kryteria akceptacji	Konto administracyjne z ograniczonymi uprawnieniami ma możliwość do tylko odczytu danych odnośnie grupy i poszczególnych zuchów.		
Dane wejściowe	-		
Warunki początkowe	Użytkownik jest zalogowany		
Warunki końcowe	Konto administracyjne z ograniczonymi uprawnieniami ma wgląd do wszystkich danych grupy.		
Sytuacje wyjątkowe	Brak		
Szczegóły implementacji	Administrator ma możliwość stworzenia nowego konta administracyjnego z ograniczonymi uprawnieniami lub przydzielenia istniejącym kontem odpowiednie upoważnienie.		
Udziałowiec	UOB 02		
Wymagania powiązane	WF 01, WO 04, WO 05		

Tabela 6.29 Ograniczony dostęp. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 14	Priorytet:	M
Nazwa	Rejestracja użytkownika		
Opis	Jest możliwość stworzenia nowego konta dla każdego typu użytkownika, a potem zalogowania do tego konta		
Kryteria akceptacji	Stworzenie każdego typu użytkownika		
Dane wejściowe	Wybranie nie zalogowanym użytkownikom opcji zarejestruj się, hasło, login oraz dodatkowe pola zależne od typu użytkownika podane przez użytkownika.		
Warunki początkowe	strona rejestracji		
Warunki końcowe	zostaje utworzone nowe konto, użytkownik zostanie zalogowany do niego		
Sytuacje wyjątkowe	Błędne dane=>użytkownik otrzymuje o tym komunikat		
Szczegóły implementacji	Porównywanie podanych loginu i hasła z oryginałami w bazę danych		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05, WO 04, WO 03, WO 01		

Tabela 6.30 Logowanie. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WF 15	Priorytet:	M
Nazwa	Wgląd do danych użytkownika		
Opis	Zuch ma wgląd do swoich danych, sprawności, „profilu, postów oraz wydarzeń w których uczestniczył		
Kryteria akceptacji	Zalogowany zuch może uzyskać dostęp do swoich sprawności, historii obecności oraz wyjazdów w których uczestniczył		
Dane wejściowe	Użytkownik wybiera opcję sprawdzenia stanu swoich sprawności, historii obecności lub wyjazdów w których uczestniczył		
Warunki początkowe	Strona główna		
Warunki końcowe	Strona pokazująca sprawności danego użytkownika, historii obecności lub wyjazdów w których uczestniczył		
Sytuacje wyjątkowe	Użytkownik nie ma sprawności=>wyskakuje komunikat (“Nie masz otrzymanych sprawności) Użytkownik nie ma historii obecności=>wyskakuje komunikat (“Nie masz historię obecności) Użytkownik nie uczestniczył w wyjazdach=>wyskakuje komunikat (“Nie uczestniczył w wyjazdach)		
Szczegóły implementacji	Wysłanie zapytania do bazy danych		
Udziałowiec	UOB 03		
Wymagania powiązane	WF 01, WO 04, WO 05		

Tabela 6.31 Wgląd do danych. Źródło: opracowanie własne

6.5 Wymagania pozafunkcjonalne

KARTA WYMAGANIA			
Identyfikator:	WPF 01	Priorytet:	M
Nazwa	System musi być dostępny 7 dni w tygodniu, 24 godziny na dobę.		
Opis	System musi być dostępny przez całą dobę, siedem dni w tygodniu, zapewniając użytkownikom nieprzerwany dostęp do strony i swoich kont oraz interakcji z twórcami. Dzięki wysokiej dostępności, użytkownicy mogą korzystać z aplikacji w dowolnym czasie, niezależnie od ich harmonogramu dnia. Takie podejście zwiększa zaangażowanie użytkowników oraz zapewnia stały przepływ informacji i aktywności na platformie.		
Kryteria akceptacji	System musi być dostępny 99.9% czasu w ciągu miesiąca, zapewniając użytkownikom nieprzerwany dostęp do wszystkich funkcji aplikacji. W przypadku awarii, czas reakcji na jej naprawę, nie powinien przekraczać 1 godziny, aby zminimalizować zakłócenia w dostępności. System powinien zapewniać szybki czas ładowania stron, nie przekraczający 2 sekund, nawet w godzinach szczytu użytkowania		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05		

Tabela 6.32 Dostępność systemu. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 02	Priorytet:	S
Nazwa	System musi być bezpieczny.		
Opis	System musi być zabezpieczony przed atakami zewnątrz. To znaczy że nikt nie zmoże do niego włamać i zmienić dane, również system powinien mieć odporność na DOS ataki		
Kryteria akceptacji	Każdy z programistów spróbuje włamać do systemu. Wymaganie jest akceptowane, gdy to nie uda u każdego z programistów		
Udziałowiec	UOB 02		
Wymagania powiązane	WPF 01		

Tabela 6.33 Bezpieczeństwo systemu. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 03	Priorytet:	M
Nazwa	System musi cały czas monitorować zamieszczane treści		
Opis	System musi być dostępny przez całą dobę, siedem dni w tygodniu, zapewniając użytkownikom nieprzerwany dostęp do strony i swoich kont oraz interakcji z twórcami. Dodatkowo, system musi cały czas monitorować zamieszczane treści i automatycznie blokować treści niezgodne z polityką firmy, zapewniając bezpieczne i przyjazne środowisko dla wszystkich użytkowników.		
Kryteria akceptacji	System musi mieć zaimplementowane mechanizmy automatycznej moderacji, które będą wykrywać i blokować treści niezgodne z polityką firmy w czasie rzeczywistym, zapewniając bezpieczne środowisko dla użytkowników.		
Udziałowiec	UOB 02		
Wymagania powiązane	WO 05		

Tabela 6.34 Monitorowanie treści. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 04	Priorytet:	M
Nazwa	Interfejs przyjazny i prosty w obsłudze dla dzieci i dorosłych.		
Opis	System musi być łatwy w dla użytkownika, żeby dla niego nie było problemu w dostępie do dowolnej funkcjonalności		
Kryteria akceptacji	Nowy użytkownik musi zdążyć otrzymać dostęp do dowolnej z funkcjonalności w ciągu nie więcej niż 5 minut		
Udziałowiec	UOB 02		
Wymagania powiązane	WPF 01, WO 05		

Tabela 6.35 Przyjazność interfejsu. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 05	Priorytet:	S
Nazwa	Płynne działanie strony niezależnie od liczby użytkowników.		
Opis	Niezależnie od liczby aktualnie korzystających użytkowników strona powinna działać płynnie oraz zapewniać płynny przepływ informacji.		
Kryteria akceptacji	Oczekiwanie na wyszukiwanie danych oraz ich edycję nie powinno trwać dłużej niż sekundę.		
Udziałowiec	UOB 03		
Wymagania powiązane	WPF 01		

Tabela 6.36 Niezależność od liczby użytkowników. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 06	Priorytet:	M
Nazwa	Ochrona danych użytkowników		
Opis	System powinien implementować ochronę danych w celu zapewnienia ich poufności. Ochrona danych powinna być zgodna z wytycznymi RODO.		
Kryteria akceptacji	Ochrona danych strony jest zgodna z wytycznymi RODO.		
Udziałowiec	UOB 03		
Wymagania powiązane	WPF 02		

Tabela 6.37 Ochrona danych. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 07	Priorytet:	M
Nazwa	Zabezpieczenie dostępu użytkowników do usług poprzez autoryzację.		
Opis	Weryfikacja dostępu do danych przechowywanych na stronie będzie wieloetapowa. Weryfikacja dostępu do kont o większych możliwościach będzie bardziej złożona.		
Kryteria akceptacji	Wydolność systemu powinna być zapewniona do liczby do tysiąca użytkowników.		
Udziałowiec	UOB 01		
Wymagania powiązane	WPF 01, WPF 05		

Tabela 6.38 Autoryzacja. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 08	Priorytet:	M
Nazwa	Możliwość szybkiego blokowania i usuwania kont podczas naruszeń.		
Opis	System musi pozwalać na szybkie blokowanie lub usuwanie kont w przypadku naruszeń.		
Kryteria akceptacji	Wydolność systemu powinna być zapewniona do liczby do tysiąca użytkowników.		
Udziałowiec	UOB 01		
Wymagania powiązane	WPF 01, WPF 05, WPF 07		

Tabela 6.39 Usuwanie kont. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 09	Priorytet:	M
Nazwa	System powinien spełniać wymagania RODO		
Opis	System musi udostępniać oraz chronić informacje zgodnie ze wskazaniami RODO podczas pracy w UE.		
Kryteria akceptacji	Potwierdzenie zgodności produktu z wymaganiami RODO.		
Udziałowiec	UOB 01		
Wymagania powiązane	WPF 07, WPF 06		

Tabela 6.40 RODO. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 10	Priorytet:	M
Nazwa	Dostępność strony dla osób o ograniczonej sprawności.		
Opis	Strona powinna umożliwiać funkcjonalność zgodnie ze standardami WCAG dla osób o ograniczonej sprawności.		
Kryteria akceptacji	Strona posiada funkcje zgodne ze wskazaniami WCAG.		
Udziałowiec	UOB 01		
Wymagania powiązane	WPF 04		

Tabela 6.41 Osoby o ograniczonej sprawności. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WPF 11	Priorytet:	M
Nazwa	Zabezpieczanie oraz magazynowanie danych systemu.		
Opis	System powinien tworzyć backupy ogółu danych oraz archiwizować poprzednie wersje danych.		
Kryteria akceptacji	System tworzy backupy oraz archiwizuje stare dane.		
Udziałowiec	UOB 01		
Wymagania powiązane	WPF 08, WPF 06		

Tabela 6.42 Backupy. Źródło: opracowanie własne

6.6 Interfejs z otoczeniem

Podrozdział Interfejs z otoczeniem określa sposób, w jaki system SZYSZKA komunikuje się z użytkownikami oraz systemami zewnętrznymi. Wymagania obejmują interfejs użytkownika, interfejs komunikacji frontendu i backendu oraz zasady wymiany informacji. Poniższe wymagania opisują minimalne standardy jakie musi spełniać system.

KARTA WYMAGANIA			
Identyfikator:	WI01	Priorytet :	S
Nazwa	Skalowalność systemu		
Opis	System powinien umożliwiać obsługę rosnącej liczby użytkowników oraz zwiększonego obciążenia bez pogorszenia wydajności. Architektura backendu musi zapewniać możliwość dalszej rozbudowy oraz efektywnego skalowania systemu.		
Kryteria akceptacji	System utrzymuje stabilne czasy odpowiedzi bez spadku poniżej 500 mls przy wzroście liczby jednocześnie aktywnych użytkowników do wartości 200.		
Dane wejściowe	Obciążenie generowane przez wysoką liczbę użytkowników wykonujących zapytania do API.		
Warunki początkowe	Serwer aplikacji jest uruchomiony i poprawnie działający, baza danych jest dostępna.		
Warunki końcowe	System obsługuje zwiększone obciążenie bez błędów krytycznych oraz znaczących opóźnień w odpowiedziach API.		
Sytuacje wyjątkowe	W przypadku przekroczenia maksymalnych dopuszczalnych limitów obciążenia system zwraca komunikat o niedostępności.		
Szczegóły implementacji	Brak implementacji mechanizmów skalowania; system można rozbudować w przyszłości w zależności od wymagań.		
Udziałowiec	UOB 01		
Wymagania powiązane	Brak		

Tabela 6.43 Skalowalność. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 02	Priorytet :	M
Nazwa	Wymiana danych między warstwami systemu		
Opis	System musi umożliwiać eksport i import danych pomiędzy warstwą frontendową i backendową oraz zapis i odczyt z relacyjnej bazy danych.		
Kryteria akceptacji	Komunikacja z bazą danych działa poprawnie w zaimplementowanych funkcjach.		
Dane wejściowe	Dane przekazywane pomiędzy warstwą frontendową a backendową w postaci żądań HTTP (JSON oraz multipart/form-data), treść w tym dane użytkowników oraz identyfikatory obiektów systemu.		
Warunki początkowe	System backendowy oraz frontendowy są uruchomione i posiadają aktywne połączenie z relacyjną bazą danych.		
Warunki końcowe	Dane są poprawnie przesłane do bazy danych oraz zwracane z niej zgodnie z realizowanymi opcjami.		
Sytuacje wyjątkowe	Brak połączenia z bazą danych, błędne dane wejściowe lub błędy komunikacji pomiędzy warstwami systemu, wtedy zwracany jest komunikat o błędzie.		
Szczegóły implementacji	Użyte zostały narzędzia: Rest, JSON oraz multipart na poziomie komunikacji HTTP.		
Udziałowiec	UOB 01		
Wymagania powiązane	WO 01		

Tabela 6.44 Import i eksport danych. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 03	Priorytet :	M
Nazwa	Usługa przesyłania plików (Multipart/Form-Data)		
Opis	System umożliwia przesyłanie plików pomiędzy warstwą frontendową a backendową z wykorzystaniem zapytań HTTP typu (multipart/Form-Data).		
Kryteria akceptacji	API obsługuje przesyłanie plików (POST), aktualizację przypisanych plików (PUT) w ramach zdefiniowanych zasobów systemu.		
Dane wejściowe	Żądania HTTP typu multipart/form-data zawierające pliki oraz dane identyfikujące zasób, z którym pliki są powiązane.		
Warunki początkowe	Backend jest uruchomiony i dostępny, a użytkownik posiada uprawnienia do modyfikacji danego zasobu.		
Warunki końcowe	Pliki zostają zapisane, zaktualizowane lub usunięte zgodnie z wywołaną operacją, a system zwraca odpowiednią odpowiedź HTTP.		
Sytuacje wyjątkowe	Nieprawidłowo sformułowane żądania (np. brak pliku lub niepoprawny format multipart), przesłanie nieobsługiwanego pliku, brak wymaganych uprawnień użytkownika lub błędy zapisu danych skutkują zwróceniem odpowiedniego komunikatu błędu.		
Szczegóły implementacji	Operacje realizowane są przy użyciu kontrolerów Spring Web, obsługujących żądania multipart/form-data oraz mapujących operacje POST, PUT i DELETE na logikę aplikacji.		
Udziałowiec	UOB 01		
Wymagania powiązane	WO 01, WI 02		

Tabela 6.45 REST API. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 04	Priorytet :	M
Nazwa	Format wymiany danych - JSON		
Opis	Dane przesyłane między frontendem a backendem będą w przesyłane w formacie JSON we wszystkich zapytaniach nie zawierających w sobie zdjęć.		
Kryteria akceptacji	Odpowiedzi API są zwracane w formacie JSON.		
Dane wejściowe	Dane w formacie JSON są przesyłane w treści żądania HTTP.		
Warunki początkowe	Klient wysyła dane w formacie JSON.		
Warunki końcowe	Backend zwraca prawidłową treść JSON po przetworzeniu danych.		
Sytuacje wyjątkowe	Nieprawidłowy format JSON skutkuje zwróceniem komunikatu o błędzie w odpowiedzi HTTP.		
Szczegóły implementacji	Automatyczna serializacja i deserializacja danych w formacie JSON realizowana przez mechanizmy frameworka Spring Boot.		
Udziałowiec	UOB 01		
Wymagania powiązane	WO 01, WI 02		

Tabela 6.46 Format danych. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 05	Priorytet :	M
Nazwa	komunikacja przez REST API		
Opis	System będzie udostępniać jednolity interfejs komunikacyjny REST(REST API), używany przez frontend do wykonywania operacji na danych, w tym przesyłania danych w formacie JSON oraz multipart/form-data..		
Kryteria akceptacji	API będzie obsługiwać żądania: GET, POST, PUT oraz DELETE dla wyznaczonych zasobów.		
Dane wejściowe	Żądania HTTP wysyłane przez frontend.		
Warunki początkowe	Backend jest uruchomiony i połączony z frontendem użytkownika poprzez sieć.		
Warunki końcowe	Backend zwraca odpowiedź zgodną ze specyfikacją endpointu.		
Sytuacje wyjątkowe	Nieprawidłowe żądania i dane w nich będą skutkowały odpowiedzią informującą o błędzie.		
Szczegóły implementacji	Zaimplementowane będą kontrolery Spring Web oraz mapowanie endpointów na operacje aplikacji.		
Udziałowiec	UOB 01		
Wymagania powiązane	WI 05, WO 01		

Tabela 6.47 REST API. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 06	Priorytet :	M
Nazwa	Obsługa błędów		
Opis	System zapewnia spójną obsługę błędów poprzez zwracanie precyzyjnych komunikatów w odpowiedzi na nieprawidłowe żądania klienta lub błędy po stronie serwera, zgodnie ze standardami REST.		
Kryteria akceptacji	Odpowiedzi błędów są zwracane w ujednoliconym formacie (JSON).		
Dane wejściowe	Nieprawidłowo sformułowane żądania HTTP klienta lub błędne żądania po stronie serwera.		
Warunki początkowe	- Backend jest uruchomiony i dostępny przez interfejs REST API. - Klient wysyła żądanie HTTP do jednego z dostępnych endpointów systemu.		
Warunki końcowe	- System zwraca odpowiedź HTTP zawierającą informację o błędzie i jego opis.		
Sytuacje wyjątkowe	- Przesłanie nieprawidłowego formatu danych (np. błędny JSON lub multipart). - Próba wykonania operacji na nie istniejącym zasobie. - Brak wymaganych uprawnień użytkownika. - Błędy zapisu lub odczytu danych (np. błąd bazy danych, błąd zapisu pliku).		
Szczegóły implementacji	- Obsługa błędów realizowana jest po stronie backendu z wykorzystaniem mechanizmów Spring Boot, takich jak obsługa wyjątków w kontrolerach REST oraz komunikaty zwracane w formacie JSON.		
Udziałowiec	UOB 01		
Wymagania powiązane	Brak		

Tabela 6.48 REST API. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 07	Priorytet :	M
Nazwa	Uwierzytelnianie użytkownika		
Opis	System będzie umożliwiać uwierzytelnianie użytkownika za pomocą tokenów JWT nadawanych podczas logowania.		
Kryteria akceptacji	- System umożliwia uwierzytelnienie użytkownika na podstawie poprawnych danych logowania. - Po poprawnym uwierzytelnieniu system generuje i zwraca token JWT. - Token JWT jest poprawnie weryfikowany przy dostępie do zabezpieczonych zasobów API. - Próba dostępu do chronionych zasobów bez odpowiedniego tokenu skutkuje błędem.		
Dane wejściowe	Podane dane logowania użytkownika (login i hasło).		
Warunki początkowe	System backendowy oraz frontendowy są uruchomione.		
Warunki końcowe	- W przypadku pomyślnego uwierzytelnienia użytkownik otrzymuje ważny token JWT. - W przypadku niepoprawnych danych logowania system odrzuca żądanie i zwraca odpowiedni komunikat błędu.		
Sytuacje wyjątkowe	- Podanie nieprawidłowych danych logowania skutkuje brakiem uwierzytelnienia.. - Próba użycia nieważnego lub wygasłego tokenu JWT skutkuje odrzuceniem żądania. - Brak tokenu JWT przy dostępie do zabezpieczonych zasobów skutkuje odmową dostępu.		

Szczegóły implementacji	- Uwierzytelnianie użytkownika realizowane jest z wykorzystaniem Spring Security. - Po poprawnej weryfikacji danych logowania generowany jest token JWT. - Token JWT jest dołączany do kolejnych żądań HTTP. - Weryfikacja tokenów odbywa się przed dostępem do chronionych endpointów API.
Udziałowiec	UOB 01
Wymagania powiązane	WF 01

Tabela 6.49 REST API. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	WI 08	Priorytet :	M
Nazwa	Autoryzacja użytkownika		
Opis	System będzie umożliwiać użytkownikom dostęp do operacji na poziomie ich wynikających z ról uprawnień.		
Kryteria akceptacji	Dostęp do zasobów jest uzależniony od roli użytkownika. Użytkownik o odpowiednich uprawnieniach może wykonać dozwolone operacje, jeśli użytkownik nie posiada wystarczających uprawnień do wykonania operacji nie będzie ona wykonana.		
Dane wejściowe	Token JWT przesyłany w nagłówku żądania HTTP.		
Warunki początkowe	- Zdefiniowane są role i uprawnienia. - TOKEN użytkownika jest ważny i zawiera informacje o roli użytkownika.		
Warunki końcowe	Użytkownik uzyskuje dostęp do zasobu wtedy i tylko wtedy kiedy posiada odpowiednie uprawnienia.		
Sytuacje wyjątkowe	- Brak aktualnego tokenu skutkuje brakiem dostępu. - Użytkownik próbuje uzyskać dostęp do zasobu, do którego nie posiada uprawnień.		
Szczegóły implementacji	- Autoryzacja realizowana jest z wykorzystaniem mechanizmów Spring Security. - Role użytkowników są przypisywane na etapie uwierzytelniania i zawarte w tokenie JWT. - Dostęp do zasobów API jest ograniczany na podstawie ról użytkowników.		
Udziałowiec	UOB 01		
Wymagania powiązane	WI 08, WI 07, WF 01		

Tabela 6.50 REST API. Źródło: opracowanie własne

6.7 Wymagania na środowisko

KARTA WYMAGANIA			
Identyfikator:	ŚD 01	Priorytet	M
Nazwa	System Operacyjny		
Opis	Aplikacja powinna działać na popularnych systemach operacyjnych.		
Kryteria akceptacji	Aplikacja będzie działała na Systemach Windows(10 i nowszych wersjach), oraz Android(9.0 i nowszych wersjach).		
Udziałowiec	UOB 01		
Wymagania powiązane	Brak		

Tabela 6.51 System operacyjny. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	ŚD 02	Priorytet	M
Nazwa	Przeglądarki internetowe		
Opis	Aplikacja powinna być kompatybilna z popularnymi przeglądarkami internetowymi.		
Kryteria akceptacji	Aplikacja będzie działała na przeglądarkach: Google Chrome, Mozilla Firefox, Opera, Microsoft Edge		
Udziałowiec	UOB 01		
Wymagania powiązane	Brak		

Tabela 6.52 przeglądarki internetowe. Źródło: opracowanie własne

KARTA WYMAGANIA			
Identyfikator:	ŚD 03	Priorytet :	M
Nazwa	Dostęp do sieci internetowej.		
Opis	Aplikacja potrzebuje działającego dostępu do sieci internetowej umożliwiającej komunikacji warstwy frontendu oraz backendu.		
Kryteria akceptacji	Aplikacja pozwala na poprawne korzystanie z funkcjonalności systemu przy aktywny połączeniu internetowy. Dostęp do internetu musi mieć zarówno urządzenie klienta jak i użytkownika.		
Udziałowiec	UOB 01		
Wymagania powiązane	Brak		

Tabela 6.53 przeglądarki internetowe. Źródło: opracowanie własne

Rozdział 7

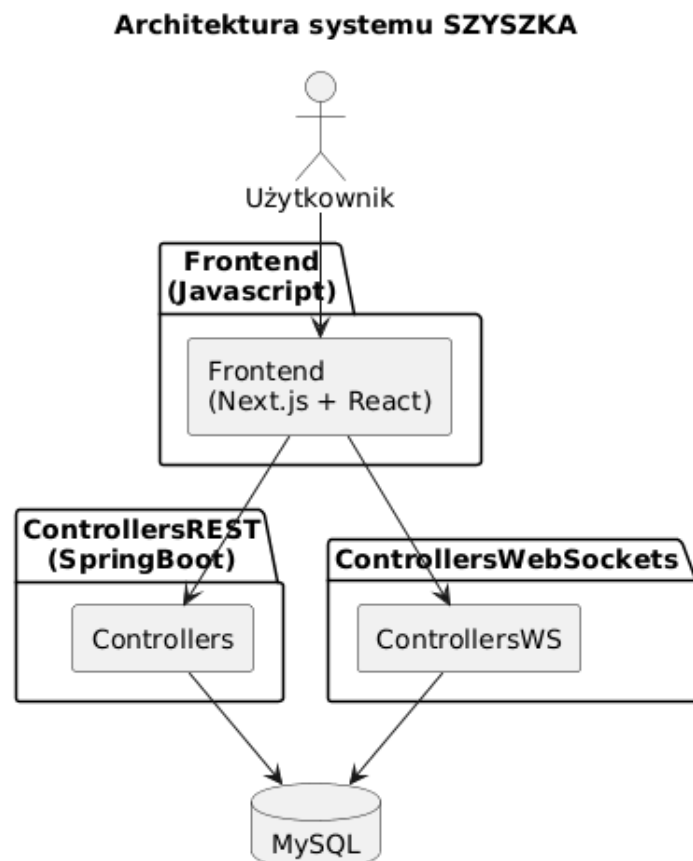
Projektowanie

7.1 Architektura

7.1.1 Ogólny opis architektury

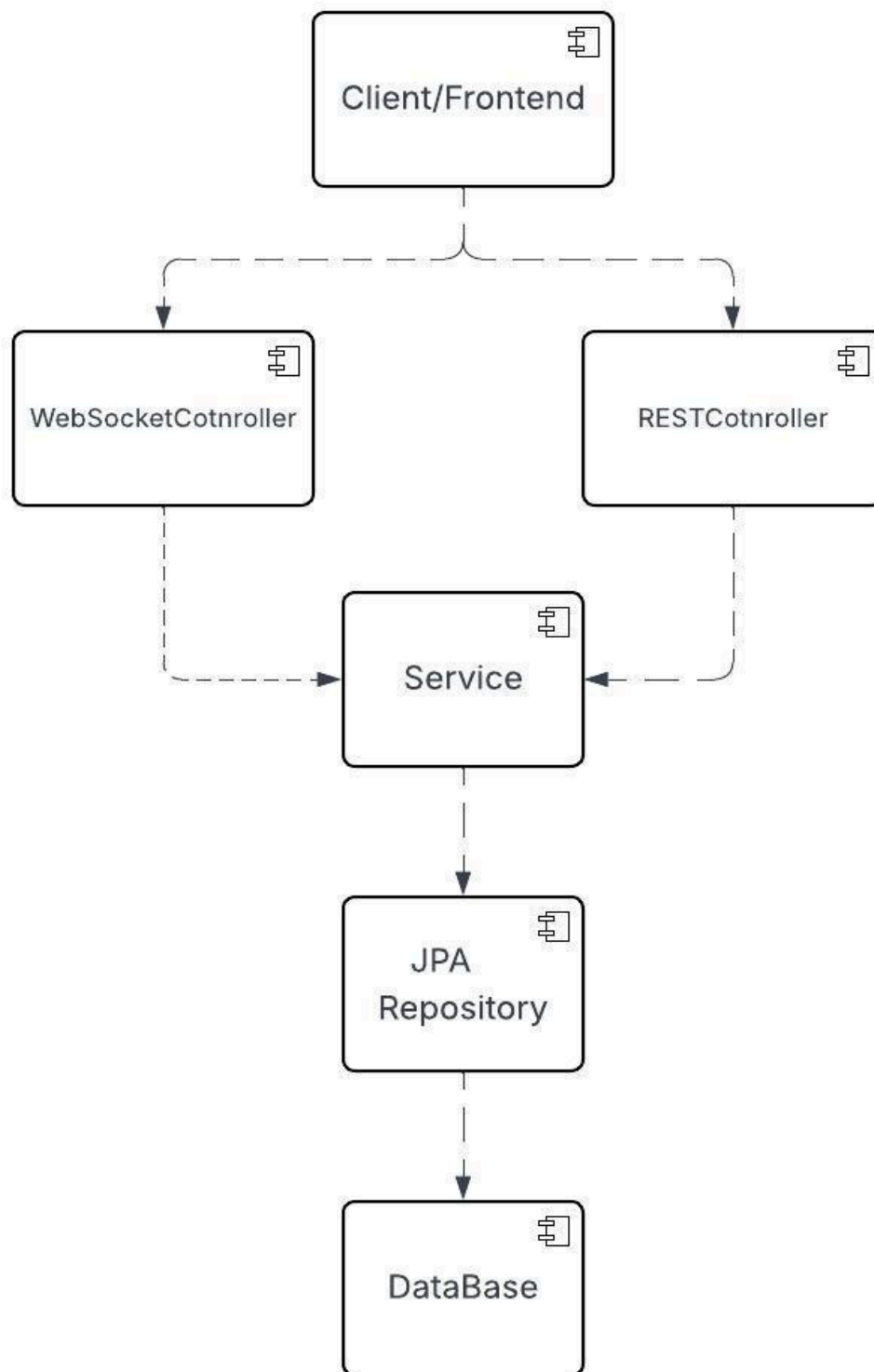
System został zaprojektowany w oparciu o architekturę warstwową, jej celem jest podzielenie odpowiedzialności pomiędzy poszczególne elementy aplikacji. Architektura systemu wydziela warstwy obsługi żądań, logiki biznesowej oraz dostępu do danych. Każda z tych warstw ma określony zakres funkcjonalności i komunikuje się tylko z warstwami które z nią sąsiadują. Ogranicza to zależności pomiędzy komponentami i ułatwia utrzymanie oraz rozwijanie systemu.

7.1.2 Diagram architektury



Rysunek 7.1 Diagram Architektury Źródło: opracowanie własne

7.1.3 Komponenty



Rysunek 7.2 Diagram Komponentów Źródło: opracowanie własne

Komponent bazy danych - System korzysta z relacyjnej bazy danych w której przechowuje trwałe dane aplikacji, takie jak listę członków, dane użytkowników, historię czatów postów. Baza danych zapewnia integralność danych oraz współpracuje z serwerem backendowy poprzez hibernate.

Frontend - Ten komponent odpowiada za interakcję z użytkownikiem oraz prezentację danych. Ma za zadanie pozwolić użytkownikowi na intuicyjne korzystanie z funkcjonalności systemu. Celem tego komponentu jest: obsługa interfejsu użytkownika, wysyłanie zapytań HTTP do backendu, połączenie z Websocket aby użytkownik miał dostęp do czatów w czasie rzeczywistym oraz odbieranie i wyświetlanie wiadomości wysyłanych przez serwer.

Kontorelly Rest - Obsługują zapytania które przychodzą do systemu oraz przekazują je do warstwy logiki biznesowej, same w sobie nie zawierają logiki biznesowej.

Kontroler WebSocket - obsługuje komunikację w czasie rzeczywistym pomiędzy klientem a serwerem. Ma za zadanie umożliwiać natychmiastową wymianę danych bez potrzeby wysyłania wielu zapytań przez klienta.

Serwisy - odpowiadają za całą logikę biznesową systemu oraz przetwarzanie danych według wymagań funkcjonalnych. Serwisy pośredniczą między kontrolerami a bazą danych

Obiekty transferu danych (DTO) - pozwalają na przekazywanie danych między warstwami aplikacji bez przesyłania niepotrzebnych do danego celu danych. Pozwala to kontrolować ilość informacji przekazywanych poza serwisem oraz zwiększa bezpieczeństwo danych

Repozytoria JPA - Odpowiadają za dostęp do danych w systemie, umożliwiają wykonanie podstawowych działań na danych np, zapis, odczyt, usuwanie, aktualizacje. Repozytoria mapują encję na tabele bazy danych co zapewnia spójność struktury systemu

7.1.4 Uzasadnienie wyboru architektonicznych

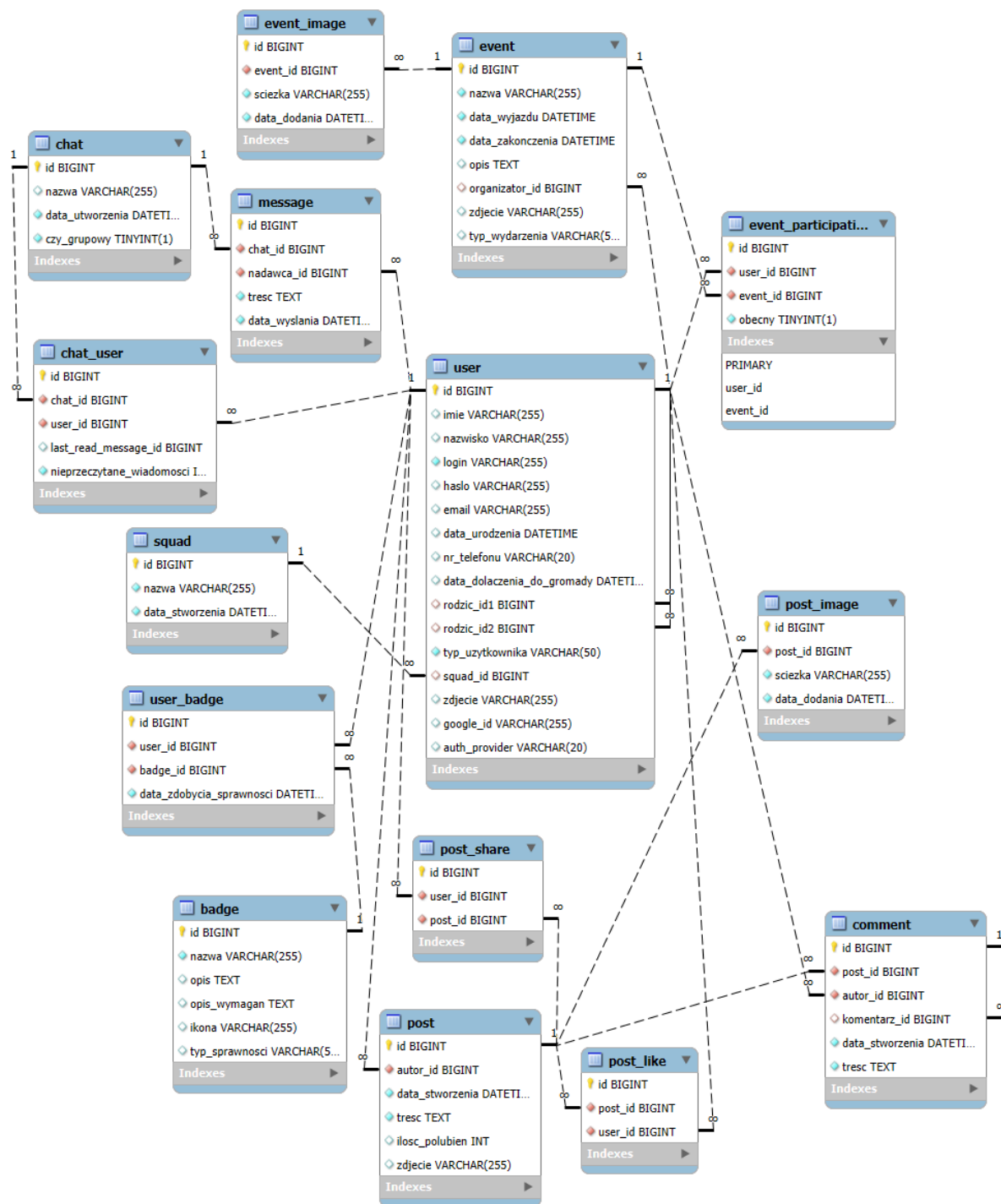
Architektura warstwowa pozwala na rozdzielenie odpowiedzialności na poszczególne komponenty systemu, co ułatwia jego utrzymanie, testowanie i rozwijanie. Wymienione wcześniej rozwiązania ograniczają zależności między warstwami dzięki czemu możliwe jest wprowadzanie zmian bez istotnego wpływu na pozostałe elementy systemu.

7.2 Projekt bazy danych

7.2.1 Cel i rola bazy danych

Baza danych stanowi kluczowy element systemu Szyszka, odpowiada za przechowywanie, organizację oraz udostępnianie danych wykorzystywanych przez aplikację. Od jej działania zależy również spójność integralność oraz bezpieczeństwo danych użytkowników oraz publikowanych treści.

Zastosowanie relacyjnej bazy danych umożliwia tworzenie zależności pomiędzy obiektami modelowymi systemu, takimi jak użytkownicy, posty, zdjęcia czy reakcje oraz realizację operacji CRUD (Create, Read, Update, Delete).



Rysunek 7.3 Projekt bazy danych. Źródło: opracowanie własne

7.2.2 System zarządzania bazą danych

W projekcie zastosowano relacyjny system zarządzania bazą danych (RDBMS), zgodny z językiem SQL. System ten został wybrany ze względu na:

- Szerokie wsparcie relacji i kluczy obcych które zapewnia.
- Możliwość definiowania ograniczeń w celu zapewnienia integralności danych.
- Wysoka stabilność i dojrzałość technologii.
- Łatwa integracja z warstwą backendową aplikacji.

Dostęp do bazy danych jest realizowany przez warstwę serwisową aplikacji z wykorzystaniem mechanizmów ORM (Object-Relational Programming), które umożliwiają automatyczne mapowanie encji aplikacji na struktury tabelaryczne w relacyjnej bazie danych.

7.2.3 Model danych

Model danych został zaprojektowany w oparciu o analizę wymagań funkcjonalnych systemu. Obejmuje on zestaw tabel reprezentujących główne byty domenowe oraz relacje pomiędzy nimi.

Do najważniejszych encji system należą:

- Użytkownik (user) - Przechowuje dane identyfikacyjne użytkowników, dane osobiste użytkownika oraz dane potrzebne do autoryzacji i uwierzytelniania..
- Post (post) - reprezentuje treści publikowane przez użytkowników.
- Polubienie posta (post_like) - umożliwia rejestrowanie reakcji użytkowników na treści zamieszczanych postów.
- Udostępnienie posta (post_share) - Powiązuje użytkowników z polubionymi przez nich postami.
- komentarz (comment) - umożliwia rejestrowanie reakcji użytkowników na treści w formie tekstowej..
- Wydarzenie (event) - reprezentuje wydarzenia organizowane w systemie wraz z powiązanymi danymi.
-

- Czat (chat) - Reprezentuje kanały komunikacji. Czat może być powiązany z grupą użytkowników jak szóstka lub służyć komunikacji pojedynczych użytkowników.
- Szóstka (squad) - Tabela opisuje jednostki organizacyjne w systemie. Zawiera informacje i opis grup użytkowników nazywanych szóstkami w celu identyfikacji przynależności zuchów do konkretnej grupy
- Sprawność (badge) - Zawiera informacje na temat sprawności zdobytych lub możliwych do zdobycia przez użytkownika.
- Uczestnictwo (event_participation) - Tabela reprezentuje uczestnictwa łącząc użytkownika z wydarzeniem.
- Wiadomość (message) - Tabela przechowuje dane wiadomości przypisanych do czatów i użytkowników.

7.2.4 Występujące relacje między tabelami:

- jeden-do-wielu - Są realizowane przez klucze obce, przykładowo jeden post posiadający wiele zdjęć oraz jeden użytkownik może być twórcą wielu postów.
- wiele-do-wielu - Relacje te są realizowane pośrednio przez tabele asocjacyjne. Przykładem jest relacja (czat-użytkownik) realizowana przez tablicę (czat_uzytkownik).

Relacje te są realizowane za pomocą kluczy obcych, co zapewnia spójność danych i eliminuje możliwość występowania osieroconych rekordów.

7.2.5 Przechowywanie plików multimedialnych

Pliki multimedialne (zdjęcia) nie są przechowywane bezpośrednio w bazie danych. W bazie przechowywana jest jedynie ścieżka do pliku zapisanego na serwerze. Pozwala to na:

- zmniejszenie ilości danych przechowywanych w bazie,
- poprawia wydajność operacji na bazie danych,
- oraz upraszcza zarządzanie plikami.

7.2.6 Bezpieczeństwo danych

Dostęp do bazy danych jest ograniczony wyłącznie do warstwy backendowej aplikacji, operacje dostępu do danych są w niej chronione mechanizmami autoryzacji użytkowników.

Wykorzystano również transakcje(Transactional) co zapewnia wykonanie operacji zapisu w pełni i zapobiega błędom niepełnego wykonania operacji.

7.3 Projekt wizualny systemu

Projekt wizualny systemu stanowi istotny element procesu tworzenia aplikacji, wpływający bezpośrednio na jej użyteczność, czytelność oraz odbiór przez użytkowników. Odpowiednio zaprojektowany interfejs użytkownika ułatwia nawigację, wspiera intuicyjne korzystanie z funkcjonalności oraz zwiększa komfort pracy z systemem.

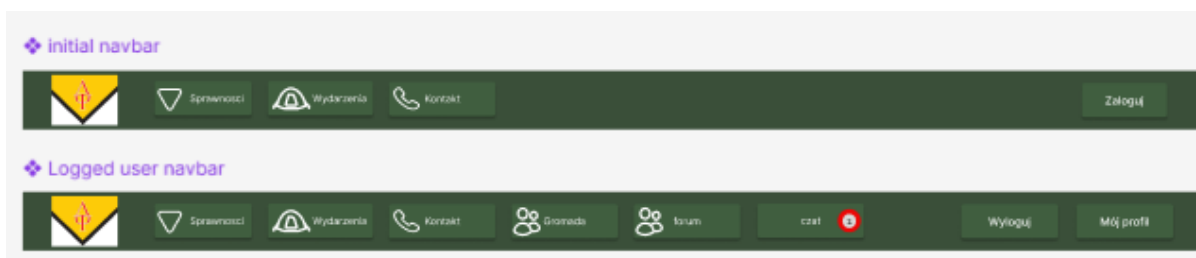
W niniejszym rozdziale przedstawiono proces projektowania warstwy wizualnej systemu z wykorzystaniem narzędzia Figma, które umożliwia tworzenie interaktywnych makiet, prototypów oraz spójnych komponentów interfejsu. Zastosowanie Figmy pozwala na szybkie iterowanie projektów, testowanie rozwiązań oraz efektywną współpracę zespołu projektowego.

Celem tego etapu było opracowanie spójnego i czytelnego interfejsu, dostosowanego do potrzeb użytkowników oraz założeń funkcjonalnych systemu, a także przygotowanie projektu, który może stanowić podstawę do dalszych prac implementacyjnych.

Poniżej znajdują się zdjęcia najważniejszych komponentów i funkcjonalności zrobionych w Figmie

7.3.1 Paski nawigacyjne

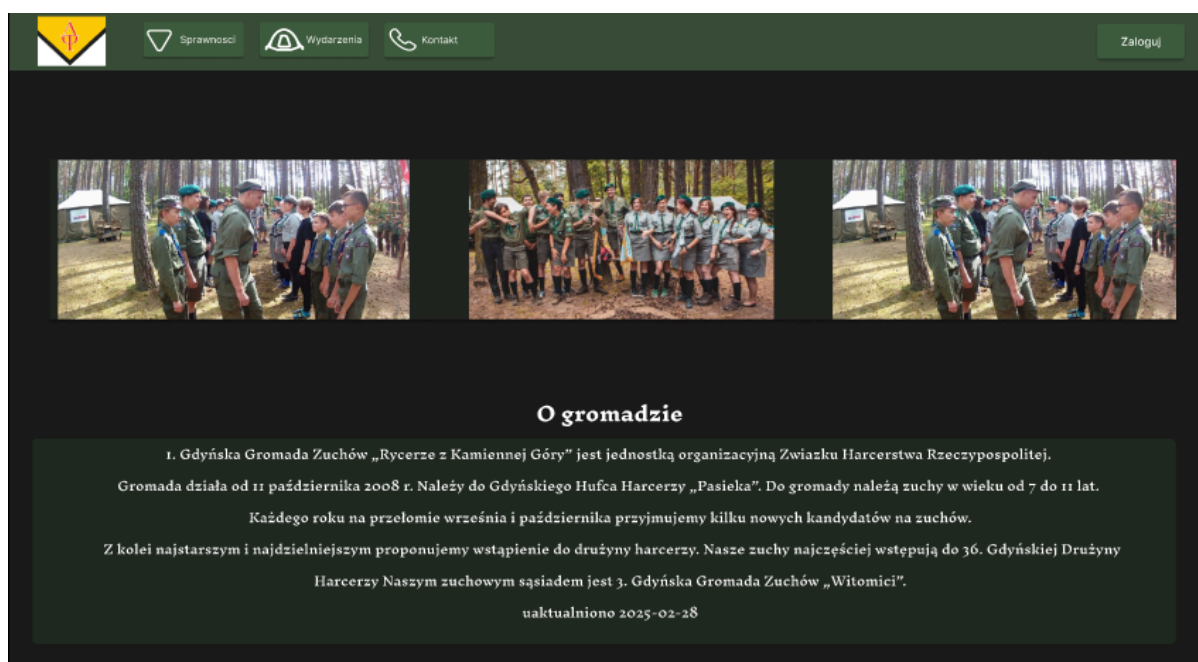
Do przekierowania pomiędzy stronami są głównie wykorzystywane paski nawigacyjne. Aplikacja ma 2 paski nawigacyjne: jeden do niezalogowanego użytkownika (górny pasek), a drugi do zalogowanego (dolny pasek). Każdy z tych pasków ma przyciski przekierowujące na jedną ze stron naszej aplikacji.



Rysunek 7.4 Prototyp. Paski nawigacyjne . Źródło: opracowanie własne

7.3.2 Prototyp. Strona główna

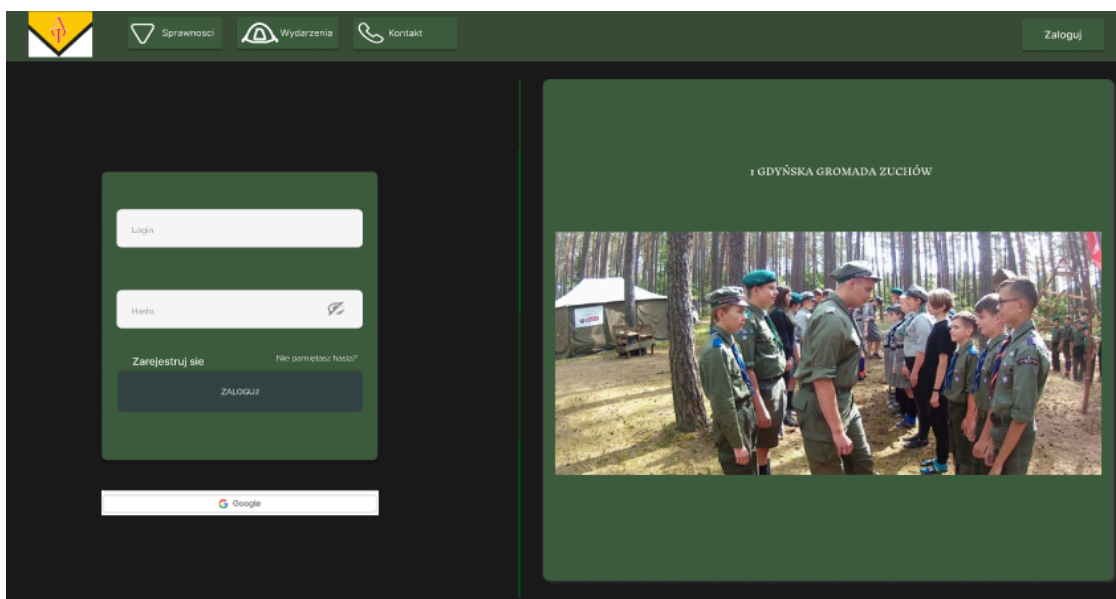
Przy wejściu do strony aplikacji użytkownik będzie widział stronę główną, na której jest karuzela obrazów oraz opis gromady. Kliknięcie na logo aplikacji powraca użytkownika do strony głównej.



Rysunek 7.5 Prototyp. Strona główna. Źródło: opracowanie własne

7.3.3 Prototyp. Strona logowania

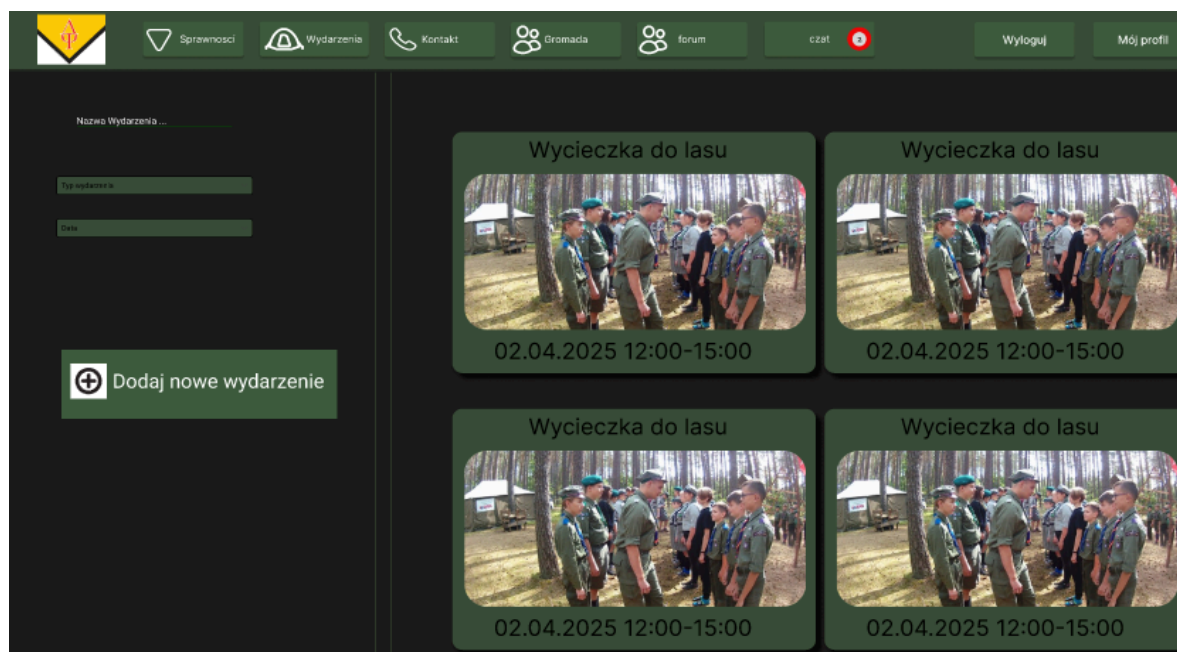
Kliknięcie na przycisk “zaloguj” w pasku nawigacyjnym przekierowuje do strony logowania. Na tej stronie przy wprowadzeniu prawidłowych danych lub logowania kontem google można zalogować się do aplikacji. Po kliknięciu “Zarejestruj się” użytkownik jest przekierowany do strony rejestracyjnej.



Rysunek 7.6 Prototyp. Strona logowania. Źródło: opracowanie własne

7.3.4 Prototyp. Strona wydarzeń

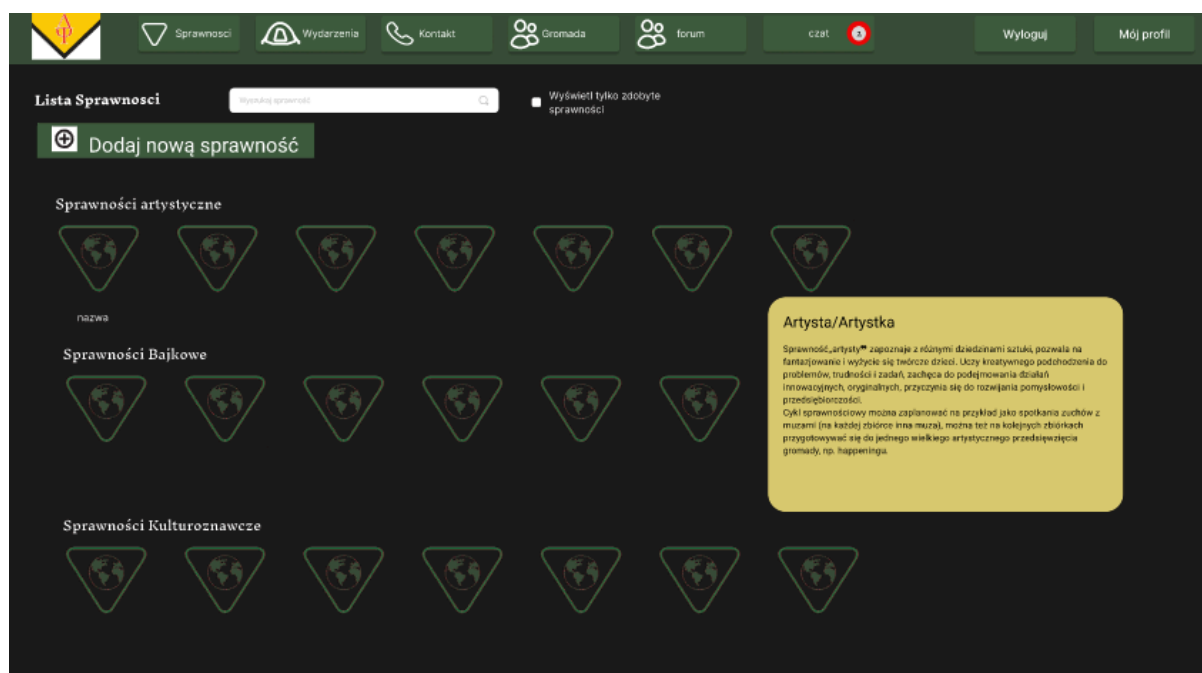
Przycisk “wydarzenia” przekierowuje do strony wydarzeń. Na niej są umieszczone wydarzenia od najnowszych do najstarszych, a także ich filtracja według nazwy, typu oraz daty. Przycisk do dodawania wydarzeń jest widoczny tylko administratorom.



Rysunek 7.7 Prototyp. Strona wydarzeń. Źródło: opracowanie własne

7.3.5 Prototyp. Strona sprawności

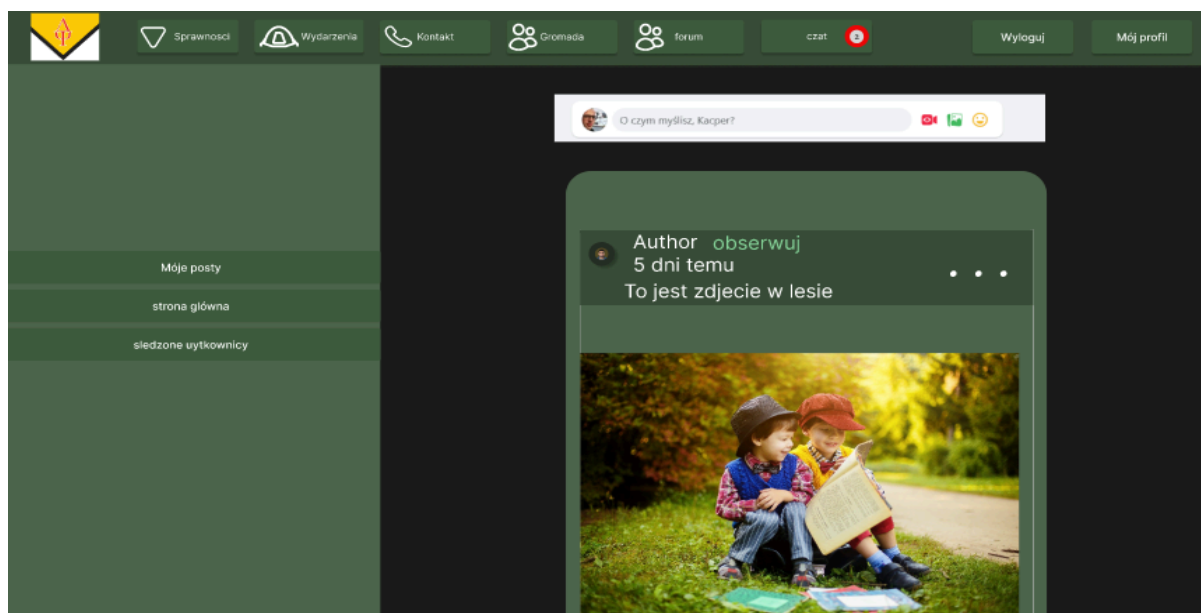
Przycisk “Sprawności” przekierowuje do strony wydarzeń. Na niej są umieszczone sprawności posortowane według typu, a także ich filtracja według nazwy. Przy najechaniu na ikonę sprawności jest widoczny jej opis. Przycisk do dodawania sprawności jest widoczny tylko administratorom.



Rysunek 7.8 Prototyp. Strona Sprawności. Źródło: opracowanie własne

7.3.6 Prototyp. Strona forum

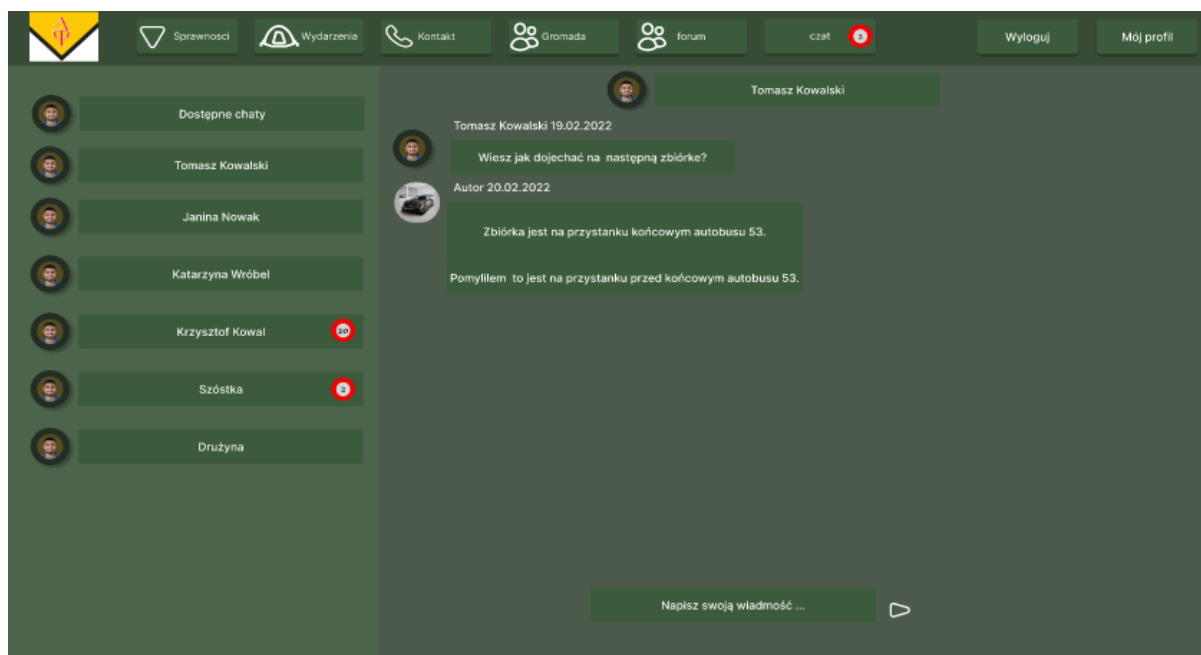
Przycisk “forum” przekierowuje do strony forum. Na niej są wyświetlone posty od najnowszych do najstarszych, a także przycisk, który otwiera okno do tworzenia nowego posta. Możesz też przejść do strony, która pokazuje tylko posty danego użytkownika, a także do strony, która pokazuje użytkowników, których posty obserwujesz



Rysunek 7.9 Prototyp. Strona forum. Źródło: opracowanie własne

7.3.7 Prototyp. Strona czaty

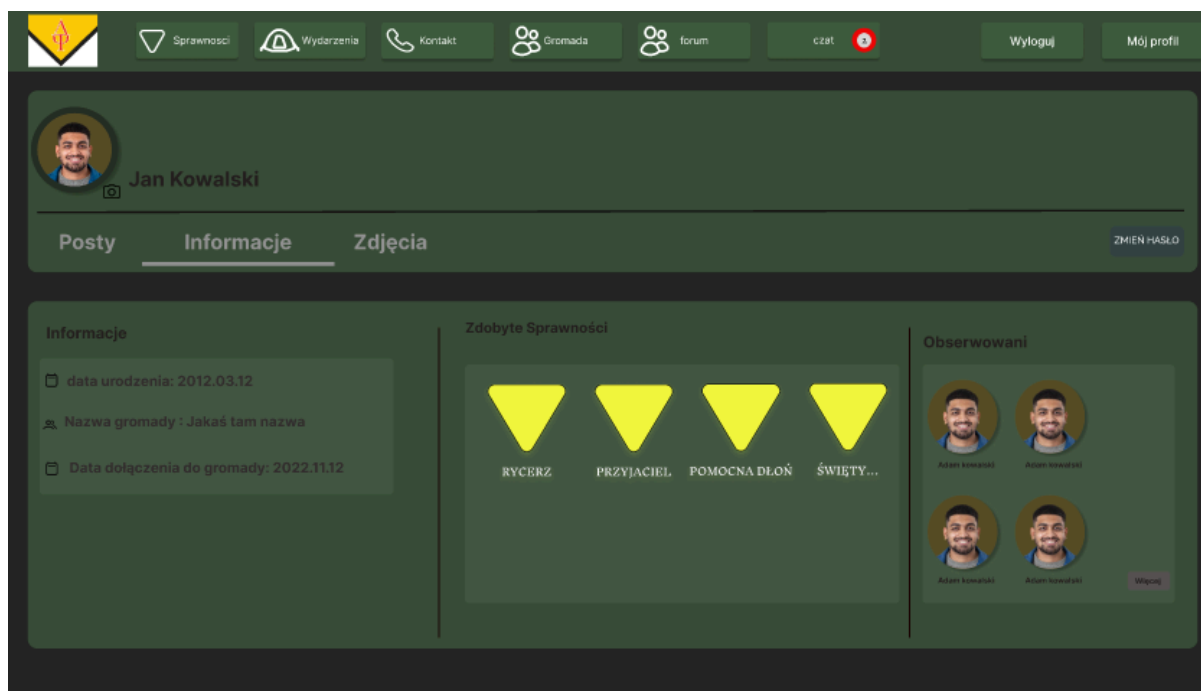
Przycisk “Czat” przekierowuje do strony czatów. Na niej są w lewej stronie umieszczone nazwy wszystkich czatów w których jesteś uczestnikiem, a w prawej stronie czat który teraz obserwujesz.



Rysunek 7.10 Prototyp. Strona Czatów. Źródło: opracowanie własne

7.3.8 Prototyp. Strona profilu

Przycisk “mój profil” przekierowuje do strony profilu danego użytkownika. Na niej jest informacja o użytkowniku, możliwość jej zmiany oraz zdobyte sprawności, opublikowane posty i zdjęcia



Rysunek 7.11 Strona profilu. Źródło: opracowanie własne

Rozdział 8

Implementacja

Rozdział przedstawia sposób implementacji systemu informatycznego którego projekt i założenia zostały opisane we wcześniejszych częściach tego dokumentu. W tym rozdziale opisane zostały kluczowe funkcjonalności systemu oraz zastosowane rozwiązania technologiczne zarówno po stronie serwera jak i klienta.

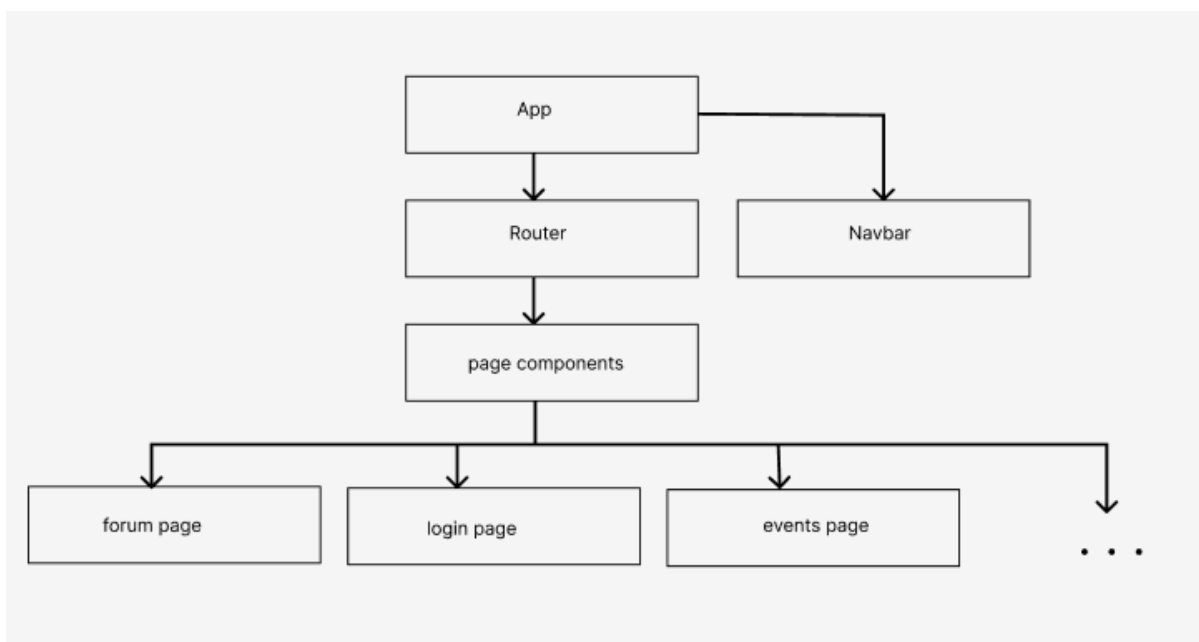
System został zaimplementowany w architekturze klient-serwer i składa się z dwóch głównych części: aplikacji frontendowej oraz aplikacji backendowej. Część frontendowa odpowiada za interakcję z użytkownikiem, prezentację danych oraz obsługę logiki interfejsu, natomiast część backendowa realizuje logikę biznesową systemu, zarządza danymi oraz odpowiada za bezpieczeństwo systemu i komunikację z bazą danych.

Komunikacja między frontendem a backendem odbywa się za pomocą protokołu HTTP. Większość danych przesyłane jest przy użyciu architektury REST API w formacie JSON, w przypadku zdjęć zastosowano format multipart/form-data. Autoryzacja użytkowników realizowana jest z wykorzystaniem mechanizmu tokenów JWT.

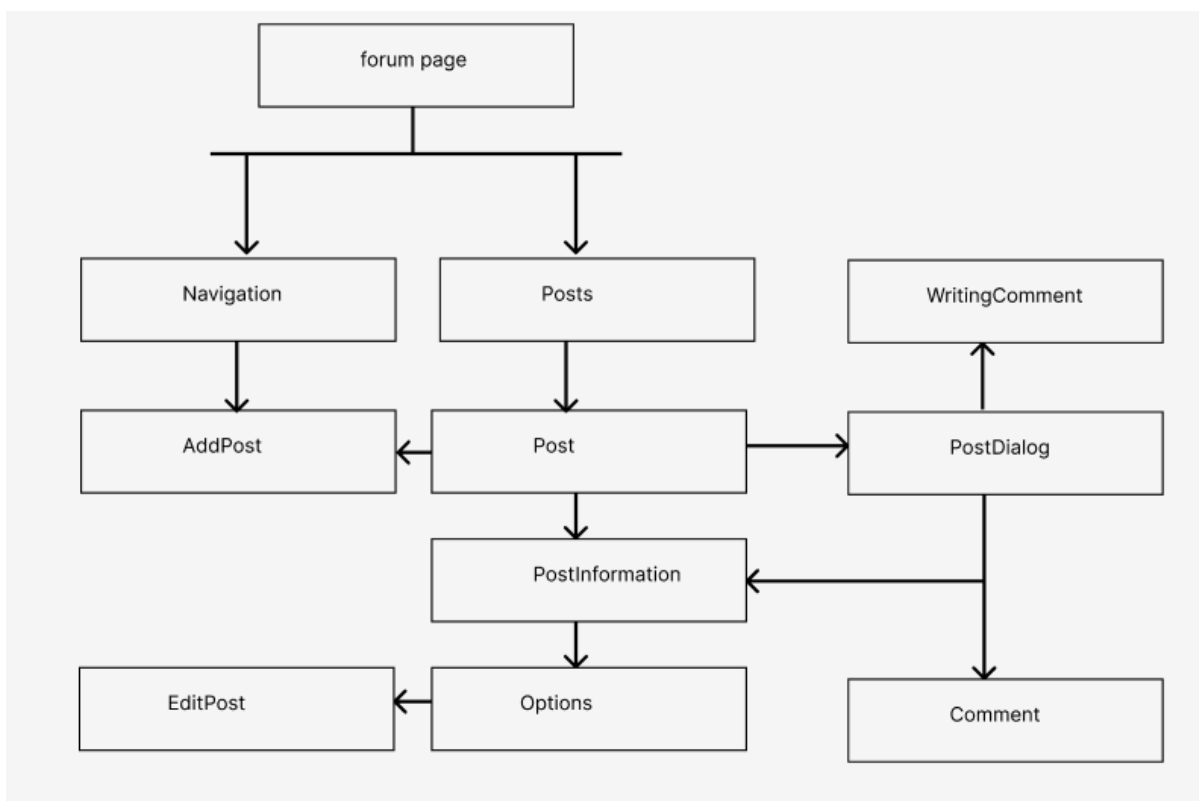
W dalszej części rozdziału szczegółowo opisano implementację frontendu systemu (rozdział 8.2), oraz implementację backendu wraz z jego architekturą i kluczowymi komponentami w (rozdział 8.3).

8.1 Architektura

Do stworzenia warstwy frontendowej została wykorzystana architektura oparta na komponentach. Oznacza to, że frontend jest podzielony na niezależne komponenty, z których każdy odpowiada za generowanie określonego widoku interfejsu użytkownika. Komponenty te mogą być następnie składane oraz renderowane niezależnie, co ułatwia ponowne wykorzystanie kodu i jego utrzymanie.



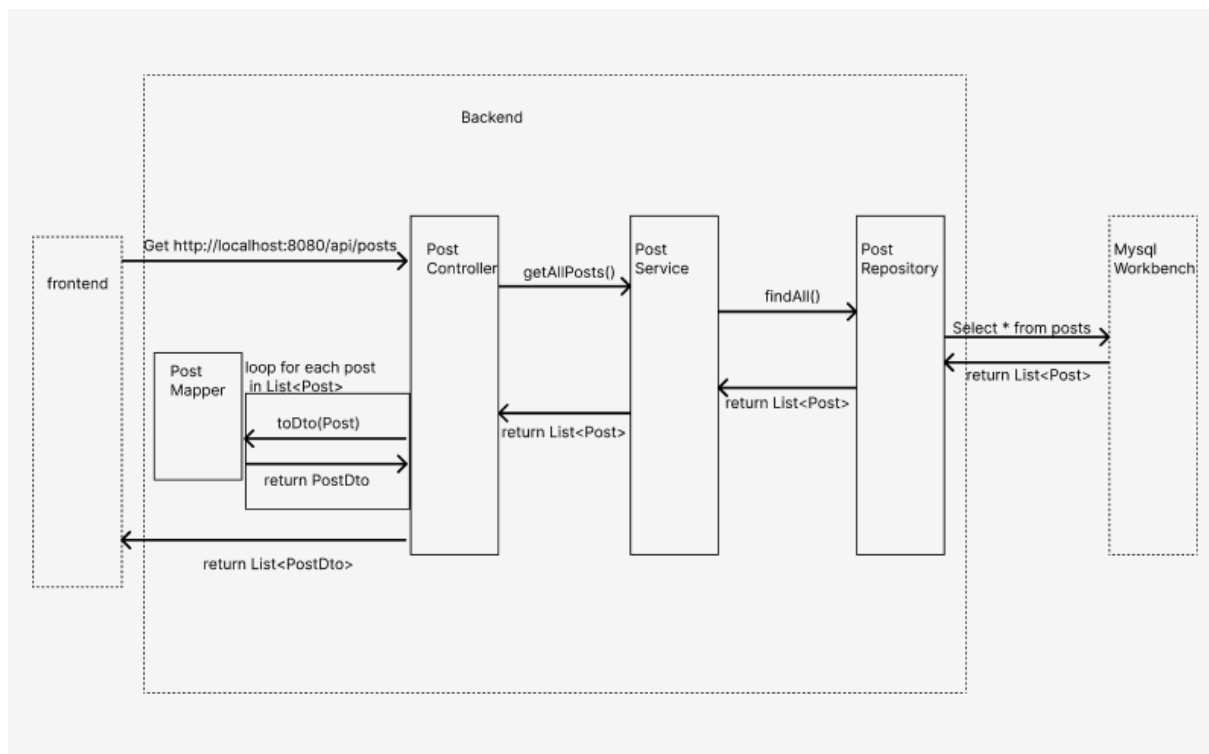
Rysunek 8.1 Diagram komponentów. Źródło: opracowanie własne



Rysunek 8.2 Diagram komponentów. Forum page. Źródło: opracowanie własne

Do stworzenia warstwy backendowej została wykorzystana architektura warstwowa. Backend został podzielony na warstwy, z których każda posiada jasno określoną

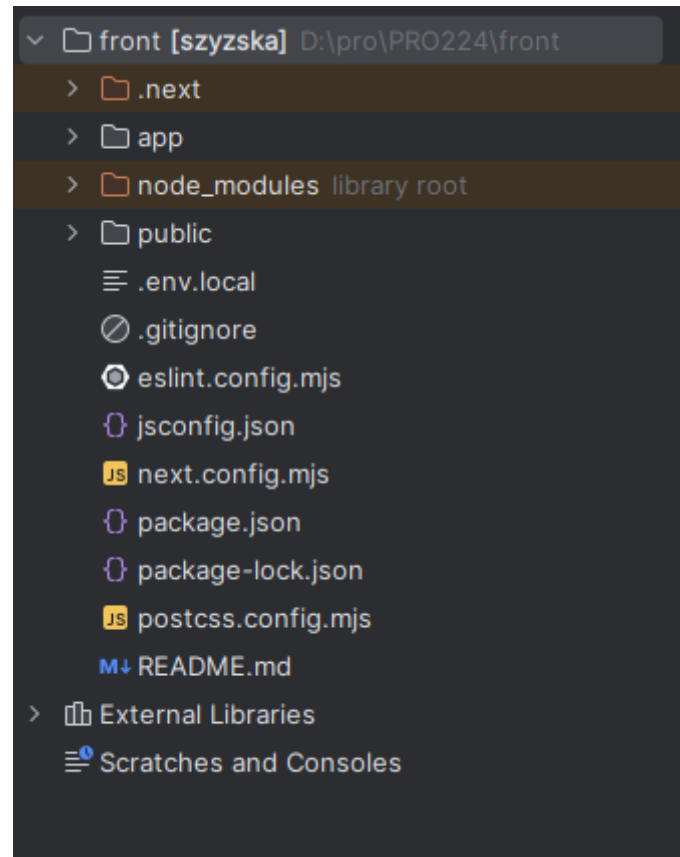
odpowiedzialność. Warstwa kontrolerów odpowiada za obsługę zapytań HTTP oraz przekazywanie ich do warstwy serwisów. Warstwa serwisów zawiera całą logikę aplikacyjną. Warstwa repozytoriów odpowiada za komunikację z bazą danych. Dodatkowo w aplikacji wykorzystano obiekty DTO (Data Transfer Object), które służą do bezpiecznego i kontrolowanego przekazywania danych pomiędzy warstwami aplikacji oraz pomiędzy backendem a frontendem, oddzielając wewnętrzny model domenowy od formatu danych przesyłanych w odpowiedziach HTTP.



Rysunek 8.3 Diagram sekwencji. Źródło: opracowanie własne

8.2 Implementacja frontendu

8.2.1 Struktura plików



Rysunek 8.4 Struktura plików. Źródło: opracowanie własne

Projekt został zaprojektowany w frameworku Next.js za pomocą narzędzia `npx create-next-app` [8]. To pozwoliło od początku mieć dobrą i zrozumiałą strukturę plików.

Katalog projektu zawiera takie foldery i pliki jak:

- Readme.md to plik, w którym jest opis konfiguracji i startu projektu
- package.json to plik, zawierający zależności, w którym opisane wszystkie potrzebne biblioteki i skrypty
- package-lock.json to plik, zawierający zależności, w którym opisane wszystkie potrzebne biblioteki, a także zależności wygenerowane przez Next.js
- postcss.config.mjs to plik konfiguracyjny postcss używany przez Tailwind CSS
- next.config.mjs to plik konfiguracyjny Next.js
- jsconfig.config.mjs to plik konfiguracyjny JS
- eslint.config.mjs to plik konfiguracyjny ESLint
- .gitignore to plik, opisujące foldery i pliki które nie zostają wrzucone do gita, bo mają za duży rozmiar, jak node_modules, lub zawierają tajne dane, jak .env.local

- .env.local to plik zawierający tajne dane, takie jak klucz Google, lub dane które mogą zmienić się podczas tworzenia projektu, takie jak port backendu
- public to plik do zdjęć przechowywanych w projekcie
- .next i node_modules to foldery z bibliotekami
- app to folder, zawierający logikę aplikacji

8.2.2 Wykorzystane biblioteki

```
"dependencies": {
  "@stomp/stompjs": "^7.2.1",
  "formik": "^2.4.6",
  "next": "15.3.0",
  "next-auth": "^4.24.13",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-icons": "^5.5.0",
  "react-virtuoso": "^4.17.0",
  "react-window": "^2.2.3",
  "sockjs-client": "^1.6.1",
  "yup": "^1.6.1"
},
"devDependencies": {
  "@eslint/eslintrc": "^3",
  "@tailwindcss/postcss": "^4",
  "eslint": "^9",
  "eslint-config-next": "15.3.0",
  "tailwindcss": "^4"
}
```

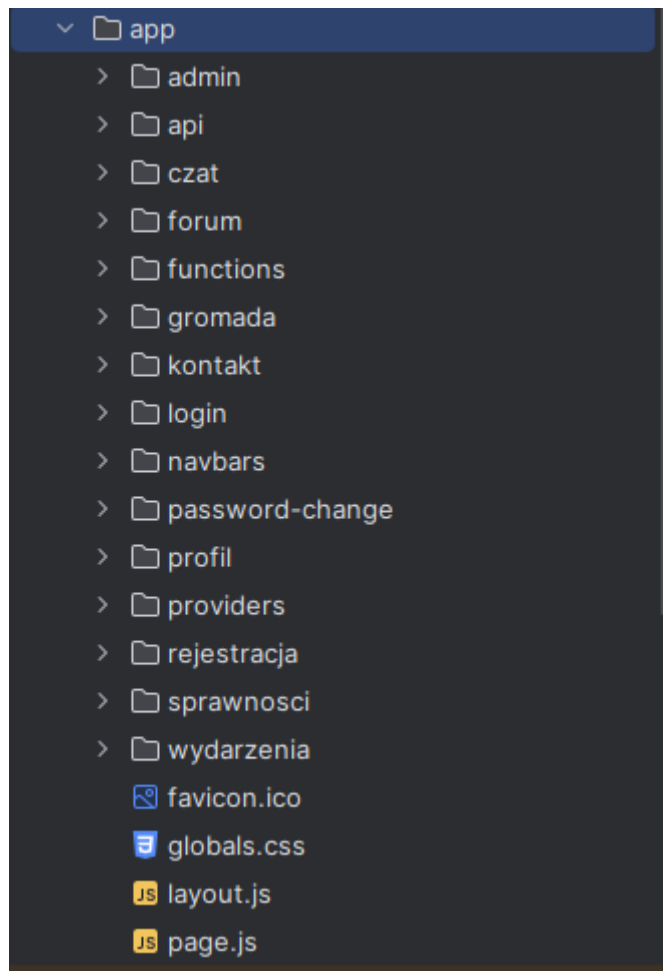
Rysunek 8.5 Package.json. Źródło: opracowanie własne

- biblioteki formik i yup zostały wykorzystane do stworzenia form logowania rejestracji i innych. Biblioteka formik pozwala na śledzenie stanu wszystkich znajdujących w formie pól. Biblioteka yup jest wykorzystywana do walidacji danych wewnątrz formy, na przykład sprawdzając czy pole nie jest puste.
- biblioteki sockjs-client @stomp/stompjs są potrzebne do wdrożenia mechanizmu Websocket, który pozwala na nasłuchiwanie na porcie backendu i natychmiast otrzymanie wysłanych przez niego komunikatów

- biblioteka react-virtuoso jest wykorzystywana do stworzenia list wirtualnych, które przyspieszają ładowanie stron przy dużych ilościach danych
- biblioteka react-icons zawiera w sobie wiele ikon, część z których została użyta w aplikacji
- biblioteka react jest potrzebna do tworzenia komponentów i zarządzania stanem komponentów
- biblioteka react-dom jest potrzebna do renderowania stworzonych komponentów w przeglądarce
- biblioteka next jest używana przez framework Next.js

8.2.3 Folder app

Do poruszania się po stronach w projekcie jest używany mechanizm App Router, który pozwala na zrozumianą rozmieszczenie stron [9]. Każdy folder w app zawierający plik page.js automatycznie tworzy nową stronę zawierającą treść pliku page.js pod adresem /nazwa folderu. Pliki layout.js pozwalają na manipulowanie układem stron w folderze w którym znajdują, a także we wszystkich podfolderach. Favicon.ico to ikona aplikacji, która jest wyświetlana w przeglądarce.



Rysunek 8.6 Folder app. Źródło: opracowanie własne

8.2.4 Layout

Do zrobienia stylu stron został wykorzystany Tailwind CSS [9]. To pozwala na przejrzyste ustawienie wyglądu komponentów poprzez className. Plik layout.js (Rysunek 8.3) pobiera plik global.css, zawierający implementacje Tailwind CSS (Rysunek 8.4), wykorzystuje komponent GlobalProvider, zawierający kluczowe, często używane funkcjonalności, które są następnie udostępniane wszystkim komponentom aplikacji. Plik ten odpowiada również za wyświetlanie komponentu <Navbar /> na każdej stronie systemu. ‘use client’ oznacza, że ten komponent jest wywoływany tylko po stronie klienta.

```

'use client'
import './globals.css';
import GlobalProvider from '@app/providers/GlobalProvider';
import Navbar from '@app/navbars/Navbar';

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body >
        <GlobalProvider>
          <Navbar/>
          {children}
        </GlobalProvider>
      </body>
    </html>
  );
}

```

Rysunek 8.7 Layout. Źródło: opracowanie własne

```

@import "tailwindcss";
@tailwind base;

@tailwind components;
@tailwind utilities;

```

Rysunek 8.8 Tailwind CSS. Źródło: opracowanie własne

8.2.5 Hooki

Do przechowywania stanów danych oraz reagowanie na zmiany w aplikacji są wykorzystywane hooki wbudowane w React. Spośród nich są useContext, który pozwala na przechowywanie stanu i funkcji przy zmianie aktywnego komponentu, useState, który przechowuje wartość, useRef który przechowuje referencje do komponentu, useEffect, który pozwala na wywołanie funkcji od razu po załadowaniu komponentu, useRouter, który umożliwia poruszanie po stronach bez odnawiania strony.

Na poniższym zdjęciu znajdują się stany i efekty, używane w komponencie GlobalProvider, dostępne na każdej stronie aplikacji. Przy załadowaniu aplikacja wywołuje funkcję get_me(), która próbuje pobrać użytkownika i ustawia loading na false. Jeżeli nie udało się pobrać

użytkownika i klient znajduje się na stronie potrzebującej autoryzacji jest on przekierowywany do strony logowania.

```
const [loading, setLoading] = useState(true);
const [user, setUser] = useState(null)
const [edit, setEdit] = useState({})
const router = useRouter()
useEffect(()=>{
  get_me()
},[])

useEffect(() => {
  if(loading)return
  if(!user){
    const path=window.location.pathname if(!(path==="/login"||path==="/rejestracja
    ||path==="/"||path==="/kontakt"||path.startsWith("/sprawnosci")||path.startsWith("/wydarzenia")))
    router.replace("/login")
  }
}, [user,loading]);
```

Rysunek 8.9 Hooks. Źródło: opracowanie własne

8.2.6 Komunikacja z backendem

Komunikacja z backendem została zrobiona poprzez funkcję fetch, która umożliwia wysyłanie żądań HTTP na wskazany adres URL przekazany jako pierwszy parametr, wraz z dodatkowymi opcjami określonymi w drugim parametrze.

W poniższym kodzie jest wysłano zapytanie typu GET do endpointu `/api/uzytkownicy/me` backendu zapisanego w .env.local. Credentials:"include" pozwala na manipulacje ciasteczkami, w których jest przechowywany token JWT, co pozwala na autentykację klienta po stronie backendu.

```

const get_me=()=>{
  const me=async ()=>
  {
    try {
      const res = await fetch(
        `${process.env.NEXT_PUBLIC_BACKEND_PORT}/api/uzytownicy/me`,
        { credentials: "include" }
      );

      if (!res.ok) {
        setUser(null);
        setLoading(false)
      } else {
        const data = await res.json();
        setUser(data);
        setLoading(false)
      }
    } catch {
      setUser(null);
      setLoading(false)
    }
  }
  me()
}

```

Rysunek 8.10 Komunikacja z backendem. Źródło: opracowanie własne

8.2.7 Pasek nawigacyjny

Do poruszania się po stronach jest wykorzystywany pasek nawigacyjny, w którym znajdują się przyciski z linkami do poszczególnych stron. W pasku nawigacyjnym zawsze znajdują się linki do strony głównej, kontaktu, wydarzeń i sprawności. Jeżeli użytkownik nie jest zalogowany w pasku też znajduje się link do logowania. W innym przypadku przypadku do paska są dodawane linki do forów, czatów, wylogowania i profilu użytkownika. Jeżeli użytkownik jest drużynowym lub przybocznym też widzi link do strony administracyjnej

```

const buttons = [
  { label: "sprawności", img: "/images/navbar/sprawnosci_logo.png", path: "/sprawnosci" },
  { label: "wydarzenia", img: "/images/navbar/wydarzenia_logo.png", path: "/wydarzenia" },
  { label: "kontakt", img: "/images/navbar/kontakt_logo.png", path: "/kontakt" },

];

if(user && user.login){
  buttons.push({ label: "forum", img: "/images/navbar/forum_logo.png", path: "/forum" })
  buttons.push({ label: "czat", icon: <FaMessage />, path: "/czat" })
}

const rightButtons = user && user.login ? [
  { label: "log out", icon: <FaSignOutAlt />, action: logOut },
  { label: "mój profil", icon: <FaAddressCard />, path: "/profil" },
]: [ { label: "log in", icon: <FaSignInAlt />, path: "/login" } ];

if(user && (user.typUzytkownika === "Druzynowy" || user.typUzytkownika === "Przyboczny"))
  buttons.push({ label: "gromada", icon: <FaPerson />, path: "/admin/gromada" })

```

Rysunek 8.11 Pasek nawigacyjny-przyciski. Źródło: opracowanie własne

```

<nav className="hidden fixed top-0 w-full bg-[#3A4F39] h-[50px] z-50 sm:flex justify-between
items-center overflow-x-auto overflow-y-hidden whitespace-nowrap px-2">
  <div className="flex items-center space-x-2">
    <button onClick={{e} => pushClick(e, "/")}>
      
    </button>

    {buttons.map((btn, idx) => (
      <button
        key={idx}
        onClick={{e} => pushClick(e, btn.path)}
        className="flex items-center bg-[#405E3F] shadow-md px-[6px] py-1"
      >
        {btn.img} && <img src={btn.img} className="h-8 w-10" alt="" />
        {btn.icon} && <span className="h-8 w-8 flex items-center justify-center">{btn.icon}</span>
        <span className="ml-1 hidden lg:inline">{btn.label}</span>
      </button>
    ))}
  </div>

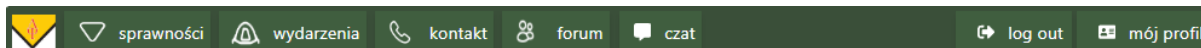
  <div className="flex items-center space-x-1">
    {rightButtons.map((btn, idx) => (
      <button
        key={idx}
        onClick={{e} => btn.action ? btn.action(e) : pushClick(e, btn.path)}
        className="flex items-center justify-center bg-[#405E3F] shadow-md px-2 py-1"
      >
        {btn.icon} && <span className="h-8 w-10 flex items-center justify-center">{btn.icon}</span>
        <span className="hidden lg:inline">{btn.label}</span>
      </button>
    ))}
  </div>
</nav>

```

Rysunek 8.12 Pasek nawigacyjny-renderowanie. Źródło: opracowanie własne



Rysunek 8.13 Pasek niezalogowanego użytkownika. Źródło: opracowanie własne



Rysunek 8.14 Pasek zalogowanego użytkownika. Źródło: opracowanie własne

8.2.8 Strona główna

Strona główna pokazuje informacje o gromadzie. Na niej znajduje się karuzela obrazków które zmieniają się każde 5 sekund i opis gromady.

```
useEffect(() => {
  const timeout = setTimeout(updateCarousel, 5000);
  return () => clearTimeout(timeout);
}, [currentIndex]);

function updateCarousel() {
  const offset = -((currentIndex + 1) % (items.length - 1)) * 50;
  setCurrentIndex(prev => prev + 1);
  if (carouselInner.current) {
    carouselInner.current.style.transform = `translateX(${offset}%)`;
  }
}

return (
  <div className="pt-[50px]">

    { /* CAROUSEL */ }
    <div className="relative w-full overflow-hidden">
      <div
        ref={carouselInner}
        className="flex transition-transform duration-500 ease-in-out"
      >
        {items.map(item => (
          <img
            key={item.id}
            src={item.src}
            alt={item.alt}
            className="min-w-[50%] p-2 object-cover"
          />
        ))}
      </div>
    </div>
  </div>
)
```

```

    </div>
  </div>

  { /* TEKST */ }
  <div className="text-center mx-[5%] mt-6 p-5 bg-[#222822] text-lg text-white">
    <p className="text-2xl mb-4 bg-[#1A1919] py-2">O gromadzie</p>
    {text.map((l, i) => (
      <p key={i} className="mb-2">
        {l.linia}
      </p>
    ))}
  </div>

</div>
);

```

Rysunek 8.15. Strona główna. Źródło: opracowanie własne

8.2.9 Logowanie

Do obsługi formularzy wykorzystano bibliotekę Formik [11]. Umożliwia ona walidację danych w czasie rzeczywistym za pomocą atrybutu validationSchema, zarządzanie stanem formularza oraz obsługę zdarzenia onSubmit, które wywoływane jest bezpośrednio po kliknięciu przycisku wysyłania formularza. Dodatkowo Formik posiada wbudowane mechanizmy sprawdzające, czy formularz został zmodyfikowany (dirty) oraz czy walidacja zakończyła się powodzeniem (isValid). Pozwala to na dynamiczne włączanie i wyłączanie przycisku zatwierdzającego formularz.

Formularz logowania (rysunek 8.13) zawiera dwa pola: login oraz hasło, które po kliknięciu przycisku „Zaloguj” są przekazywane do funkcji login. Funkcja login (rysunek 8.14) wysyła zapytanie do backendu z wykorzystaniem funkcji fetch. Jeżeli odpowiedź nie zawiera błędu, pobierane są dane użytkownika, a następnie następuje przekierowanie na stronę /profile.

Na stronie znajduje się również przycisk logowania za pomocą konta Google, który odbiera od usługi Google token idToken i zapisuje go w sesji (rysunek 8.15). Token ten jest następnie przesyłany do backendu w celu uwierzytelnienia użytkownika.

```

const { pushClick, logIn, googleLogin, user } = useContext(GlobalContext);
const { data: session, status } = useSession();
const sentRef = useRef(false);

useEffect(() => {
  if (!user) return;
  pushClick(null, "/profile");
}, [user]);

useEffect(() => {
  if (status === "authenticated" && !sentRef.current) {
    if (!session?.googleIdToken) return;
    googleLogin(session.googleIdToken);
    sentRef.current = true;
  }
}, [status, session]);

.....

<div className="flex-[2] min-w-[300px] flex justify-center items-center">
  <Formik
    initialValues={{ login: "", haslo: "" }}
    validationSchema={Yup.object({
      login: Yup.string().required("Login jest wymagany"),
      haslo: Yup.string().required("Haslo jest wymagane"),
    })}
    onSubmit={(values, { resetForm }) => {
      logIn(values);
      resetForm();
    }}
  >
    {({ dirty, isValid }) => (
      <Form className="w-[80%] max-w-md rounded-xl bg-[#405E3F] p-8 shadow-lg flex flex-col gap-3">
        <p className="text-white font-semibold">Login</p>
        <Field
          name="login"
          className="rounded-md px-3 py-2 outline-none"
          placeholder="napisz login"
        />
        <ErrorMessage name="login" component="div" className="text-red-400 text-sm"/>
      </Form>
    )}
  </div>

```

```

<p className="text-white font-semibold mt-2">Hasło</p>
<Field
  type="password"
  name="hasło"
  className="rounded-md px-3 py-2 outline-none"
  placeholder="napisz hasło"
/>
<ErrorMessage name="hasło" component="div" className="text-red-400 text-sm"/>

<div className="flex justify-between text-sm text-gray-200 mt-2">
  <button
    type="button"
    onClick={e => pushClick(e, "/rejestracja")}
    className="hover:underline"
  >
    Zarejestruj się
  </button>
  <button
    type="button"
    onClick={e => pushClick(e, "/password-change")}
    className="hover:underline"
  >
    Nie pamiętasz hasła?
  </button>
</div>

<button
  type="submit"
  disabled={!dirty || !isValid}
  className="mt-4 rounded-md bg-[#354545] py-2 text-white
disabled:opacity-50 disabled:cursor-not-allowed
hover:bg-[#2d3e3e]"
>
  Log In
</button>

<button
  type="button"
  onClick={() => signIn("google")}

```



```

        className="mt-2 rounded-md border border-white py-2 text-white
        hover:bg-white hover:text-black transition"
      >
        Sign in with Google
      </button>
    </Form>
  )}
</Formik>
</div>

```

Rysunek 8.16. Strona logowania. Źródło: opracowanie własne

```

const login=(values)=>{
  const f=async (values)=>{
    await fetch(`${process.env.NEXT_PUBLIC_BACKEND_PORT}/api/auth/login`,
      {
        method:"POST",
        headers: {'Content-type':"application/json"},
        credentials: "include",
        body:JSON.stringify(
          values
        )
      })
    .then(()=>{
      get_me()
    })
    .catch(err=>alert("wystąpił błąd przy logowaniu"))
  }
  f(values)
}

```

Rysunek 8.17. funkcja login. Źródło: opracowanie własne

```

import NextAuth from 'next-auth';
import GoogleProvider from 'next-auth/providers/google';

export const authOptions = {
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    }),
  ],
  secret: process.env.NEXTAUTH_SECRET,
  callbacks: {
    async jwt({ token, account }) {
      if (account) {
        token.googleIdToken = account.id_token;
      }
      return token;
    },
    async session({ session, token }) {
      // przekazujemy do sesji w przeglądarce
      session.googleIdToken = token.googleIdToken;
      return session;
    },
  },
};

const handler = NextAuth(authOptions);
export { handler as GET, handler as POST };

```

Rysunek 8.18. Logowanie Google. Źródło: opracowanie własne

8.2.10 Forum

W celu zwiększenia wydajności podczas ładowania dużych list zastosowano bibliotekę React Virtuoso (Rysunek 8.16), która pozwala na renderowanie tylko tych komponentów, które są widoczne użytkownikowi zamiast całej strony. To pozwala użytkownikowi korzystać ze strony nie czekając kiedy wszystkie dane zostaną załadowane, co jest bardzo ważne dla stron forum i czat, które mogą zawierać w nieskończoną listę danych.

Na stronie forum też zostały użyte okna modalne (Rysunek 8.17) co pozwala użytkownikom na dodawanie edycję lub przeglądanie szczególnej informacji o poście bez potrzebowania zmiany strony. Takie rozwiązanie poprawia komfort użytkowania oraz usprawnia interakcję z systemem.

```
<Virtuoso
  components={{
    Header: () => <DodawaniaPostu/>
  }}
  useWindowScroll
  className={"h-[600px] rounded-lg"}
  totalCount={reversed.length}
  itemContent={(i) => <Post setShow={setShow} post={reversed[i]} />}
/>
```

Rysunek 8.19. React Virtuoso. Źródło: opracowanie własne

```
<div className="bg-[#4D644C] flex items-center px-4 py-3 mb-8 rounded-md">
  
  <button
    className="text-red-600 z-20 px-4 py-2 rounded-full bg-[#336250] hover:bg-[#2b5140]"
    onClick={() => {
      dialog.current.showModal();
      document.body.style.overflow = "hidden";
    }}
  >
    Chcesz dodać post?
  </button>
</div>

{/* Dialog */}
<dialog
  ref={dialog}
  className="fixed left-[20vw] top-[80px]
h-[500px] w-[500px]"
  onClose={() => (document.body.style.overflow = "auto")}
  onCancel={() => (document.body.style.overflow = "auto")}
  ...
```

Rysunek 8.20. Okno Modalne. Źródło: opracowanie własne

8.2.11 Czaty

W celu możliwości ciągłej komunikacji komunikacji zostały użyte WebSocket i biblioteki @stomp/stompjs i sockjs-client. Oni pozwalają na stworzenie klienta który będzie

nasłuchiwać na backendzie i przy pojawieniu nowej wiadomości na adresach ukazanych przez subscribe (w poniższym przykładzie to `/topic/chat/${czatId}`) od razu otrzymywać tę wiadomość. Atrybut klienta `publish` pozwala wysyłać zapytania do backendu.

```
useEffect(() => {
  if (!czatId) return;
  const stompClient = new Client({
    websocketFactory: () =>
      new SockJS(`${process.env.NEXT_PUBLIC_BACKEND_PORT}/ws`),
    onConnect: () => {
      stompClient.subscribe(`/topic/chat/${czatId}`, (msg) => {
        setCzat((prev) => { return [...prev, "wiadomosci": [...prev.wiadomosci||[], JSON.parse(msg.body)]] });
      });
    },
    onStompError: (frame) => {
      console.error(frame);
    },
  });
  stompClient.activate();
  setClient(stompClient);

  return () => stompClient.deactivate();
}, [czatId]);

const sendMessage = () => {
  if (client && message.trim()) {
    client.publish({
      destination: `/app/chat.send/${czatId}`,
      body: JSON.stringify({
        uzytkownikId: user.id,
        czatId: czatId,
        tresc: message,
      }),
    });
    setMessage("");
  }
};
```

Rysunek 8.21. WebSocket. Źródło: opracowanie własne

8.2.12 Wydarzenia

W celu zwiększenia wydajności podczas ładowania dużych list w komponencie Wydarzenia zastosowano paginację. W odróżnieniu od wirtualnej listy, paginacja umożliwia łatwiejszą orientację w liście, ponieważ każdy element znajduje się na konkretnej stronie, co pozwala na jego późniejsze szybkie odnalezienie.

Dodatkowo, do wyszukiwania wydarzeń wykorzystano filtry: według nazwy (wyszukujący wszystkie wydarzenia zawierające wpisany ciąg znaków), według typu oraz według daty.

```
useEffect( ()=>{  
  const sort= ()=>{  
    setWydarzeniaSortowane(wydarzenia.filter(w=>  
      w.nazwa.includes(nazwa)&&  
      (w.typ===typ||typ=== "Typ wydarzenia")&&  
      (data=== ""||new Date(w.dataWyjazdu).getTime()>new Date(data).getTime()  
    ))  
    setPage(1)  
    if(inputRef.current)  
      inputRef.current.value=1  
  }  
  sort()  
},[wydarzenia, nazwa, typ, data])
```

Rysunek 8.22. Sortowanie wydarzeń. Źródło: opracowanie własne

```

const
wydarzeniaNaStronie=wydarzeniaSortowane.slice(page*wydarzeniaPerPage,page*wydarzeniaPerPage)

return (
  <div>

    <div className={"flex flex-row"}>
      <Filter/>
      {/*<div className={"wydarzenia-vertical-line"} ></div>*/}
      <div className={"flex flex-row flex-wrap"}>
        {wydarzeniaNaStronie.map((wydarzenie, i) => (
          <WydarzenieMale wydarzenie={wydarzenie} key={i} width={elementWidth - 20}/>
        ))}
        <Pagination liczbaStron={liczbaStron} page={page} setPage={setPage}
inputRef={inputRef} />
      </div>

    </div>
  </div>
)

```

Rysunek 8.23. Wyświetlanie wydarzeń. Źródło: opracowanie własne

```

return(
  <ul className={"pagination"} style={{display: 'flex', flexDirection: 'row', width: "100%",
justifyContent: "center"}}>
    <li>
      <button onClick={() => {
        inputRef.current.value=page-1;
        setPage(page - 1)
      }}
        style={page<=1?{visibility:"hidden",marginRight: "10px"}:{marginRight:
"10px"}}>poprzednia strona
      </button>
    </li>
    {/* {page <= 1 && <li >poprzednia strona</li>} */}
    <li><label><input type={"number"} ref={inputRef}
      onChange={(e) => {
        if(e.target.value>liczbaStron){
          setPage(liczbaStron)
          e.target.value=liczbaStron;
        }
        else if (e.target.value && e.target.value > 0) setPage(e.target.value)
      }} style={{width: "50px"}} defaultValue={page}/></label></li>
    <li>
      <button onClick={() => {
        setPage(page + 1)
        inputRef.current.value=page+1;
      }}
        style={page >= liczbaStron ? {visibility:"hidden",marginLeft: "10px"}:{marginLeft:
"10px"}}>następna strona
      </button>
    </li>
    {page >= liczbaStron&&<li></li>}
  </ul>
)

```

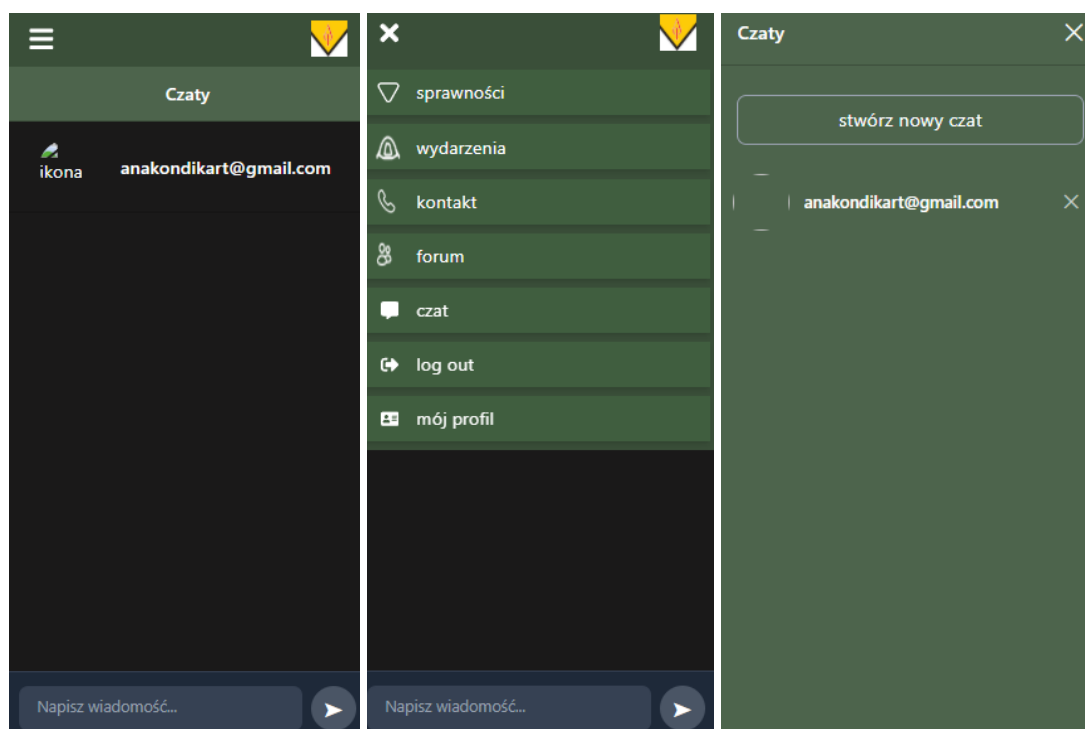
Rysunek 8.24. Komponent paginacji. Źródło: opracowanie własne

8.2.13 Responsywność

Zgodnie z wymaganiami użytkownik powinien mieć możliwość bezproblemowego wykonania tych samych operacji niezależnie od wykorzystywanego narzędzia-zarówno

telefonu, jak i komputera. W tym ważną rolę odgrywa responsywność, która umożliwia dynamiczne dostosowanie układu strony do rozmiaru ekranu.

Do tego w naszej aplikacji zostały użyte wbudowane w tailwind klasy:sm(szerokość $\geq 640\text{px}$), md (szerokość $\geq 768\text{px}$), i lg (szerokość $\geq 1024\text{px}$). To pozwala na szybką zmianę układu strony w zależności od szerokości ekranu, zapewniając spójne i wygodne korzystanie z aplikacji na różnych urządzeniach.



Rysunek 8.25. Rzuty ekrany wyglądu w wersji mobilnej. Źródło: opracowanie własne

8.3 Implementacja backendu

Backend Aplikacji został zaimplementowany w języku Java 17 z wykorzystaniem frameworka Spring Boot 3.5.4, który umożliwił szybkie tworzenie aplikacji opartej na architekturze warstwowej, oraz ułatwia konfigurację różnych komponentów systemu.

Do obsługi warstwy persistencji danych użyto Spring Data JPA, które pozwala na mapowanie obiektowo relacyjne pomiędzy encjami a bazą danych MySQL. Repozytoria JPA

pozwalają na wykonywanie operacji CRUD bez pisania zapytań SQL do bazy danych co zmniejsza nakład pracy i bezpieczeństwo kodu.

Do obsługi warstwy bezpieczeństwa i autoryzacji użyto Spring Security, które odpowiada za uwierzytelnianie i autoryzację użytkowników poprzez kontrolę dostępu do zasobów systemu w zależności od ról użytkowników. Do uwierzytelniania zastosowano JWT (JSON Web Token) do umożliwienia integrację z frontendem. Dodatkowo poza standardowym logowaniem za pomocą loginu i hasła skorzystano z bibliotek Google OAuth do logowania za pomocą konta Google.

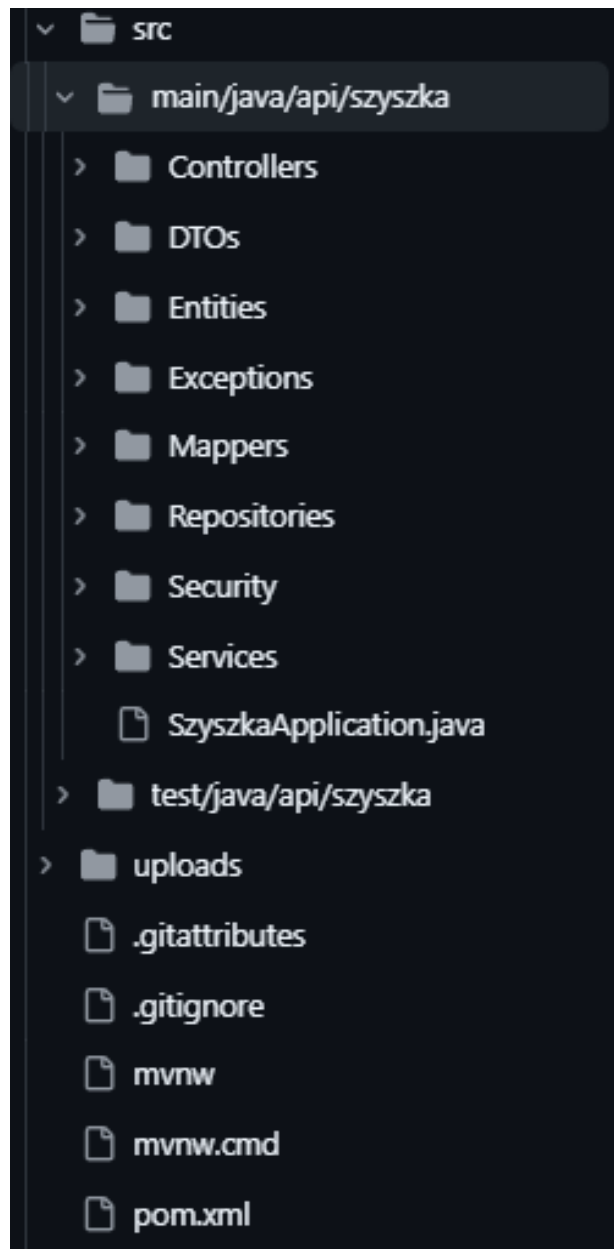
Komunikacja w czasie rzeczywistym wykorzystana w module czatu została zaimplementowana przy pomocy WebSocketów i protokołu STOMP, który jest wspierany przez Spring Web Socket. Pozwala to na wysyłanie wiadomości pomiędzy użytkownikami bez konieczności odpytywania serwera.

Biblioteki wykorzystywane w systemie:

- Spring Boot Starter Data JPA - Obsługa baz danych przez JPA/Hibernate
- Spring Boot DevTools - Narzędzia developerskie do szybkiego restartu
- MySQL Connector - Sterownik do bazy danych MySQL
- Spring Boot Starter WebSocket - Obsługa komunikacji WebSocket
- Google API Client - Klient do serwisów Google API
- JWT (Java JWT) - Tworzenie i weryfikacja tokenów JWT
- Spring Boot Starter Web - Tworzenie aplikacji webowych REST API
- Spring Boot Starter Security - Zabezpieczenia i autentykacja
- Lombok - Automatyczne generowanie getterów/setterów
- Spring Boot Test - Narzędzia do testowania aplikacji

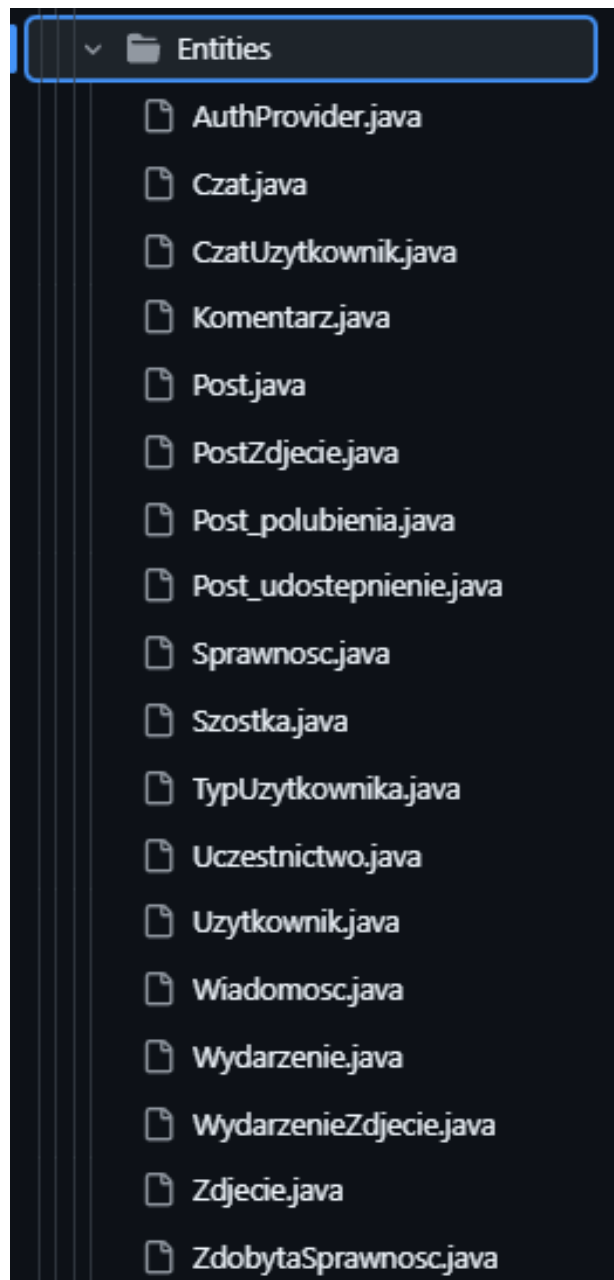
8.3.1 Struktura

Zaimplementowano architekturę warstwową, poszczególne katalogi odpowiadają komponentom aplikacji. Projekt backendu Struktura projektu jest przedstawiony na Rysunku 8.26. Wszystkie zmiany były umieszczane w repozytorium GitHub. Skład katalogu jest przedstawiony na Rysunek 8.27.



Rysunek 8.26.Struktura projektu na GitHub. Źródło: opracowanie własne

Katalog Entities zawiera klasy, które reprezentują strukturę danych przechowywaną w bazie danych. Encje są mapowane na tabele z bazy danych.



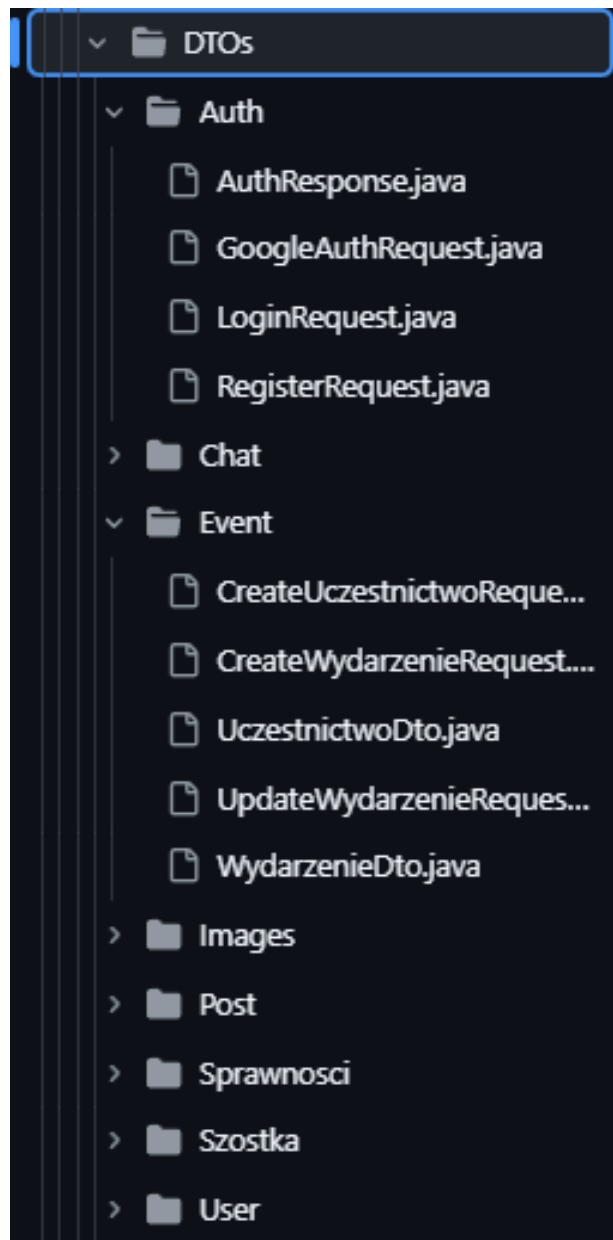
Rysunek 8.27.Struktura katalogu Entities. Źródło: opracowanie własne

Katalog repositories zawiera interfejsy repozytoriów, które odpowiadają za dostęp do bazy danych. Repozytoria odpowiadają za takie funkcje jak zapis, odczyt, aktualizacja czy usuwanie danych z bazy danych wykorzystując bibliotekę `org.springframework.data.jpa`. Skład katalogu jest przedstawiony na Rysunek 8.28.



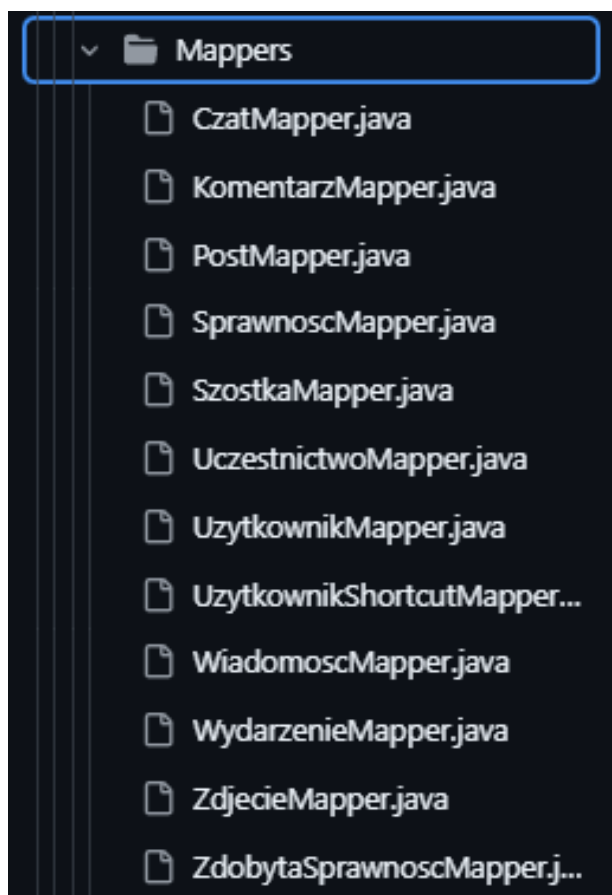
Rysunek 8.28.Struktura katalogu Repositories. Źródło: opracowanie własne

Katalog DTOs zawiera klasy obiektów transferu danych (DTO), które są wykorzystane do przekazywania informacji pomiędzy warstwami aplikacji. Te obiekty pozwalają na kontrolowanie jak dużo i jakie konkretnie dane są przekazywane między warstwami systemu. Skład katalogu jest przedstawiony na Rysunek 8.29.



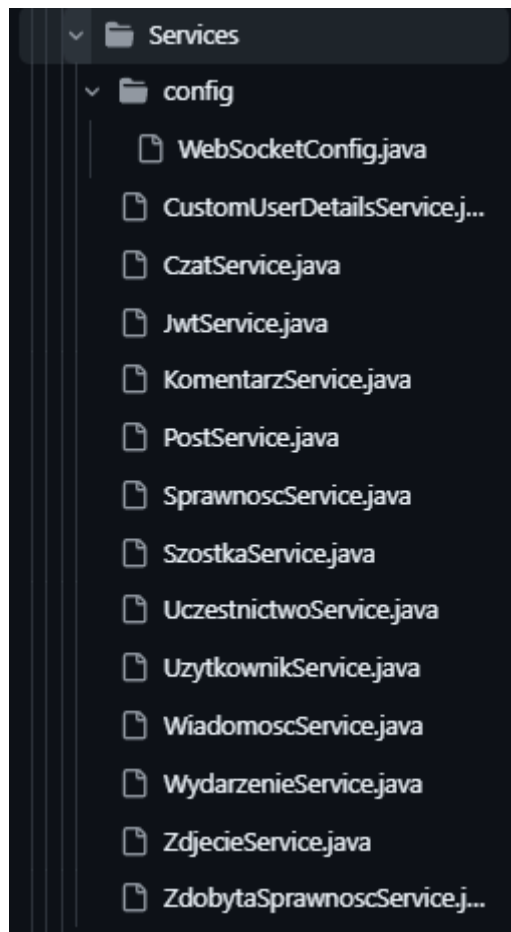
Rysunek 8.29.Struktura katalogu DTOs. Źródło: opracowanie własne

Katalog Mappers zawiera klasy odpowiedzialne za mapowanie encji na obiekty DTO, pozwala to ograniczyć zależności pomiędzy warstwami aplikacji. Skład katalogu jest przedstawiony na Rysunek 8.30.



Rysunek 8.30. Struktura katalogu Mappers. Źródło: opracowanie własne

Katalog Services zawiera klasy, które implementują logikę biznesową systemu. Odpowiadają za przetwarzanie danych oraz zawierają funkcję wykorzystywane w kontrolerach. W nich jest zawarta logika aplikacji. Skład katalogu jest przedstawiony na Rysunek 8.31.



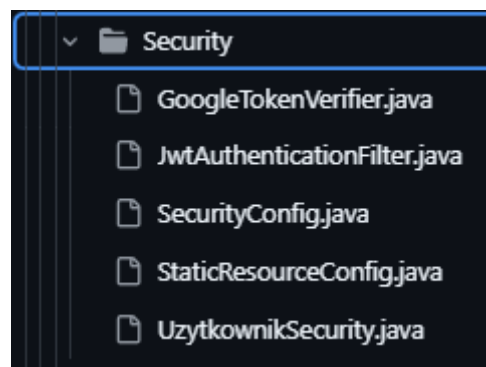
Rysunek 8.31.Struktura katalogu Services. Źródło: opracowanie własne

Katalog Controllers zawiera klasy, które obsługują żądania przychodzące do systemu i odpowiadają za przyjmowanie danych oraz wstępną walidację, po czym przekazują je do Serwisów. Skład katalogu jest przedstawiony na Rysunek 8.32.



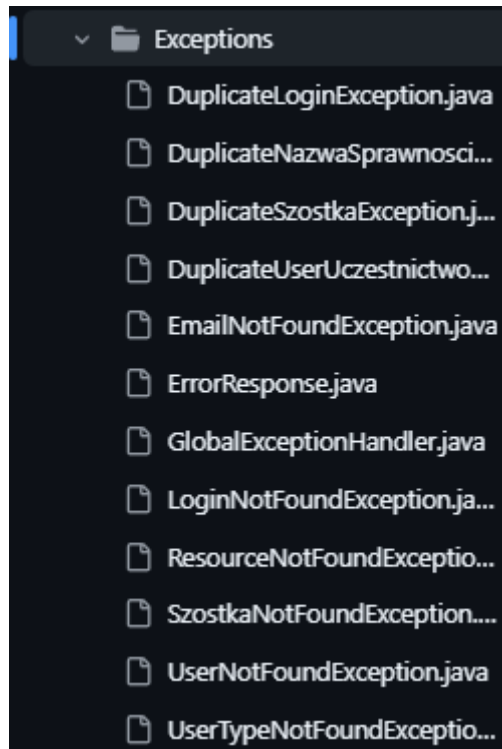
Rysunek 8.32. Struktura katalogu Controllers Źródło: opracowanie własne

Katalog Security zawiera klasy odpowiedzialne za bezpieczeństwo systemu oraz konfigurację uwierzytelniania i autoryzacji. Skład katalogu jest przedstawiony na Rysunek 8.33.



Rysunek 8.33. Struktura katalogu Security. Źródło: opracowanie własne

Katalog Exceptions zawiera klasy, które obsługują wyjątki rzucane przez różne metody w systemie. Pozwalają na zachowanie przejrzystości kodu dają czytelne informacje o problemach które mogą wystąpić podczas działania aplikacji. Skład katalogu jest przedstawiony na Rysunek 8.34.



Rysunek 8.34.Struktura katalogu Exceptions. Źródło: opracowanie własne

8.3.2 Budowa aplikacji

Przykładowy fragment encji z projektu znajduje się na rysunku 8.35. Wykorzystujemy bibliotekę lombok, która pozwala nam wygenerować gettery i settery oraz konstruktory.

```

8      @Entity
9      @Getter @Setter @NoArgsConstructor @AllArgsConstructor
10     @Table(name = "user")
11     public class Uzytkownik {
12
13         @Id
14         @GeneratedValue(strategy = GenerationType.IDENTITY)
15         private Long id;
16
17         private String imie;
18         private String nazwisko;
19
20         @Column(unique = true, nullable = false)
21         private String login;
22
23         private String haslo;
24         private String email;
25         private LocalDateTime dataUrodzenia;
26         private String nrTelefonu;
27         private LocalDateTime dataDolaczeniaDoGromady;
28         private String zdjecie;
29         @ManyToOne
30         @JoinColumn(name = "rodzic_id1")
31         private Uzytkownik rodzic1;
32
33         @ManyToOne
34         @JoinColumn(name = "rodzic_id2")
35         private Uzytkownik rodzic2;
36
37         @Enumerated(EnumType.STRING)
38         @Column(name = "typ_uzytkownika", nullable = false)
39         private TypUzytkownika typUzytkownika = TypUzytkownika.DEFAULT;
40
41
42         @ManyToOne
43         @JoinColumn(name = "squad_id")
44         private Szostka szostka;

```

Rysunek 8.35. Fragment kodu Encji Użytkownik. Źródło: opracowanie własne

Na Rysunku 8.36 przedstawiony jest interfejs `UzytkownikRepository` implementujący `JpaRepository`, które odpowiada za dostęp do danych w tabeli “user” w bazie danych. Zawiera metody pozwalające na wyszukiwanie użytkownika po różnych parametrach takich jak login czy szóstka.

```

1  package api.szyszka.Repositories;
2
3
4  import api.szyszka.Entities.Uzytkownik;
5  import org.springframework.data.jpa.repository.JpaRepository;
6  import java.util.Optional;
7  import java.util.List;
8
9  public interface UzytkownikRepository extends JpaRepository<Uzytkownik, Long> {
10     Optional<Uzytkownik> findByLogin(String login);
11     List<Uzytkownik> findBySzostkaId(Long szostkaId);
12     List<Uzytkownik> findByRodzic1IdOrRodzic2Id(Long rodzicId1, Long rodzicId2);
13     Optional<Uzytkownik> findByEmail(String email);
14
15     List<Uzytkownik> findByTypUzytkownika(String userType);
16
17     Optional<Uzytkownik> findByGoogleId(String googleId);
18
19 }

```

Rysunek 8.36. Użytkownik Repository. Źródło: opracowanie własne

Enum TypUzytkownika przedstawiony na Rysunku 8.37. Zawiera typy wykorzystywane typy użytkownika w tym typ DEFAULT nadawany nowo zarejestrowanym użytkownikom.

```

1  package api.szyszka.Entities;
2
3  public enum TypUzytkownika {
4      DRUZYNOWY,
5      PRZYBOCZNY,
6      ZUCH,
7      RODZIC,
8      DEFAULT
9  }

```

Rysunek 8.37. Enum TypUzytkownika. Źródło: opracowanie własne

UzytkownikDto na Rysunku 8.38, a także ChangeUserTypeRequest na Rysunku 8.39 to przykłady tego jak wyglądają nasze obiekty transferu danych, na przykładzie ChangeUserTypeRequest widać, że ten DTO zawiera tylko jedno pole: TypUzytkownika.

Ogranicza to ilość informacji przekazywanych w przypadku chęci zmiany typu użytkownika przez administratora i dzięki wykorzystaniu tego obiektu nie są przesyłane zbędne bądź wrażliwe dane.

```
1  package api.szyszka.DTOS.User;
2
3  import api.szyszka.Entities.TypUzytkownika;
4  import lombok.*;
5
6  @Getter
7  @Setter
8  @NoArgsConstructor
9  @AllArgsConstructor
10 public class UzytkownikDto {
11     private Long id;
12     private String imie;
13     private String nazwisko;
14     private String login;
15     private String email;
16     private String nrTelefonu;
17     private TypUzytkownika typUzytkownika;
18 }
```

Rysunek 8.38. UzytkownikDto. Źródło: opracowanie własne

```
1  package api.szyszka.DTOS.User;
2
3  import api.szyszka.Entities.TypUzytkownika;
4  import lombok.Data;
5
6  @Data
7  public class ChangeUserTypeRequest {
8     private TypUzytkownika nowaRola;
9 }
```

Rysunek 8.39. ChangeUserTypeRequest. Źródło: opracowanie własne

Mapper Encji Uzytkownik przedstawiony na Rysunku 8.40. zawiera między innymi funkcje mapującą encję na DTO oraz różne funkcję tworzące instancje Encji Uzytkownik na podstawie np. CreateUzytkownikRequest.

```

10  ✓ public class UzytkownikMapper {
11
12  ✓   public static UzytkownikDto toDto(Uzytkownik u) {
13       if (u == null) return null;
14
15       return new UzytkownikDto(
16           u.getId(),
17           u.getImie(),
18           u.getNazwisko(),
19           u.getLogin(),
20           u.getEmail(),
21           u.getNrTelefonu(),
22           u.getTypUzytkownika() != null ? u.getTypUzytkownika().name() : null
23       );
24   }
25  ✓   public static Uzytkownik fromCreateRequest(CreateUzytkownikRequest r) {
26       if (r == null) return null;
27
28       Uzytkownik u = new Uzytkownik();
29       u.setImie(r.getImie());
30       u.setNazwisko(r.getNazwisko());
31       u.setLogin(r.getLogin());
32       u.setHaslo(r.getHaslo());
33       u.setEmail(r.getEmail());
34       u.setNrTelefonu(r.getNrTelefonu());
35
36       u.setTypUzytkownika(
37           r.getTypUzytkownika() != null
38               ? TypUzytkownika.valueOf(r.getTypUzytkownika().toUpperCase())
39               : TypUzytkownika.DEFAULT
40       );
41
42       return u;
43   }

```

Rysunek 8.40. UzytkownikMapper. Źródło: opracowanie własne

Serwis Użytkownika przedstawiony na Rysunkach 8.41-8.44 jest jednym z elementów warstwy logiki biznesowej, która odpowiada za zarządzanie użytkownikami systemu. UżytkownikService obsługuje operacje takie jak rejestracja, uwierzytelnianie, modyfikację danych użytkowników oraz kontrolę ich uprawnień. Główne zadania tego serwisu to: Rejestracja nowych użytkowników oraz sprawdzanie unikalności danych, obsługa logowania i generowania JWT, zarządzanie typami użytkowników i kontrolowanie uprawnień, pobieranie danych użytkowników oraz ich aktualizacja, integracja z logowaniem przez Google. Serwis wykorzystuje AuthenticationManager z SpringSecurity do uwierzytelniania użytkowników przy logowaniu. Po zweryfikowaniu danych generowany jest token JWT który umożliwia autoryzację kolejnych żądań. Kodowanie haseł odbywa się za pomocą Password Encodera który pozwala na przechowywanie zaszyfrowanych haseł. Klasa posiada adnotację “@Transactional” co oznacza, że każda operacja wykonywana na jej metodach jest wykonywana jako transakcje bazodanowe co zapewnia wycofanie zmian w ramach wystąpienia błędu.

```

59  public void register(RegisterRequest req) {
60      if (uzytkownikRepository.findByLogin(req.getLogin()).isPresent()) {
61          throw new DuplicateLoginException("Login already used");
62      }
63
64      Uzytkownik u = new Uzytkownik();
65      u.setLogin(req.getLogin());
66      u.setHaslo(passwordEncoder.encode(req.getHaslo()));
67      u.setImie(req.getImie());
68      u.setNazwisko(req.getNazwisko());
69      u.setTypUzytkownika(TypUzytkownika.DEFAULT);
70      u.setEmail(req.getEmail());
71      u.setDataUrodzenia(req.getDataUrodzenia());
72      u.setDataDolaczeniaDoGromady(LocalDateTime.now());
73
74      uzytkownikRepository.save(u);
75  }
76  public String login(LoginRequest req) {
77      authenticationManager.authenticate(
78          new org.springframework.security.authentication.UsernamePasswordAuthenticationToken(
79              req.getLogin(),
80              req.getHaslo()
81          )
82      );
83      return jwtService.generateToken(req.getLogin());
84  }

```

Rysunek 8.41. UzytkownikService. Źródło: opracowanie własne

```

173  public List<Uzytkownik> getParents(Long childId){
174      Uzytkownik child = uzytkownikRepository.findById(childId)
175          .orElseThrow(() -> new UserNotFoundException(childId));
176      List<Uzytkownik> parents = new ArrayList<>();
177      if(child.getRodzic1() != null){
178          parents.add(child.getRodzic1());
179      }
180      if (child.getRodzic2() != null){
181          parents.add(child.getRodzic2());
182      }
183      return parents;
184  }
185  public List<Uzytkownik> getUsersByType(String typUzytkownika) {
186      List<Uzytkownik> users = uzytkownikRepository.findByTypUzytkownika(typUzytkownika);
187      if (users.isEmpty()) {
188          throw new UserTypeNotFoundException(typUzytkownika);
189      }
190      return users;
191  }
192  public List<Uzytkownik> getUsersBySzostka(Long szostkaId){
193      if (uzytkownikRepository.findBySzostkaId(szostkaId).isEmpty()){
194          throw new SzostkaNotFoundException(szostkaId);
195      }
196      return uzytkownikRepository.findBySzostkaId(szostkaId);
197  }
198
199  public String loginWithGoogle(GoogleUserData googleUser) {
200

```

Rysunek 8.42. UzytkownikService. Źródło: opracowanie własne

```

199  ✓ public String loginWithGoogle(GoogleUserData googleUser) {
200
201      //Czy użytkownik już istnieje po googleId
202      var userOpt = uzytkownikRepository.findByGoogleId(googleUser.getGoogleId());
203      if (userOpt.isPresent()) {
204          return jwtService.generateToken(userOpt.get().getLogin());
205      }
206
207      // Czy istnieje konto nie google z tym samym email
208      if (googleUser.getEmail() != null) {
209          var byEmail = uzytkownikRepository.findByEmail(googleUser.getEmail());
210          if (byEmail.isPresent()) {
211              Uzytkownik u = byEmail.get();
212              u.setGoogleId(googleUser.getGoogleId());
213              u.setAuthProvider(AuthProvider.GOOGLE);
214              uzytkownikRepository.save(u);
215
216              return jwtService.generateToken(u.getLogin());
217          }
218      }
219
220
221      // 3. NOWY UŻYTKOWNIK (rejestracja przez Google)
222      Uzytkownik u = new Uzytkownik();
223      u.setLogin(googleUser.getEmail()); // login = email
224      u.setEmail(googleUser.getEmail());
225      u.setImie(googleUser.getImie());
226      u.setNazwisko(googleUser.getNazwisko());
227      u.setGoogleId(googleUser.getGoogleId());
228      u.setAuthProvider(AuthProvider.GOOGLE);
229      u.setTypUzytkownika(TypUzytkownika.DEFAULT);
230      u.setDataDolaczeniaDoGromady(LocalDateTime.now());
231
232      uzytkownikRepository.save(u);
233
234      return jwtService.generateToken(u.getLogin());
235  }
236
237  }

```

Rysunek 8.43. UzytkownikService. Źródło: opracowanie własne

```

11 import api.szyszka.Entities.TypUzytkownika;
12 import api.szyszka.Entities.Uzytkownik;
13 import api.szyszka.Exceptions.*;
14 import api.szyszka.Repositories.UzytkownikRepository;
15 import org.springframework.security.authentication.AuthenticationManager;
16 import org.springframework.security.crypto.password.PasswordEncoder;
17 import org.springframework.stereotype.Service;
18 import org.springframework.transaction.annotation.Transactional;
19
20 import java.time.LocalDateTime;
21 import java.util.ArrayList;
22 import java.util.List;
23
24 @Service
25 @Transactional
26 public class UzytkownikService {
27
28     private final UzytkownikRepository uzytkownikRepository;
29     private final PasswordEncoder passwordEncoder;
30     private final JwtService jwtService;
31     private final AuthenticationManager authenticationManager;
32
33     public UzytkownikService(UzytkownikRepository uzytkownikRepository, PasswordEncoder passwordEncoder, JwtService jwtService, AuthenticationManager authenticationManager) {
34         this.uzytkownikRepository = uzytkownikRepository;
35         this.passwordEncoder = passwordEncoder;
36         this.jwtService = jwtService;
37         this.authenticationManager = authenticationManager;
38     }

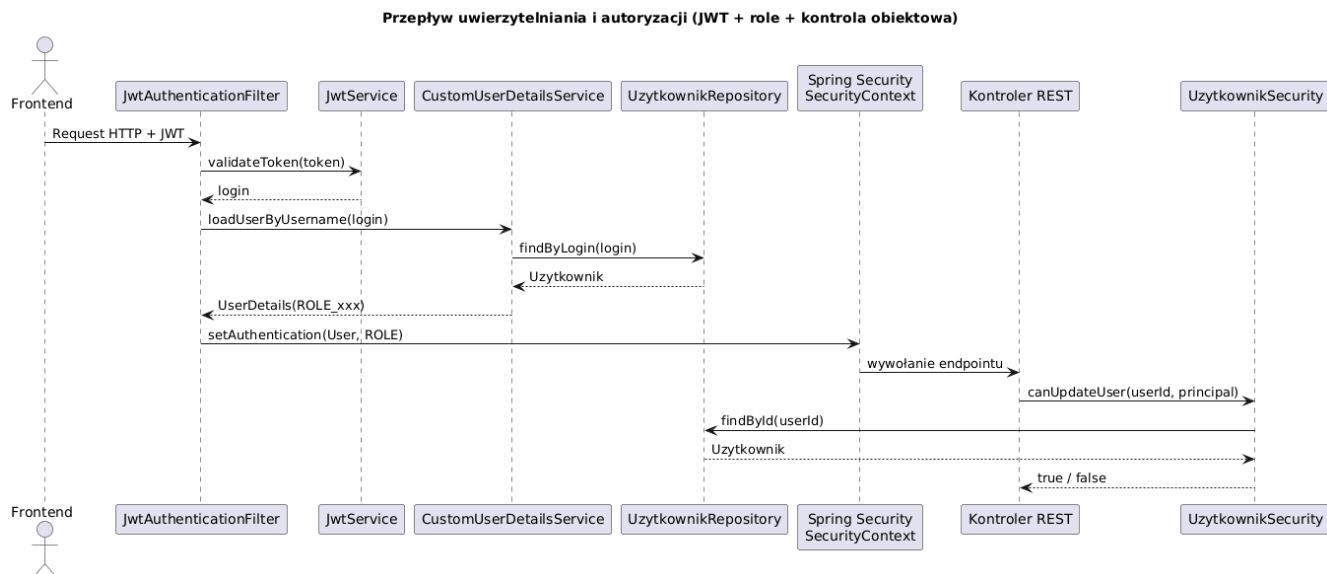
```

Rysunek 8.44. UzytkownikService. Źródło: opracowanie własne

Najważniejsze funkcjonalności backendu to: Zarządzanie użytkownikami i ich rolami, autoryzacja i uwierzytelnianie użytkowników, obsługa wydarzeń oraz sprawności, obsługa postów i komentarzy, czat w czasie rzeczywistym.

8.3.3 Mechanizm bezpieczeństwa

System bezpieczeństwa został oparty o Spring Security i JWT, klasa Security Config określa globalne reguły dostępu do zasobów oraz integrację z JWT. Oprócz tego zastosowano kontrolę dostępu na poziomie danych przy pomocy klasy UzytkownikSecurity, który pozwala na weryfikację, czy użytkownik ma prawo do modyfikacji konkretnego rekordu w bazie danych. Diagram sekwencji Rysunek 8.45 przedstawia przebieg uwierzytelniania i autoryzacji w systemie. Po odebraniu żądania z tokenem JWT jest on weryfikowany, następnie ładowany jest użytkownik wraz z odpowiadającą mu rolą, zapisanie informacji w kontekście bezpieczeństwa Spring Security i kontrola dostępu do zasobów na poziomie ról jak i konkretnych obiektów.



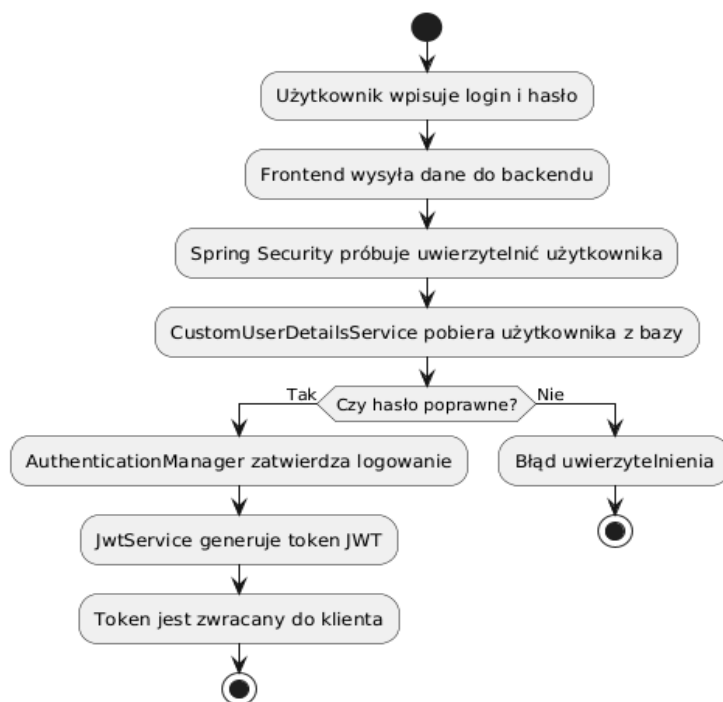
Rysunek 8.45. Przepływ uwierzytelniania i autoryzacji. Źródło: opracowanie własne

System pozwala na dwa sposoby logowania użytkownika

1. Login + hasło
2. Logowanie poprzez google(Oauth)

Podczas Rejestracji Użytkownik wysyła dane RegisterRequest Rysunek 8.47

Hasło jest hashowane za pomocą PasswordEncoder, po walidacji danych są one zapisywane w bazie.



Rysunek 8.46. Diagram sekwencji logowania. Źródło: opracowanie własne

```

47  ✓    public void register(RegisterRequest req) {
48          if (uzytkownikRepository.findByLogin(req.getLogin()).isPresent()) {
49              throw new DuplicateLoginException("Login already used");
50          }
51
52          Uzytkownik u = new Uzytkownik();
53          u.setLogin(req.getLogin());
54          u.setHaslo(passwordEncoder.encode(req.getHaslo()));
55          u.setImie(req.getImie());
56          u.setNazwisko(req.getNazwisko());
57          u.setTypUzytkownika(TypUzytkownika.DEFAULT);
58          u.setEmail(req.getEmail());
59          u.setDataUrodzenia(req.getDataUrodzenia());
60          u.setDataDolaczeniaDoGromady(LocalDateTime.now());
61
62          uzytkownikRepository.save(u);
63      }

```

Rysunek 8.47. AuthService metoda register. Źródło: opracowanie własne

Po poprawnym zarejestrowaniu użytkownik posiada `typUzytkownika = DEFAULT`, a drużynowy może mu nadać inny typ zależnie od jego roli w gromadzie.

Przy logowaniu standardowym za pomocą loginu i hasła użytkownik jest wysłany `LoginRequest` Rysunek 8.48 i Spring weryfikuje dane. Jeśli dane są poprawne generowany jest token JWT, który jest przechowywany w ciasteczkach (Rysunki 8.49, 8.50), jeśli użytkownik pozostaje zalogowany przeglądarka dołącza token do każdego requestu i w ten sposób jest uwierzytelniany przy każdej operacji.

```

64  ✓    public String login(LoginRequest req) {
65          authenticationManager.authenticate(
66              new org.springframework.security.authentication.UsernamePasswordAuthenticationToken(
67                  req.getLogin(),
68                  req.getHaslo()
69              )
70          );
71          return jwtService.generateToken(req.getLogin());
72      }

```

Rysunek 8.48. UzytkownikService metoda login. Źródło: opracowanie własne

```

43         // POBRANIE JWT Z COOKIE
44         String token = extractTokenFromCookie(request);
45         if (token == null) {
46             filterChain.doFilter(request, response);
47             return;
48         }
49         String username = jwtService.extractUsername(token);
50
51         if (username != null &&
52             SecurityContextHolder.getContext().getAuthentication() == null) {
53
54             UserDetails userDetails =
55                 userDetailsService.loadUserByUsername(username);
56
57             if (jwtService.isTokenValid(token)) {
58                 UsernamePasswordAuthenticationToken authToken =
59                     new UsernamePasswordAuthenticationToken(
60                         userDetails,
61                         null,
62                         userDetails.getAuthorities()
63                     );
64
65                 SecurityContextHolder.getContext()
66                     .setAuthentication(authToken);
67             }
68         }
69
70         filterChain.doFilter(request, response);
71     }

```

Rysunek 8.49. JwtService fragment kodu. Źródło: opracowanie własne

```

45     @PostMapping("/login")
46     public ResponseEntity<?> login(
47         @RequestBody LoginRequest req,
48         HttpServletResponse response
49     ) {
50         String token = userService.login(req);
51
52         ResponseCookie cookie = ResponseCookie.from("accessToken", token)
53             .httpOnly(true)
54             .secure(true)
55             .path("/")
56             .sameSite("None")
57             .maxAge(Duration.ofMinutes(60))
58             .build();
59         System.out.println(cookie);
60         response.addHeader(HttpHeaders.SET_COOKIE, cookie.toString());
61
62         return ResponseEntity.ok().build();
63     }

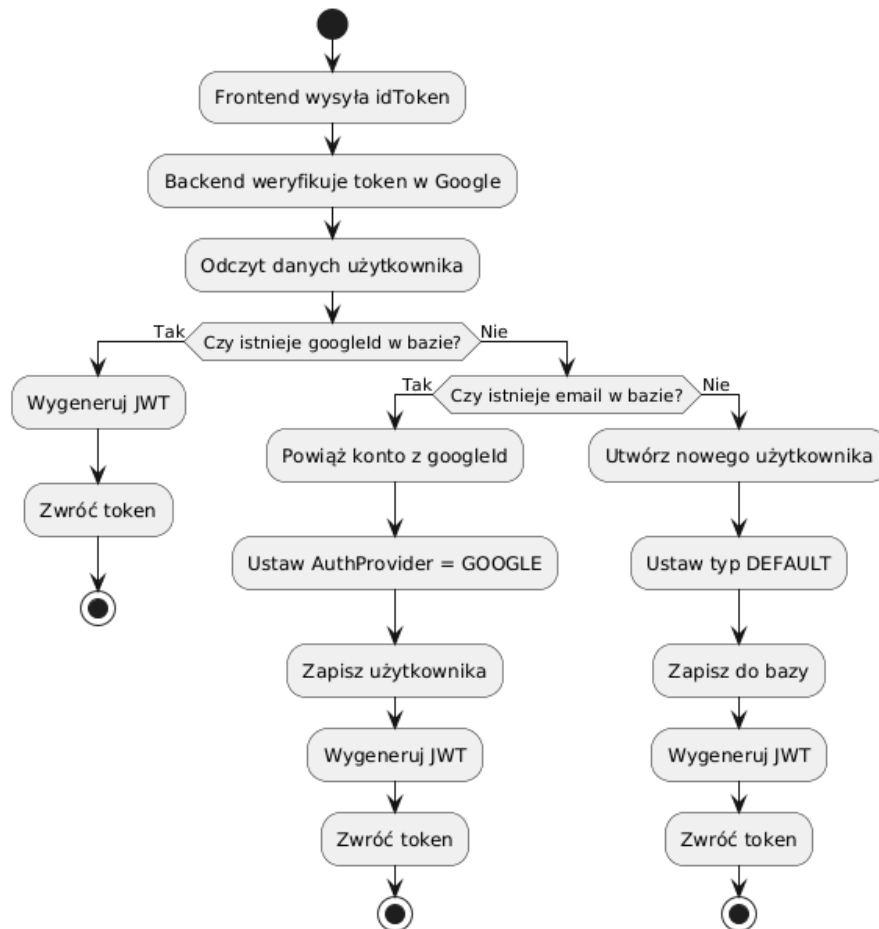
```

Rysunek 8.50. AuthController funkcja login. Źródło: opracowanie własne

Logowanie za pomocą Google odbywa się w inny sposób przedstawiony na diagramie czynności Rysunek 8.51. Ten proces pozwala na uwierzytelnienie użytkownika bez konieczności zakładania lokalnego konta z hasłem.

Klient loguje się za pomocą google a następnie do backendu jest przesyłany token identyfikacyjny, który zawiera dane użytkownika. idToken jest odbierany w obiekcie GoogleAuthRequest Rysunek 8.52, po czym backend weryfikuje token i wyciąga z tokenu dane użytkownika, które następnie mapuje na obiekt GoogleUserData Rysunek 8.53. Metoda loginWithGoogle Rysunek najpierw sprawdza czy baza danych już zawiera użytkownika z danym googleId, jeśli tak to generuje token JWT i loguje użytkownika. Jeśli nie istnieje jeszcze użytkownik z danym googleId sprawdzany jest email i jeśli istnieje użytkownik z takim emailiem konto zostaje powiązane z kontem google poprzez zapisanie googleId a następnie użytkownik zostanie zalogowany. Dzięki temu użytkownik może korzystać z jednego konta obojętnie jakiś sposób logowania preferuje.

W przypadku gdy użytkownik nie posiada konta google ani konta lokalnego system tworzy nowego użytkownika w bazie danych oraz zapisuje jego dane oraz ustawia typ użytkownika na DEFAULT.



Rysunek 8.51. Diagram Czynności Logowanie Google . Źródło: opracowanie własne

```

1  package api.szyszka.DTOs.Auth;
2
3  import lombok.Data;
4
5  @Data
6  public class GoogleAuthRequest {
7      private String idToken;
8  }

```

Rysunek 8.52. GoogleAuthRequest. Źródło: opracowanie własne

```

1  package api.szyszka.DTOs.User;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5
6  @Data
7  @AllArgsConstructor
8  public class GoogleUserData {
9      private String googleId;
10     private String email;
11     private String imie;
12     private String nazwisko;

```

Rysunek 8.53. GoogleUserData. Źródło: opracowanie własne

Role i uprawnienia użytkowników

Do kontroli uprawnień wykorzystywane jest pole `typUzytkownika`. Role, które są dostępne w systemie to: `DRUZYNOWY` (pełni funkcja administratora), `PRZYBOCZNY` (ma niektóre z praw administratora), `ZUCH` (główny i najczęściej stosowany typ użytkownika), `RODZIC` (powiązany z swoim dzieciem poprzez klucze obce w tabeli w bazie danych) oraz `DEFAULT` (podstawowy typ użytkownika nadawany przy rejestracji czekający na nadanie konkretnej roli przez administratora). Rysunek 8.54.

```

1  package api.szyszka.Entities;
2
3  public enum TypUzytkownika {
4      DRUZYNOWY,
5      PRZYBOCZNY,
6      ZUCH,
7      RODZIC,
8      DEFAULT
9  }

```

Rysunek 8.54. Enum TypUzytkownika. Źródło: opracowanie własne

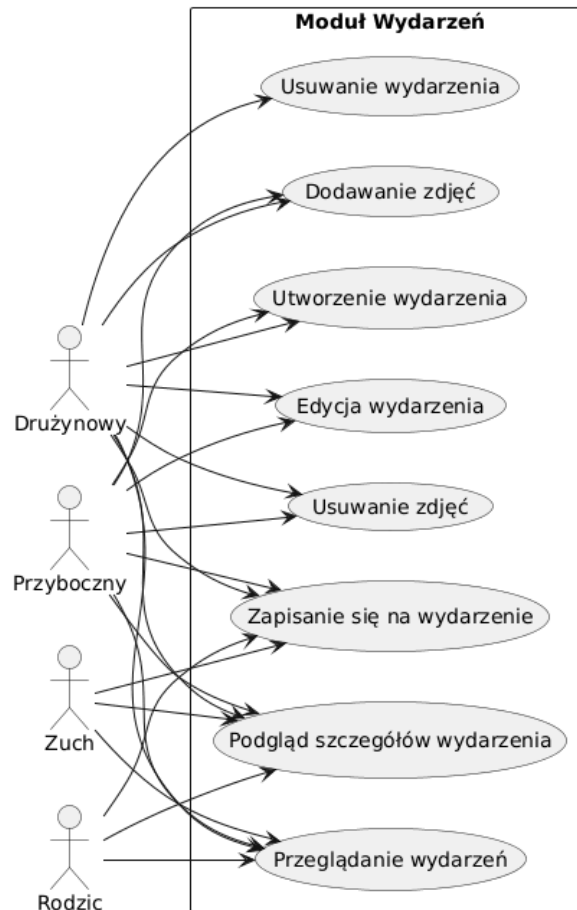
Klasa `CustomUserDetailsService` Rysunek 8.55 pozwala na pobieranie danych użytkownika z bazy danych za pomocą loginu oraz przekształcenie do postaci obiektu `UserDetails` co pozwala na wykorzystanie go w procesie autoryzacji i uwierzytelniania. Wcześniej

wspomniany typ użytkownika jest mapowany na role w Spring Security co pozwala na kontrolę uprawnień użytkowników.

```
15
16  @Service
17  @RequiredArgsConstructor
18  public class CustomUserDetailsService implements UserDetailsService {
19
20      private final UzytkownikRepository repo;
21
22      @Override
23      public UserDetails loadUserByUsername(String login) throws UsernameNotFoundException {
24          Uzytkownik u = repo.findByLogin(login)
25              .orElseThrow(() -> new UsernameNotFoundException("User not found"));
26
27          String role = "ROLE_" + u.getTypUzytkownika().name();
28
29          if (u.getHaslo() != null) {
30              return User.builder()
31                  .username(u.getLogin())
32                  .password(u.getHaslo())
33                  .authorities(role)
34                  .build();
35          } else {
36              return User.builder()
37                  .username(u.getLogin())
38                  .password("null")
39                  .authorities(role)
40                  .build();
41          }
42      }
43  }
```

Rysunek 8.55. CustomUserDetailsService . Źródło: opracowanie własne

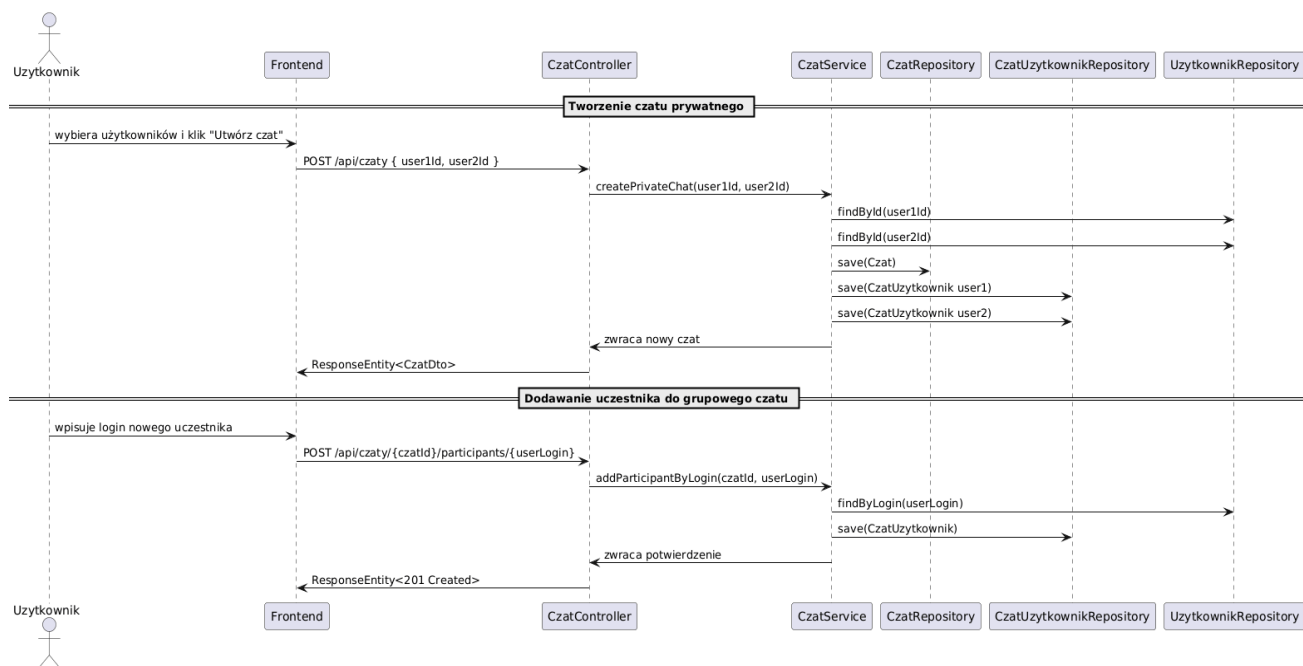
Podział użytkowników na role pozwala nam na kontrole dostępu do konkretnych funkcjonalności systemu. Możemy to zaobserwować na przykładzie WydarzenieController. Czynności możliwe do wykonania przez dany typ użytkownika są przedstawione na diagramie Rysunek 8.56



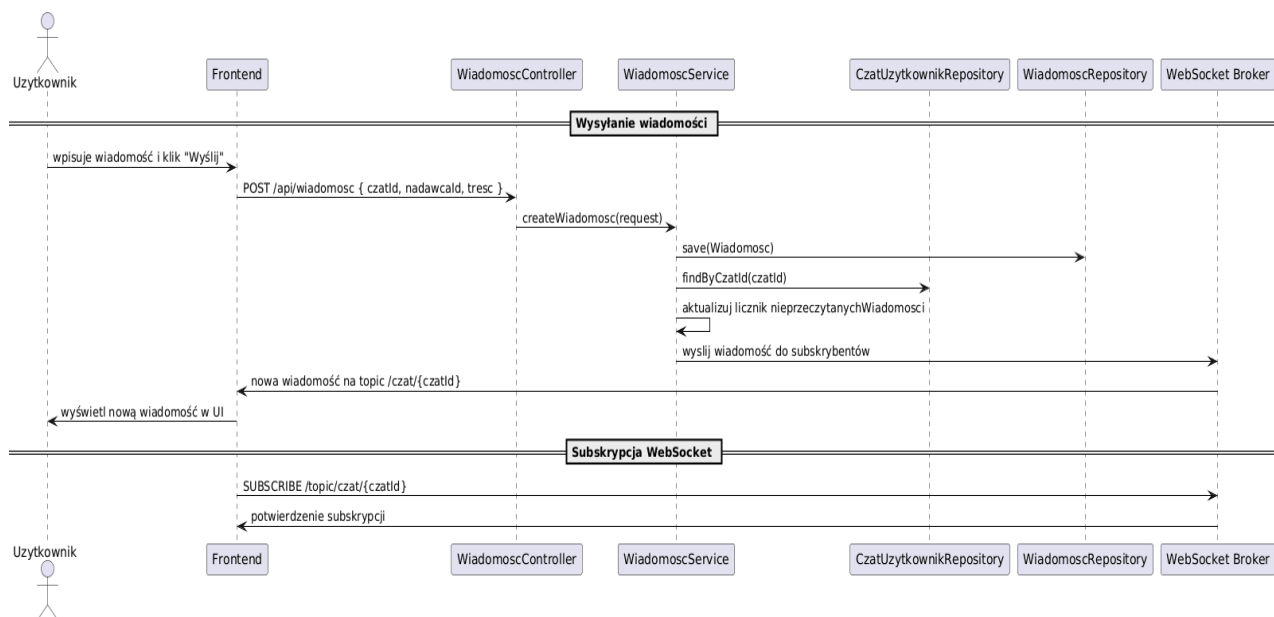
Rysunek 8.56. Diagram Przypadków użycia Wydarzenie . Źródło: opracowanie własne

Moduł komunikacji - czat w czasie rzeczywistym

Moduł czatu w czasie rzeczywistym w aplikacji został oparty na połączeniu REST API oraz WebSocketów. Logika biznesowa czatu zawiera tworzenie czatów prywatnych oraz grupowych, dodawanie i usuwanie uczestników, wysyłanie wiadomości oraz zliczanie nieprzeczytanych przez użytkownika wiadomości z danego czatu. Została ona zaimplementowana w CzatService. Aby umożliwić komunikację w czasie rzeczywistym wykorzystano protokół WebSocket z obsługą STOM, który jest skonfigurowany w klasie WebSocketConfig, definiuje on broker wiadomości (/topic, /queue), prefix aplikacji (/app) oraz kanały prywatne(/user) co pozwala podzielić kanały komunikacji na wiadomości kierowane do serwera oraz rozsyłane do klientów. Pozwala to na natychmiastowe dostarczanie wiadomości do odpowiednich użytkowników w czasie rzeczywistym. Diagram przedstawiony na Rysunek 8.57 pokazuje przebieg tworzenia czatów oraz dodawanie uczestników. Diagram sekwencji wysyłania wiadomości z WebSocket Rysunek 8.58



Rysunek 8.57. Diagram sekwencji tworzenia czatów . Źródło: opracowanie własne



Rysunek 8.58. Diagram sekwencji wysyłania wiadomości . Źródło: opracowanie własne

Rozdział 9.

Testowanie

Celem testowania było zweryfikowanie poprawności działania aplikacji oraz sprawdzenie, czy zaimplementowane funkcjonalności spełniają założenia projektowe. Zastosowane zostały testy jednostkowe oraz testy integracyjne co umożliwiała ocenę poprawności zarówno pojedynczych komponentów, jak i współdziałania modułów systemu.

Do analizy jakości testów wykorzystano narzędzie JaCoCo[12], umożliwiające pomiar pokrycia kodu testami.

9.1 Testy jednostkowe

Testy jednostkowe zostały zastosowane w celu sprawdzenia działania pojedynczych klas oraz metod, w szczególności warstwy serwisów. Warstwa ta zawiera główną logikę biznesową aplikacji.

Testy te koncentrują się na:

- metodach serwisów,
- obsłudze wyjątków,
- poprawnym przetwarzaniu danych wejściowych i wyjściowych.

Testy jednostkowe nie obejmują elementów takich jak:

- klasy DTO,
- konfiguracja aplikacji,
- kontrolery HTTP.

Takie podejście jest zgodne z dobrymi praktykami, według których testy jednostkowe powinny skupiać się na logice biznesowej, a nie na kodzie infrastrukturalnym.

9.2 Testy integracyjne

Testy integracyjne zostały zastosowane w celu weryfikacji współpracy pomiędzy poszczególnymi komponentami systemu, takimi jak:

- integracja warstwy serwisów z bazą danych,
- mechanizmy bezpieczeństwa..

W testach integracyjnych część danych wejściowych, takich jak daty, była przekazywana bezpośrednio w testach, mimo że w docelowej architekturze systemu są one generowane po stronie warstwy klienckiej (frontend). Zabieg ten miał na celu izolację testowanej logiki biznesowej oraz weryfikację poprawności działania metod serwisowych niezależnie od interfejsu użytkownika.

Testy te uruchamiane są w kontekście Spring Boot i obejmują zależności pomiędzy komponentami, co pozwala na realistyczną ocenę działania aplikacji [13].

9.3 Analiza pokrycia kodu testami

Na podstawie raportu wygenerowanego przez narzędzie JaCoCo uzyskano następujące wyniki pokrycia kodu testami:

Warstwa aplikacji	Pokrycie instrukcji
Kontrolery	3%
Serwisy	71%
Bezpieczeństwo	44%
Encje	97%
Cała aplikacja	41%

9.4 Interpretacja Wyników

Najwyższe pokrycie testami uzyskano dla warstwy encji oraz serwisów, co wskazuje na dobre przetestowanie kluczowej logiki biznesowej aplikacji. Niższe wartości dla kontrolerów oraz klas DTO wynika to z faktu, że są to warstwy techniczne, które nie zawierają istotnej logiki biznesowej i nie były testowane testami jednostkowymi.

Rozdział 10.

Prezentacja rozwiązania

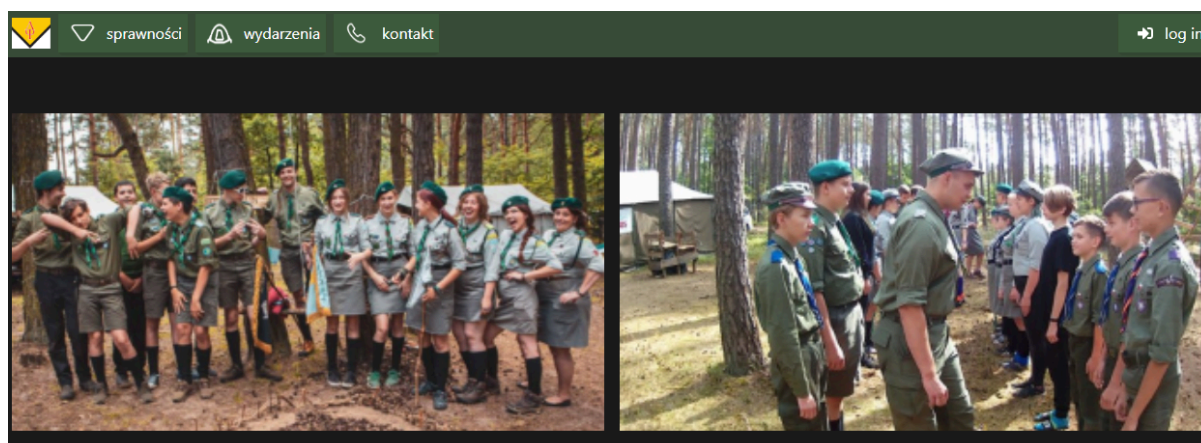
10.1 Strony dostępne bez logowania

Aplikacja posiada kilka stron dostępnych niezależnie od tego, czy użytkownik jest zalogowany do systemu, czy nie.

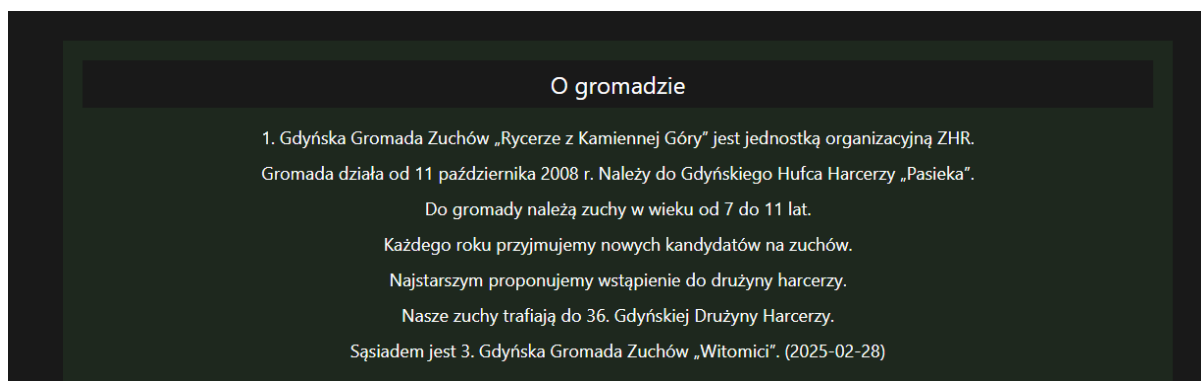
10.1.1 Strona główna

Wszystkie strony aplikacji są ze sobą powiązane za pomocą paska nawigacyjnego, który umożliwia użytkownikowi przechodzenie do dowolnego widoku niezależnie od aktualnie wyświetlanej strony. Takie rozwiązanie zapewnia intuicyjny dostęp do potrzebnych informacji oraz ułatwia poruszanie się po aplikacji.

Pierwszym widokiem, z którym styka się użytkownik po uruchomieniu aplikacji, jest strona główna. Zawiera ona opis gromady, a także karuzelę zdjęć wykonanych podczas wydarzeń organizowanych przez gromadę.



Rysunek 10.1 Karuzela zdjęć. Źródło: opracowanie własne

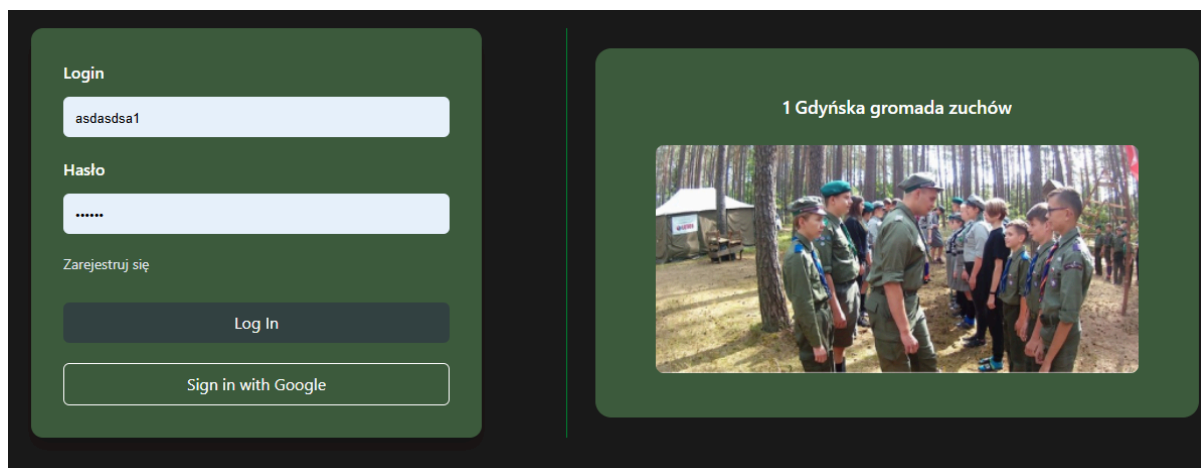


Rysunek 10.2 Opis gromady. Źródło: opracowanie własne

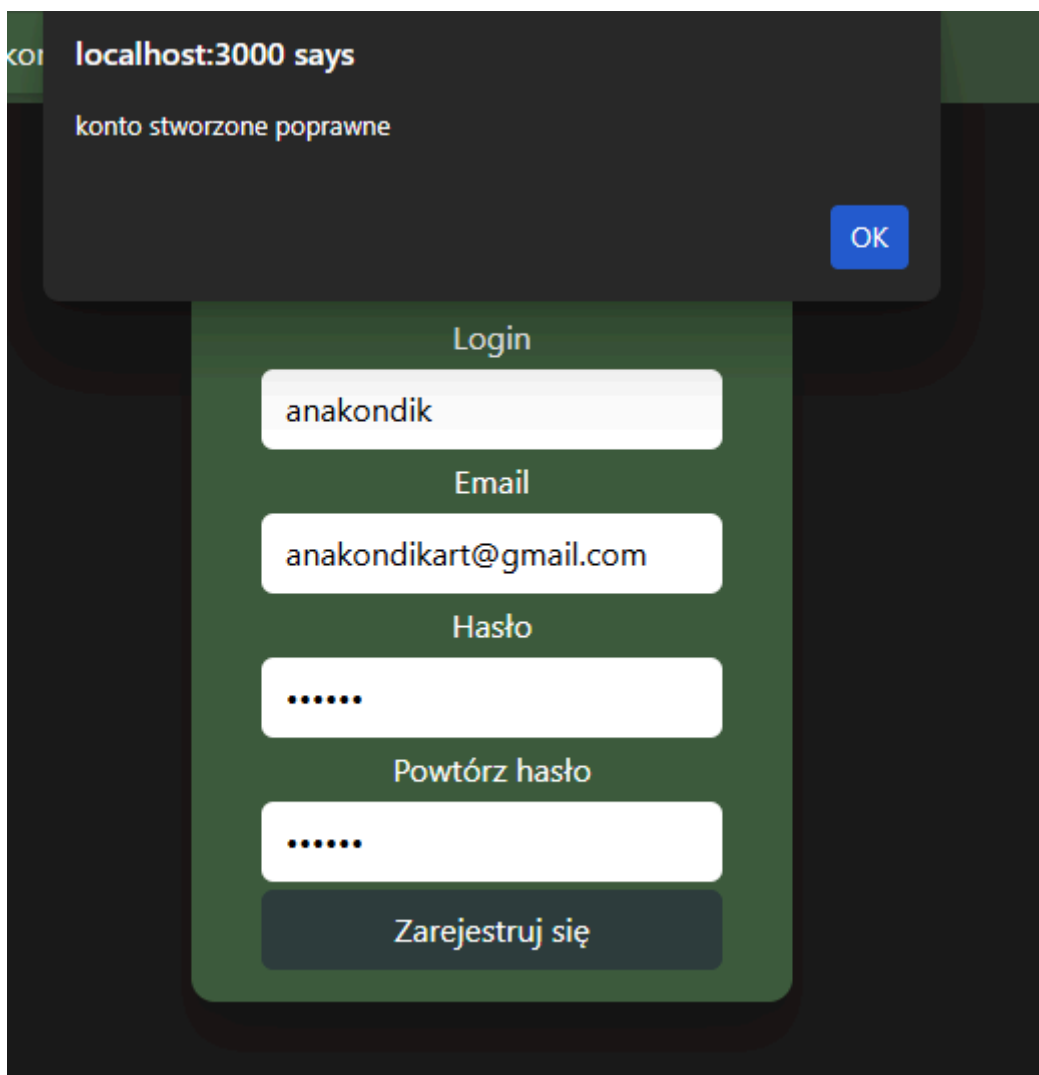
10.1.2 Strony logowania i rejestracji

Korzystanie z wybranych funkcji aplikacji wymaga zalogowania się użytkownika, co można zrealizować na stronie logowania. W przypadku braku konta istnieje możliwość jego utworzenia poprzez kliknięcie przycisku „Zarejestruj się”.

Aplikacja obsługuje również logowanie za pomocą konta Google. W tym celu użytkownik wybiera opcję „Sign in with Google” i przechodzi proces uwierzytelnienia w usłudze Google. Przy pierwszym logowaniu z wykorzystaniem konta Google konto użytkownika w aplikacji zostaje utworzone automatycznie.



Rysunek 10.3 Forma logowania. Źródło: opracowanie własne



Rysunek 10.4 Strona rejestracji. Źródło: opracowanie własne

Login

napisz login

to pole jest wymagane

Email

napisz email

to pole jest wymagane

Hasło

.....

Powtórz hasło

powtórz hasło

to pole jest wymagane

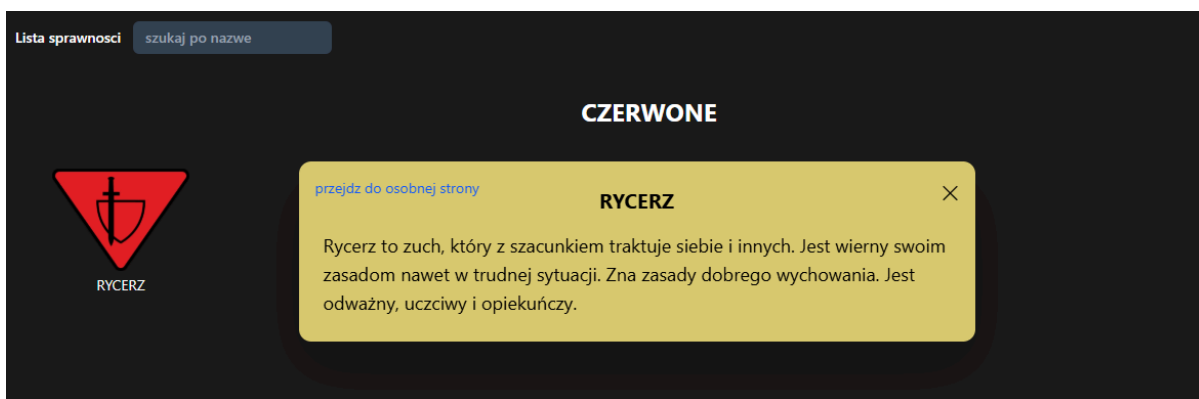
Zarejestruj się

Rysunek 10.5 Walidacja danych. Źródło: opracowanie własne

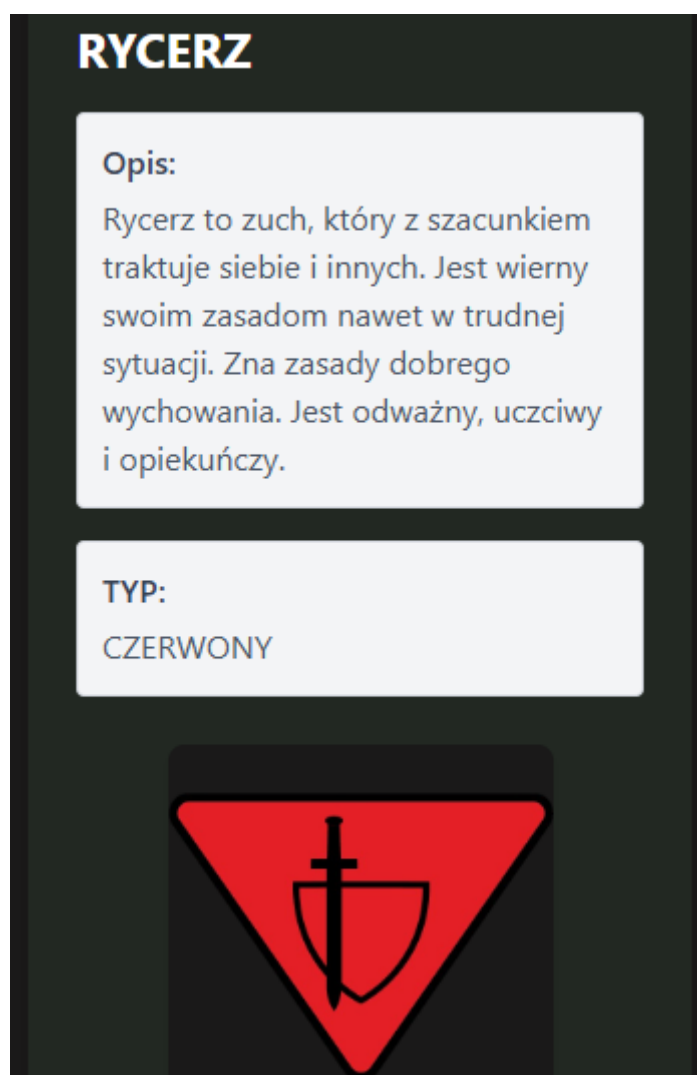
10.1.3 Strona sprawności

Strona sprawności zawiera informacje o wszystkich sprawnościach, które mogą zostać zdobyte przez zucha. Po najechniu kursorem myszy na wybraną sprawność użytkownikowi wyświetlany jest jej opis.

Na urządzeniach z mniejszym ekranem lub po kliknięciu przycisku aplikacja przekierowuje użytkownika na osobną stronę, na której informacje o danej sprawności prezentowane są w standardowej, czytelnej formie.



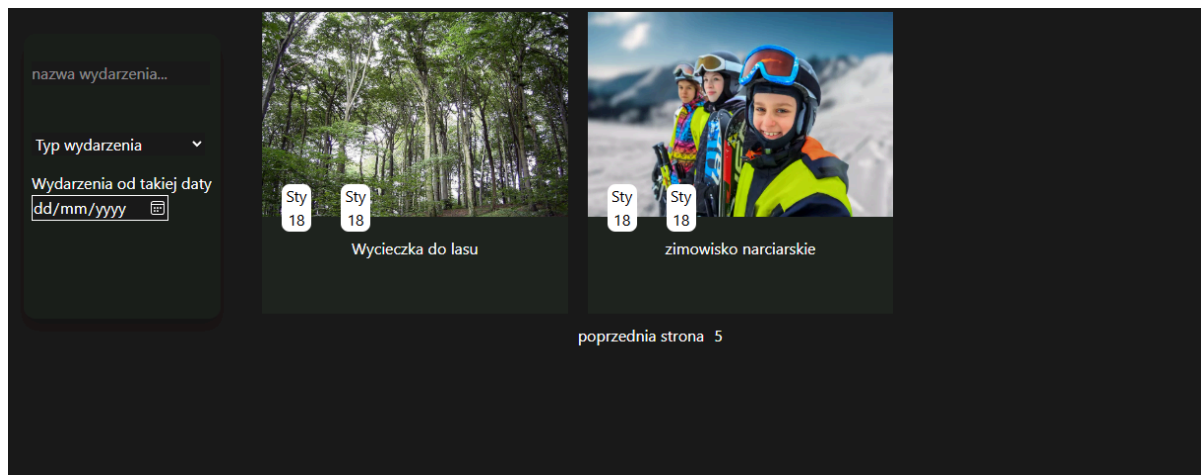
Rysunek 10.6 Strona sprawności. Źródło: opracowanie własne



Rysunek 10.7 Jedna sprawność. Źródło: opracowanie własne

10.1.4 Strona wydarzeń

Strona wydarzeń, podobnie jak strona sprawności, dostępna jest bez konieczności logowania. Użytkownik może na niej przeglądać nadchodzące oraz archiwalne wydarzenia organizowane przez gromadę.



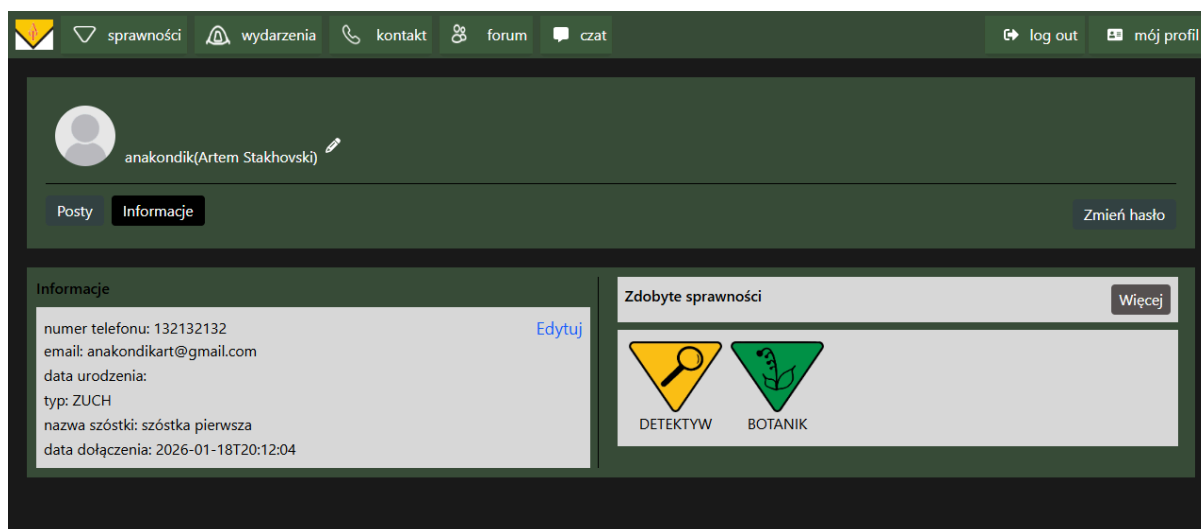
Rysunek 10.8 Strona wydarzeń. Źródło: opracowanie własne

10.2 Strony dostępne do zalogowanego użytkownika

Podczas tworzenia konta użytkownikowi przypisywany jest domyślny typ default, który pełni funkcję tymczasową i służy wyłącznie do momentu, aż drużynowy zmieni go na właściwą rolę, taką jak zuch lub rodzic.

10.2.1 Strona profilu

Dla użytkownika o typie default dostępna jest wyłącznie strona profilu. Na tej stronie użytkownik ma możliwość edycji swoich danych, co pozwala na uzupełnienie i aktualizację informacji osobowych w systemie oraz ich późniejszy wgląd przez drużynowego. Pole data dołączenia jest stałe i przypisywane automatycznie w momencie zakładania konta, natomiast pola typ użytkownika oraz nazwa szóstki są nadawane przez drużynowego.

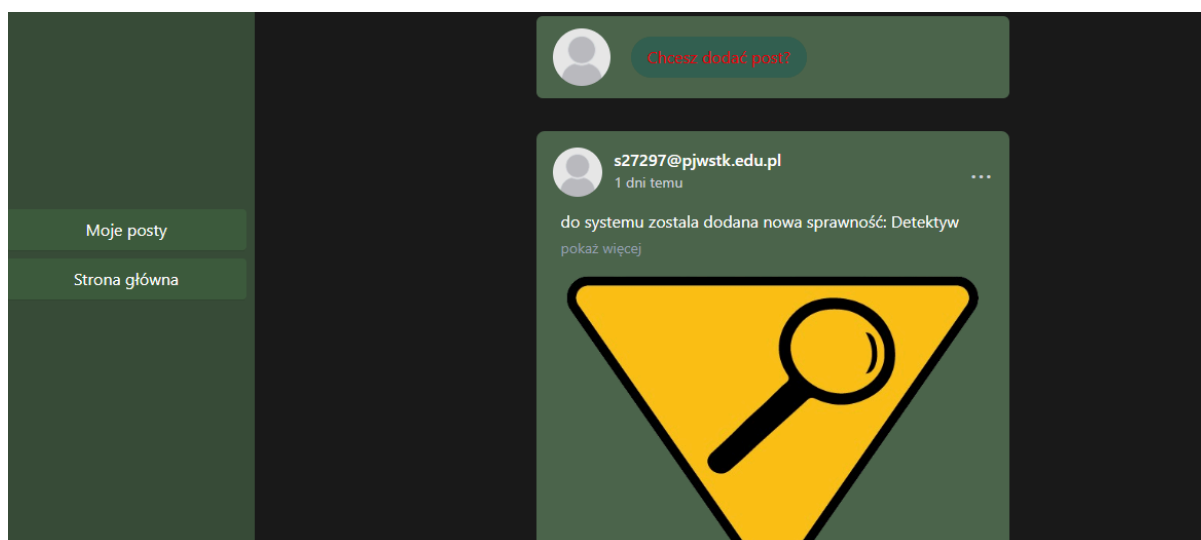


Rysunek 10.9 Strona profilu. Źródło: opracowanie własne

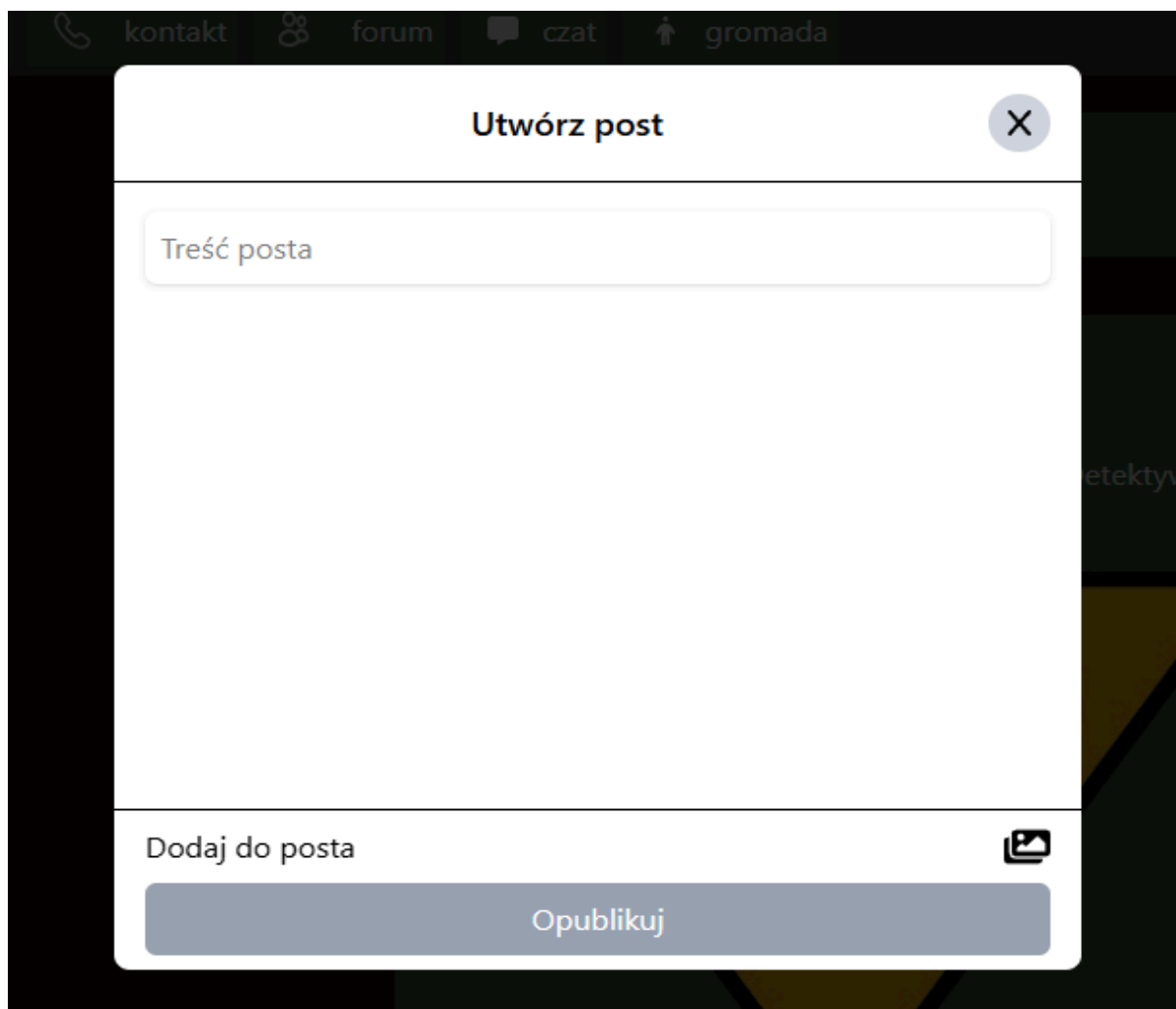
10.3 Strony dostępne dla zucha

10.3.1 Strona postów

Na stronie postów użytkownik może tworzyć swoje posty, czytać, lajkować, udostępniać i komentować posty innych. W lewej części strony znajduje się menu pozwalające na zobaczenie tylko swoich postów i wrócenie do strony ze wszystkimi. W centrum prawej części znajduje się przycisk do dodania nowego postu i lista ze wszystkimi postami. Kliknięcie na przycisk otwiera okno modalne do tworzenia postu.



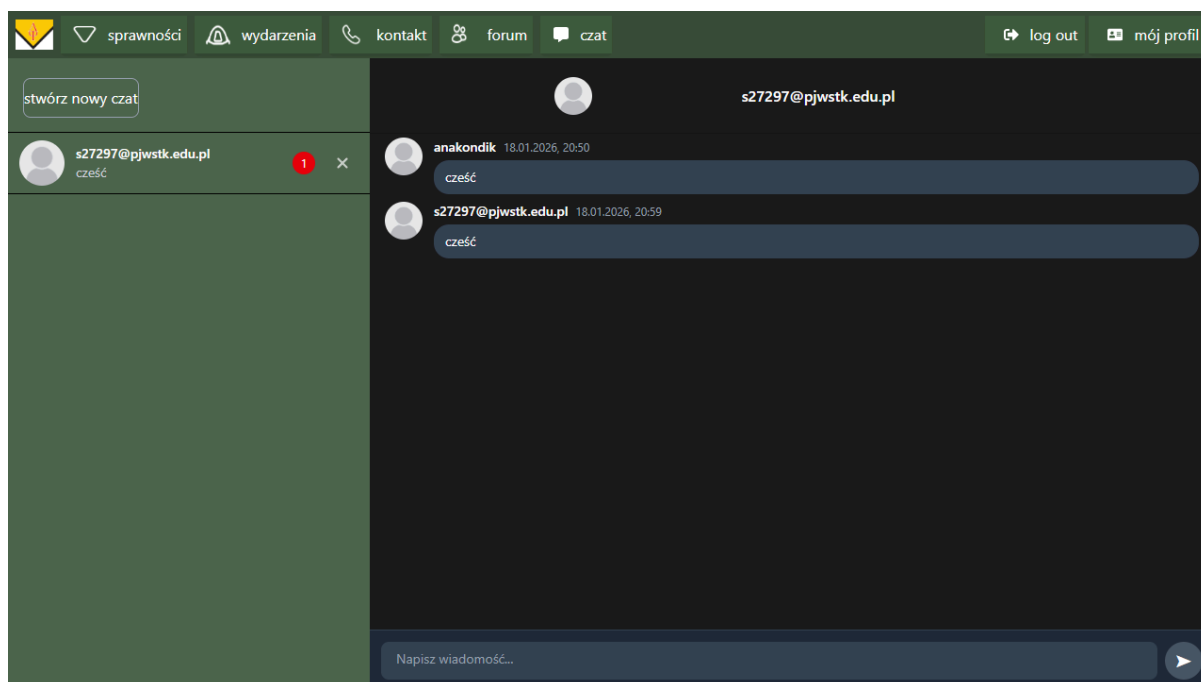
Rysunek 10.10 Strona postów. Źródło: opracowanie własne



Rysunek 10.11 Nowy post. Źródło: opracowanie własne

10.3.2 Strona czatów

Strona czatów, podobnie jak strona postów, podzielona jest na dwie części. Po lewej stronie znajduje się lista wszystkich dostępnych czatów, natomiast po prawej wyświetlana jest aktualnie otwarta rozmowa.

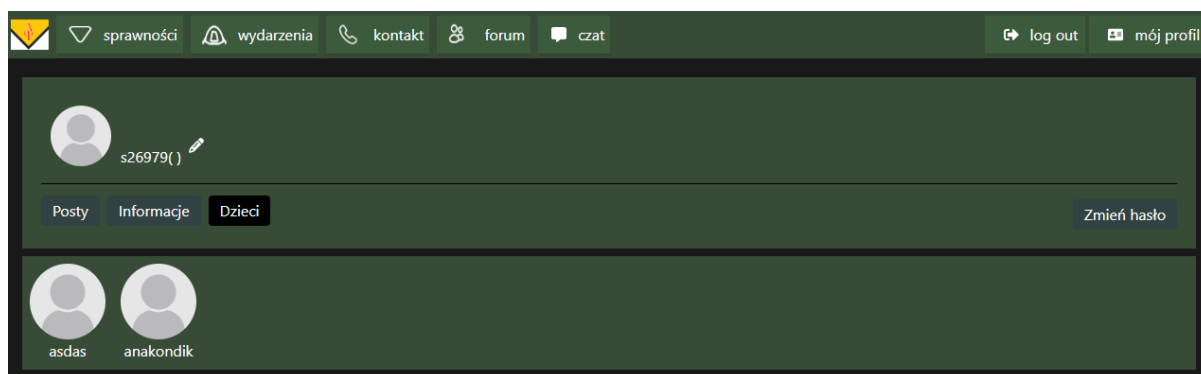


Rysunek 10.12 Strona czatów. Źródło: opracowanie własne

10.4 Strony dostępne dla rodzica

10.4.1 Obserwowanie danych dzieci

Rodzic posiada wszystkie uprawnienia dostępne dla zalogowanego użytkownika, z wyjątkiem możliwości tworzenia postów. Dodatkowo ma dostęp do strony umożliwiającej przeglądanie listy swoich dzieci oraz przechodzenie do ich profili.



Rysunek 10.13 Strona Rodzica. Źródło: opracowanie własne

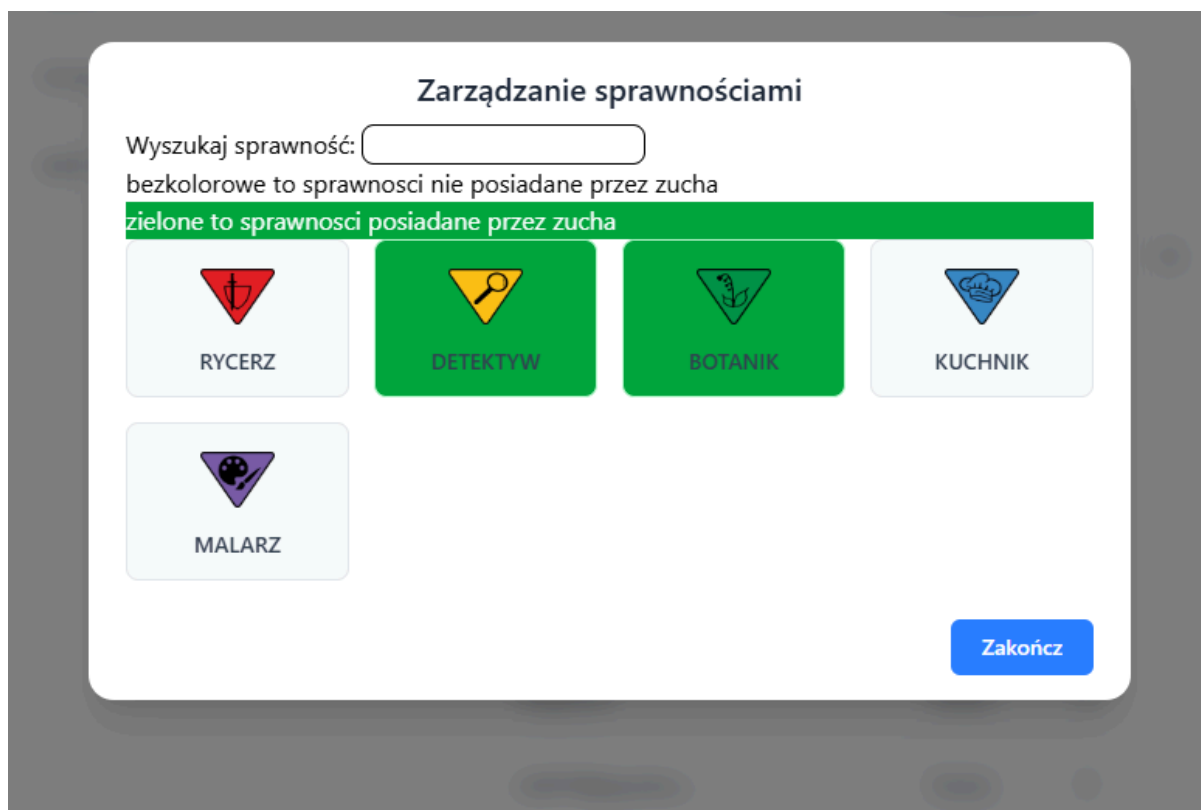
10.5 Strony dostępne dla drużynowego/przybocznego

10.5.1 Strona administracyjna

Drużynowy i przyboczny posiadają wszystkie wcześniej opisane funkcjonalności, a dodatkowo mają dostęp do panelu administracyjnego zawierającego tabelę wszystkich użytkowników. Z poziomu tej tabeli możliwe jest zmienianie typów użytkowników, edycja danych o rodzicach, przydzielenie sprawności do zucha oraz usuwanie użytkowników.

ID▲	Login	Imię	Nazwisko	Email	Numer telefonu	Typ	Rodzice	Akcje
1	s27297@pjwtke...	Artem	Stakhovski	s27297@pjwtke...	1234567	Drużynowy ▼		
2	asdasdsa1	qwerty	q			Przyboczny ▼		
3	asdasddvd	Anrzej	qwe			Przyboczny ▼		
5	asdas	qqwe	qew	s2729@w	123456742	Zuch ▼	54	zarządzaj sprawnościami
9	asdasdsa111			qwe@e.e		Default ▼		
10	asdasdsa13123			s2v@d.2		Default ▼		
11	asdasdsa1x			x@x.x		Default ▼		
12	asdasdsa1e21			21@w.s		Default ▼		
13	asdasdsa1xxxx			vds@vd.w		Rodzic ▼		
54	s26979	Artem		s26979@pjwtke...		Rodzic ▼		

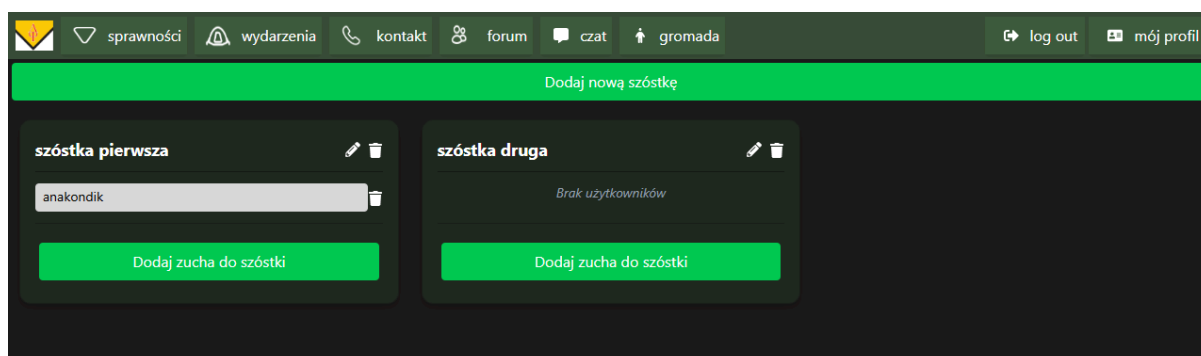
Rysunek 10.14 Tabela użytkowników. Źródło: opracowanie własne



Rysunek 10.15 Zarządzanie sprawnościami. Źródło: opracowanie własne

10.5.2 Strona szóstek

Drużynowy i przyboczni mają również dostęp do strony umożliwiającej organizowanie zuchów w szóstki. Rozwiązanie to pozwala na oznaczanie wszystkich członków danej szóstki poprzez użycie jej nazwy w czatach lub postach, co znacząco usprawnia komunikację wewnątrz gromady.



Rysunek 10.16 Zarządzanie szóstkami. Źródło: opracowanie własne

10.5.3 Edycja i dodawanie danych

Oprócz panelu administracyjnego drużynowy i przyboczny mogą również dodawać nowe sprawności oraz wydarzenia do systemu, a także edytować już istniejące dane.

The screenshot shows a web application interface for adding a new skill. At the top is a dark green navigation bar with icons and labels for 'sprawności' (skills), 'wydarzenia' (events), 'kontakt', 'forum', 'czat', 'gromada', 'log out', and 'mój profil'. The main content area has a dark background. On the left, there is a form titled 'Nazwa sprawności' (Skill name) in a dark box. The form contains three sections: 'Opis:' (Description) with a text input field containing 'Opis dla użytkownika'; 'Wymagania:' (Requirements) with a text input field containing 'Opis wymagań'; and 'Typ sprawności:' (Skill type) with a dropdown menu currently showing 'CZERWONY'. Below these fields is a green button labeled 'Dodaj sprawność'. To the right of the form is a large dark square with a blue border and the text 'Kliknij aby dodać ikonę' (Click to add icon).

Rysunek 10.17 Zarządzanie sprawnościami. Źródło: opracowanie własne

Rozdział 11.

Wkład własny

11.1 Artem Stakhovski wkład pracy

Byłem odpowiedzialny za implementację warstwy frontendowej części aplikacji. Zajmowałem się stworzeniem wszystkich widoków interfejsu użytkownika. Podczas tej pracy tworzyłem stany, które aplikacja musi przechowywać, zdarzenia wywoływane po spełnieniu określonych warunków, komponenty renderowane na stronę oraz robiłem style dla tych komponentów. Odpowiedziałem również za implementację funkcji, wysyłających zapytania do warstwy backendowej, w tym zapytania typu “application/json”, “multipart/form-data”, a także obsługę subskrypcji WebSocketów. Dodatkowo zaimplementowałem routing oparty na strukturze plików.

Rozdziały książki mojego autorstwa:

- 3.5 Analiza SWOT
- 4.2 Kontekst Biznesowy
- 4.3 Kontekst społeczny
- 5.3 Harmonogram pracy
- 5.5 Analiza ryzyka
- 6.3 Wymagania ogólne i dziedzinowe
- 6.4 Wymagania funkcjonalne
- 6.5 Wymagania pozafunkcjonalne
- 7.3 Projekt wizualny systemu
- 8.1 Architektura
- 8.2 Implementacja frontendu
- 10. Prezentacja rozwiązania
- 11.1 Artem Stakhovski wkład pracy
- 12 Podsumowanie

11.2 Paweł Anuszkiewicz wkład pracy

Byłem odpowiedzialny za implementację warstwy backendowej części aplikacji SZYSZKA. Moja praca obejmowała projektowanie oraz rozwój warstwy serwerowej systemu, ze szczególnym uwzględnieniem realizacji logiki biznesowej kluczowych modułów domenowych aplikacji.

Zaprojektowałem oraz zaimplementowałem kluczowe moduły systemu, takie jak: moduł postów, moduł komentarzy, moduł szóstek, moduł wiadomości, moduł wydarzeń, moduł uczestnictwa w wydarzeniach oraz moduły dotyczące sprawności, obejmujące definicję dostępnych sprawności i ewidencję sprawności zdobytych przez zuchów.

Każdy z wymienionych modułów został zaimplementowany w oparciu o architekturę warstwową, obejmującą encje odwzorowujące model danych, repozytoria JPA odpowiedzialne za dostęp do relacyjnej bazy danych, warstwę serwisów realizującą logikę biznesową oraz kontrolery REST umożliwiające komunikację z warstwą frontendową.

Dodatkowo odpowiadałem za przygotowanie testów jednostkowych i integracyjnych, pozwalających na weryfikację poprawności działania poszczególnych modułów oraz ich współpracy z bazą danych. Podczas realizacji prac dbałem o zachowanie czytelnej architektury systemu oraz stosowanie dobrych praktyk programistycznych, co umożliwia dalszy rozwój i utrzymanie aplikacji.

Rozdziały książki mojego autorstwa:

- Streszczenie
- 3.1 Skala zjawiska harcerstwa
- 3.4 Propozycja rozwiązania
- 4.4 Kontekst etyczny
- 6.6 Interfejs z otoczeniem
- 6.7 Wymagania na środowisko docelowe
- 7.2 Baza danych
- 8 Wstęp do rozdziału
- 9 Testowanie
- 11.2 Paweł Anuszkiewicz wkład pracy

11.3 Kacper Demczyna wkład pracy

Byłem odpowiedzialny za implementację backendu aplikacji. Zajmowałem się konfigurowaniem i implementacją mechanizmów bezpieczeństwa z wykorzystaniem Spring Security, w tym obsługę uwierzytelniania i autoryzacji użytkowników, logowanie standardowe oraz za pomocą konta Google oparte na tokenie JWT. Zrealizowałem między innymi kluczowe moduły systemu takie jak moduł użytkownika oraz moduł czatu. Dodatkowo przygotowałem globalny mechanizm obsługi wyjątków (ExceptionHandler) oraz zaprojektowałem(z pomocą pozostałych członków zespołu) strukturę relacyjnej bazy danych. W ramach pracy nad modułami, za które byłem odpowiedzialny tworzyłem odpowiednie encje oraz repozytoria JPA, a także zdefiniowałem enumy określające role użytkowników oraz typy sprawności i wydarzeń. Podczas pracy starałem się zachować zasady czytelnej architektury warstwowej oraz trzymać się dobrych praktyk programowania.

Rozdziały książki mojego autorstwa:

- 1 Wstęp
- 3.2 Rich Picture
- 3.3 Analiza Konkurencji
- 4.1 Kontekst systemowy
- 5.1 Metodyka pracy
- 5.2 Podział ról i zadań
- 5.4 Infrastruktura i sposoby komunikacji
- 6.1 Udziałowcy
- 6.2 Diagram przypadków użycia
- 7.1 Architektura
- 8.3 Implementacja backendu
- 11.3 Kacper Demczyna wkład pracy

Rozdział 12.

Podsumowanie

12.1 Osiągnięte rezultaty

Realizacja projektu SZYSZKA można uznać za sukces, bo zostały spełnione najważniejsze wymagania postawione na początku pracy. Opracowana aplikacja umożliwia na podział użytkowników na role, a potem dostarczenie różnych funkcjonalności zależnie od roli.

Zuchy mogą tworzyć posty, komunikować między sobą i z drużynowym, widzieć nadchodzące wydarzenie oraz zobaczyć wszystkie posiadane przez nich sprawności. Rodzice mogą komunikować się z drużynowym i zobaczyć osiągnięcia swoich dzieci. Drużynowe oraz przyboczne posiadają uprawnienia administracyjne, pozwalające na zarządzanie treściami na stronie.

Realizacja projektu było również bardzo skuteczna dla zespołu projektowego. Pozwoliła na zdobycie praktycznej wiedzy z zakresu projektowania i implementacji aplikacji składających się z warstwy frontendowej, backendowej oraz bazy danych. Szczególnie istotne było otrzymanie wiedzy odnośnie pracy mechanizmu Json Web Token, który jest jednym z mechanizmów wykorzystywanym w aplikacji do uwierzytelnienia użytkowników. Też bardzo interesująca była wiedza odnośnie pracy mechanizmu WebSocket, który pozwala na otrzymanie wiadomości w czasie rzeczywistym bez potrzeby odświeżania strony

12.2 Plany na przyszłość

12.2.1 Przechowywanie zdjęć

Ze względu na koszty związane z wynajmem przestrzeni dyskowej w usługach chmurowych AWS, w aktualnej wersji systemu zdjęcia przechowywane są bezpośrednio na serwerze aplikacji. Takie rozwiązanie pozwala na ograniczenie kosztów na etapie prezentacji i testowania systemu. ale może powodować zapelnienie przestrzeni serwera, wydłużenie czasu odpowiedzi i spowolnienie całej aplikacji. Ze względu na to w przyszłości planujemy przenieść przechowywanie zdjęć do usług chmurowych AWS takich jak Amazon S3..

12.2.2 Publikacja

Obecnie aplikacja działa tylko w środowisku lokalnym, bo jest uruchomiona na localhoscie. W przyszłości planujemy zminimalizować zasoby zajmowane przez aplikację, za pomocą narzędzia Docker, wdrożyć aplikację na serwerze i wynajmować domenę, na której aplikacja będzie dostępna do koniecznego użytkownika.

12.2.3 Rozwój aplikacji

W przyszłości planowany jest dalszy rozwój aplikacji poprzez rozszerzenie jej funkcjonalności. Po pierwsze planujemy wdrożyć mechanizm uczestnictwa w wydarzeniach, który pozwoli na wcześniejsze zapisy na wydarzenia oraz późniejsze określenie przez drużynowego lub przybocznego listę faktycznych uczestników. Dane te będą archiwizowane, co pozwoli użytkownikom na ich późniejsze przeglądanie.

Po drugie planujemy rozwijać mechanikę postów, w taki sposób, żeby można było ukrywanie lub zgłaszanie postów innych użytkowników. Też po stronie postów planujemy dodać możliwość odpowiedzi na komentarze innych, a nie tylko pisanie nowego komentarza na końcu.

Po trzecie planowane jest wdrożenie mechanizmu powiadomień, który umożliwi użytkownikom otrzymywanie informacji o nowych wiadomościach na czatach niezależnie od tego, czy aplikacja jest aktualnie uruchomiona. Rozwiązanie to pozwoli na szybszą i bardziej efektywną komunikację za pośrednictwem aplikacji, co bezpośrednio odpowiada założonym celom systemu.

Rozdział 13.

Spis tabel

Tabela 3.1 Analiza SWOT. Źródło: opracowanie własne.....	17
Tabela 5.2 Analiza ryzyka. Źródło: opracowanie własne.....	39
Tabela 6.1 UOB 01. Źródło: opracowanie własne.....	40
Tabela 6.2 UOB 02. Źródło: opracowanie własne.....	41
Tabela 6.3 UOB 03. Źródło: opracowanie własne.....	41
Tabela 6.4 UOB 04. Źródło: opracowanie własne.....	42
Tabela 6.5 UOP 1. Źródło: opracowanie własne.....	42
Tabela 6.6 UOP 2. Źródło: opracowanie własne.....	43
Tabela 6.7 UOP 3. Źródło: opracowanie własne.....	43
Tabela 6.8 UOP 4. Źródło: opracowanie własne.....	44
Tabela 6.9 Komunikacja z bazą. Źródło: opracowanie własne.....	48
Tabela 6.10 Informację ogólne. Źródło: opracowanie własne.....	48
Tabela 6.11 Dostęp do danych. Źródło: opracowanie własne.....	49
Tabela 6.12 Klasy użytkowników. Źródło: opracowanie własne.....	49
Tabela 6.13 Strona internetowa. Źródło: opracowanie własne.....	50
Tabela 6.14 Strona internetowa. Źródło: opracowanie własne.....	50
Tabela 6.15 Specyficzna struktura. Źródło: opracowanie własne.....	51
Tabela 6.16 Zbiórki, wyjazdy oraz sprawności. Źródło: opracowanie własne.....	51
Tabela 6.17 Logowanie. Źródło: opracowanie własne.....	52
Tabela 6.18 Dostęp do sprawności. Źródło: opracowanie własne.....	53
Tabela 6.19 Strona z chatami. Źródło: opracowanie własne.....	54
Tabela 6.20 Informację ogólne. Źródło: opracowanie własne.....	55
Tabela 6.21 Tworzenie list uczestników do wydarzenia. Źródło: opracowanie własne.....	56
Tabela 6.22 Tworzenie postów. Źródło: opracowanie własne.....	57
Tabela 6.23 Sprawdzenie obecności. Źródło: opracowanie własne.....	58
Tabela 6.24 Dostęp do danych dzieci. Źródło: opracowanie własne.....	59
Tabela 6.25 Edytowanie profilu przez użytkownika. Źródło: opracowanie własne.....	60
Tabela 6.26 Kontrola kont. Źródło: opracowanie własne.....	61
Tabela 6.27 Ukrycie oraz zgłaszanie postów. Źródło: opracowanie własne.....	62
Tabela 6.28 Zarządzanie wydarzeniami. Źródło: opracowanie własne.....	63
Tabela 6.29 Ograniczony dostęp. Źródło: opracowanie własne.....	64
Tabela 6.30 Logowanie. Źródło: opracowanie własne.....	65
Tabela 6.31 Wgląd do danych. Źródło: opracowanie własne.....	66
Tabela 6.32 Dostępność systemu. Źródło: opracowanie własne.....	67
Tabela 6.33 Bezpieczeństwo systemu. Źródło: opracowanie własne.....	68
Tabela 6.34 Monitorowanie treści. Źródło: opracowanie własne.....	68
Tabela 6.35 Przyjazność interfejsu. Źródło: opracowanie własne.....	69

Tabela 6.36 Niezależność od liczby użytkowników. Źródło: opracowanie własne.....	69
Tabela 6.37 Ochrona danych. Źródło: opracowanie własne.....	70
Tabela 6.38 Autoryzacja. Źródło: opracowanie własne.....	70
Tabela 6.39 Usuwanie kont. Źródło: opracowanie własne.....	71
Tabela 6.40 RODO. Źródło: opracowanie własne.....	71
Tabela 6.41 Osoby o ograniczonej sprawności. Źródło: opracowanie własne.....	72
Tabela 6.42 Backupy. Źródło: opracowanie własne.....	72
Tabela 6.43 Skalowalność. Źródło: opracowanie własne.....	73
Tabela 6.44 Import i eksport danych. Źródło: opracowanie własne.....	74
Tabela 6.45 REST API. Źródło: opracowanie własne.....	75
Tabela 6.46 Format danych. Źródło: opracowanie własne.....	76
Tabela 6.47 REST API. Źródło: opracowanie własne.....	77
Tabela 6.48 REST API. Źródło: opracowanie własne.....	78
Tabela 6.49 REST API. Źródło: opracowanie własne.....	80
Tabela 6.50 REST API. Źródło: opracowanie własne.....	81
Tabela 6.51 System operacyjny. Źródło: opracowanie własne.....	82
Tabela 6.52 przeglądarki internetowe. Źródło: opracowanie własne.....	82
Tabela 6.53 przeglądarki internetowe. Źródło: opracowanie własne.....	83

Rozdział 14.

Spis rysunków

Rysunek 3.1 Rich Picture. Źródło: Facebook Groups.....	13
Rysunek 3.2 Konkurencja-Facebook. Źródło: Facebook Groups.....	14
Rysunek 3.3 Konkurencja-ScoutLink. Źródło - https://www.scoutlink.net	15
Rysunek 3.4 Konkurencja-Strona poszczególnej drużyny harcerskiej.....	16
Rysunek 6.1 Diagram przypadków użycia - Zuch. Źródło: opracowanie własne.....	45
Rysunek 6.2 Diagram przypadków użycia - Rodzic. Źródło: opracowanie własne.....	46
Rysunek 6.3 Diagram przypadków użycia - Przyboczny. Źródło: opracowanie własne.....	46
Rysunek 6.4 Diagram przypadków użycia - Drużynowy. Źródło: opracowanie własne.....	47
Rysunek 7.1 Diagram Architektury Źródło: opracowanie własne.....	84
Rysunek 7.2 Diagram Komponentów Źródło: opracowanie własne.....	85
Rysunek 7.3 Projekt bazy danych. Źródło: opracowanie własne.....	88
Rysunek 7.4 Prototyp. Paski nawigacyjne . Źródło: opracowanie własne.....	92
Rysunek 7.5 Prototyp. Strona główna. Źródło: opracowanie własne.....	92
Rysunek 7.6 Prototyp. Strona logowania. Źródło: opracowanie własne.....	93
Rysunek 7.7 Prototyp. Strona wydarzeń. Źródło: opracowanie własne.....	93
Rysunek 7.8 Prototyp. Strona Sprawności. Źródło: opracowanie własne.....	94
Rysunek 7.9 Prototyp. Strona forum. Źródło: opracowanie własne.....	95
Rysunek 7.10 Prototyp. Strona Czatów. Źródło: opracowanie własne.....	96
Rysunek 7.11 Strona profilu. Źródło: opracowanie własne.....	96
Rysunek 8.1 Diagram komponentów. Źródło: opracowanie własne.....	98
Rysunek 8.2 Diagram komponentów. Forum page. Źródło: opracowanie własne.....	98
Rysunek 8.3 Diagram sekwencji. Źródło: opracowanie własne.....	99
Rysunek 8.4 Struktura plików. Źródło: opracowanie własne.....	100
Rysunek 8.5 Package.json. Źródło: opracowanie własne.....	101
Rysunek 8.6 Folder app. Źródło: opracowanie własne.....	103
Rysunek 8.7 Layout. Źródło: opracowanie własne.....	104
Rysunek 8.8 Tailwind CSS. Źródło: opracowanie własne.....	104
Rysunek 8.9 Hooki. Źródło: opracowanie własne.....	105
Rysunek 8.10 Komunikacja z backendem. Źródło: opracowanie własne.....	106
Rysunek 8.11 Pasek nawigacyjny-przyciski. Źródło: opracowanie własne.....	107
Rysunek 8.12 Pasek nawigacyjny-renderowanie. Źródło: opracowanie własne.....	108
Rysunek 8.13 Pasek niezalogowanego użytkownika. Źródło: opracowanie własne.....	109
Rysunek 8.14 Pasek zalogowanego użytkownika. Źródło: opracowanie własne.....	109
Rysunek 8.15. Strona główna. Źródło: opracowanie własne.....	110
Rysunek 8.16. Strona logowania. Źródło: opracowanie własne.....	113
Rysunek 8.17. funkcja login. Źródło: opracowanie własne.....	113
Rysunek 8.18. Logowanie Google. Źródło: opracowanie własne.....	114

Rysunek 8.19. React Virtuoso. Źródło: opracowanie własne.....	115
Rysunek 8.20. Okno Modalne. Źródło: opracowanie własne.....	115
Rysunek 8.21. WebSocket. Źródło: opracowanie własne.....	116
Rysunek 8.22. Sortowanie wydarzeń. Źródło: opracowanie własne.....	117
Rysunek 8.23. Wyświetlanie wydarzeń. Źródło: opracowanie własne.....	118
Rysunek 8.24. Komponent paginacji. Źródło: opracowanie własne.....	119
Rysunek 8.25. Rzuty ekrany wyglądu w wersji mobilnej. Źródło: opracowanie własne.....	120
Rysunek 8.26.Struktura projektu na GitHub. Źródło: opracowanie własne.....	122
Rysunek 8.27.Struktura katalogu Entities. Źródło: opracowanie własne.....	123
Rysunek 8.28.Struktura katalogu Repositories. Źródło: opracowanie własne.....	124
Rysunek 8.29.Struktura katalogu DTOs. Źródło: opracowanie własne.....	125
Rysunek 8.30.Struktura katalogu Mappers. Źródło: opracowanie własne.....	126
Rysunek 8.31.Struktura katalogu Services. Źródło: opracowanie własne.....	127
Rysunek 8.32.Struktura katalogu Controllers Źródło: opracowanie własne.....	128
Rysunek 8.33.Struktura katalogu Security. Źródło: opracowanie własne.....	128
Rysunek 8.34.Struktura katalogu Exceptions. Źródło: opracowanie własne.....	129
Rysunek 8.35.Fragment kodu Encji Użytkownik. Źródło: opracowanie własne.....	130
Rysunek 8.36. Użytkownik Repository. Źródło: opracowanie własne.....	131
Rysunek 8.37. Enum TypUzytkownika. Źródło: opracowanie własne.....	131
Rysunek 8.38. UzytkownikDto. Źródło: opracowanie własne.....	132
Rysunek 8.39. ChangeUserTypeRequest. Źródło: opracowanie własne.....	132
Rysunek 8.40. UzytkownikMapper. Źródło: opracowanie własne.....	133
Rysunek 8.41. UzytkownikService. Źródło: opracowanie własne.....	134
Rysunek 8.42. UzytkownikService. Źródło: opracowanie własne.....	134
Rysunek 8.43. UzytkownikService. Źródło: opracowanie własne.....	135
Rysunek 8.44. UzytkownikService. Źródło: opracowanie własne.....	136
Rysunek 8.45. Przepływ uwierzytelniania i autoryzacji. Źródło: opracowanie własne.....	137
Rysunek 8.46. Diagram sekwencji logowania. Źródło: opracowanie własne.....	137
Rysunek 8.47. AuthService metoda register. Źródło: opracowanie własne.....	138
Rysunek 8.48. UzytkownikService metoda login. Źródło: opracowanie własne.....	138
Rysunek 8.49. JwtService fragment kodu. Źródło: opracowanie własne.....	139
Rysunek 8.50. AuthController funkcja login. Źródło: opracowanie własne.....	140
Rysunek 8.51. Diagram Czynnosci Logowanie Google . Źródło: opracowanie własne.....	141
Rysunek 8.52. GoogleAuthRequest. Źródło: opracowanie własne.....	141
Rysunek 8.53. GoogleUserData. Źródło: opracowanie własne.....	142
Rysunek 8.54. Enum TypUzytkownika. Źródło: opracowanie własne.....	142
Rysunek 8.55. CustomUserDetailsService . Źródło: opracowanie własne.....	143
Rysunek 8.56. Diagram Przypadków użycia Wydarzenie . Źródło: opracowanie własne.....	144
Rysunek 8.57. Diagram sekwencji tworzenia czatów . Źródło: opracowanie własne.....	145
Rysunek 8.58. Diagram sekwencji wysyłania wiadomości . Źródło: opracowanie własne...	145
Rysunek 10.1 Karuzela zdjęć. Źródło: opracowanie własne.....	149

Rysunek 10.2 Opis gromady. Źródło: opracowanie własne.....	150
Rysunek 10.3 Forma logowania. Źródło: opracowanie własne.....	150
Rysunek 10.4 Strona rejestracji. Źródło: opracowanie własne.....	151
Rysunek 10.5 Walidacja danych. Źródło: opracowanie własne.....	152
Rysunek 10.6 Strona sprawności. Źródło: opracowanie własne.....	153
Rysunek 10.7 Jedna sprawność. Źródło: opracowanie własne.....	153
Rysunek 10.8 Strona wydarzeń. Źródło: opracowanie własne.....	154
Rysunek 10.9 Strona profilu. Źródło: opracowanie własne.....	155
Rysunek 10.10 Strona postów. Źródło: opracowanie własne.....	155
Rysunek 10.11 Nowy post. Źródło: opracowanie własne.....	156
Rysunek 10.12 Strona czatów. Źródło: opracowanie własne.....	157
Rysunek 10.13 Strona Rodzica. Źródło: opracowanie własne.....	157
Rysunek 10.14 Tabela użytkowników. Źródło: opracowanie własne.....	158
Rysunek 10.15 Zarządzanie sprawnościami. Źródło: opracowanie własne.....	159
Rysunek 10.16 Zarządzanie szóstkami. Źródło: opracowanie własne.....	159
Rysunek 10.17 Zarządzanie sprawnościami. Źródło: opracowanie własne.....	160

Rozdział 15.

Załączniki

Płyta CD z następującą zawartością:

- pliki projektowe – pliki składające się na całość projektu
 - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
 - źródło pracy inżynierskiej.
- Anuszkiewicz_Demczyna_Stakhovski_praca_inzynerska – katalog zawierający pliki PDF i DOCX z pracą inżynierską

Bibliografia

1. DevelopmentAid.org, profil organizacji: Związek Harcerstwa Rzeczypospolitej:
<https://www.developmentaid.org/organizations/view/348121/zwiazek-harcerstwa-rzeczypospolitej-scouting-association-of-the-republic-zhr>
2. Stowarzyszenie Klon/Jawor – *Kondycja organizacji pozarządowych w Polsce 2022*:
https://www.klon.org.pl/pliki/raporty/Raport_kondycja_ngo_2022.pdf
3. Kolping International – *Model of communication and promotion in youth work*:
https://www.kolping.net/wp-content/uploads/2018/10/Model_of_communication_and_promotion_in_youth_work.pdf?utm_source=chatgpt.com
4. Główny Urząd Statystyczny, Kultura fizyczna i aktywność młodzieży w Polsce 2023, Warszawa 2024.
(<https://stat.gov.pl/obszary-tematyczne/kultura-turystyka-sport/sport/kultura-fizyczna-w-polsce-w-2023-roku%2C13%2C7.html>) dostęp 17.11.2025
5. Związek Harcerstwa Rzeczypospolitej, Statut i struktura organizacyjna ZHR, dostęp: <https://zhr.pl>, dostęp: 30.10.2025.
6. Model kaskadowy – Wikipedia, wolna encyklopedia
(https://pl.wikipedia.org/wiki/Model_kaskadowy) dostęp: 23.10.2025.
7. [Technika MoSCoW – Encyklopedia Zarządzania](#)
(https://mfiles.pl/pl/index.php/Technika_MoSCoW) dostęp: 17.01.2026
8. [Next.js by Vercel - The React Framework](#) (<https://nextjs.org/docs>) dostęp: 24.12.2025
9. [App Router - Next.js](#) (<https://nextjs.org/docs/app>) dostęp: 25.06.2025
10. [Adding custom styles - Core concepts - Tailwind CSS](#)(<https://tailwindcss.com/docs/>),
dostęp: 24.12.2025.
11. Formik (<https://formik.org/docs>) dostęp: 24.12.2025.

12. Jacoco (Java Code Coverage) – Narzędzie do mierzenia pokrycia kodu testami.
Dokumentacja techniczna na stronie:(<https://www.jacoco.org/jacoco/trunk/doc/>)
dostęp: 18.01.2026.
13. Spring Boot – Framework języka Java. Dokumentacja pod adresem:
(<https://docs.spring.io/spring-boot/docs/current/reference/html/>) dostęp: 18.01.2026.